

Deep Learning for Computer Vision

Lecture 2: Overview and Recap Deep Learning

Constantin Pape

Institute of Computer Science, Georg August Universität Göttingen



@cppape.bsky.social

<https://user.informatik.uni-goettingen.de/~pape41/>

Agenda for this week

- Training neural networks: loss functions and backpropagation
- Stochastic gradient descent and other optimizers
- Regularization
- Babysitting training neural networks
- Architecture choices
 - Activation functions
 - Depth vs. width
 - Types of architectures

Agenda for this week

- Training neural networks: loss functions and backpropagation
- Stochastic gradient descent and other optimizers
- Regularization
- Babysitting training neural networks
- Architecture choices
 - Activation functions
 - Depth vs. width
 - Types of architectures

Recap of the part on deep learning / artificial neural networks that was covered in the ML Lecture (by Alexander Ecker)

-> We will go through a lot of material quite fast today. If you haven't heard any of this before review this lecture and study the recap session.

Organization

Tutorials

- First version of tutorial assignments is available on Stud.IP:



Please check if there is something wrong with your group assignment (wrong group members or timeslot you did not have in your preferences etc.) and let us know if it is.

We can only account for cases where you cannot make it to your tutorial due to schedule problems and not due to other preferences.

I don't have a tutorial slot / group yet

You can still form a group of three and sign up in the same form as before:

- <https://forms.gle/wKyjNFi2JyAiBWGL7>

If you can't find a group: fill out this form with your tutorial preferences:

- <https://forms.gle/936GHafic6C7urHt8>
 - We will then create new groups and assign you to tutorials based on this.
-
- Fill out one of the two forms **before Thursday (06.11.2025) 14:00**.
 - We will inform you about the final group and tutorial assignment on Thursday afternoon.

Tutorial slots & Chatrooms

- Tutorial 1: Tuesday 16-18
 - Tutor: Carolin Teuer; <https://matrix.to/#/%23DLForCV-Tutorial-Tue-16-18:uni-goettingen.de>
- Tutorial 2: Wednesday 14-16
 - Tutor: Sarah Muth; <https://matrix.to/#/%23DLForCV-Tutorial-Wed-14-16:uni-goettingen.de>
- Tutorial 3: Wednesday 16-18
 - Tutor: Oleg Bakumenko; https://matrix.to/#/%23dl_cv_2526_wed_2:umg.eu
- Tutorial 4: Thursday 10-12
 - Tutor: Marei Freitag; <https://matrix.to/#/%23DLForCV-Tutorial-Thu-10-12:uni-goettingen.de>
- Tutorial 5: Thursday 14-16 (Zoom)
 - Tutor: Azhar Akhmetova; <https://matrix.to/#/%23DLForCV-Tutorial-Thu-14-16:uni-goettingen.de>
- Tutorial 6: Friday 10-12
 - Tutor: Benjamin Eckhardt; <https://matrix.to/#/%23DLForCV-Tutorial-Fri-10-12:uni-goettingen.de>
- Tutorial 7: Friday 14-16 (Zoom)
 - Tutor: Luca Freckmann; <https://matrix.to/#/%23DLForCV-Tutorial-Fri-14-16:uni-goettingen.de>

Submitting the exercises

- The first exercise needs to be submitted by Nov. 11th, 10:00 via Stud.IP.
- Upload it in the folder “Submission of Homework 1 -> Tutorial X”
(X = ID of your tutorial)
- Upload only one solution per group



/ Submission of Homework 1

Den zeitgesteuerten Ordner sehen Teilnehmende in einem von lehrenden Personen bestimmten Zeitraum. Ob Teilnehmende die Ordnerinhalte herunterladen und/oder hochladen können, kann eingestellt werden.

Sichtbar von 28.10.2025 00:00 bis 11.11.2025 10:00

Dieser Ordner ist ein Hausaufgabenordner. Es können nur Dateien eingestellt werden.

☐	Typ	Name ^	Größe	Autor/-in
☐		Tutorial 1		Archit, Anwai
☐		Tutorial 2		Archit, Anwai
☐		Tutorial 3		Archit, Anwai
☐		Tutorial 4		Archit, Anwai
☐		Tutorial 5		Archit, Anwai
☐		Tutorial 6		Archit, Anwai
☐		Tutorial 7		Archit, Anwai

A note on the exercises

Training a single model may take up to 30 min to train, even with a GPU.

Keep in mind:

- Start early!
- Once you got the basics up and running, you can distribute tasks among the group, then collect and discuss results

Tutorials & Exercise Presentation

- Tutors will assign you timeslots for presenting.
 - Each timeslot will be 2-3 groups, so you may need to wait a bit.
 - Be on time! (Otherwise you may get a failed attempt!)
- We calculate with 15 minutes per exercise presentation, which is very short...
 - Divide the exercise in 3 parts, 3 members of the groups present these parts.
 - Groups of four: rotate who presents between the weeks.
 - **If you don't also present with your group you will not get the exam qualification, even if your group passes enough exercises.**
 - Prepare your part well so that you can present it in 5 minutes!
 - You go through the notebook with the results to present your solution.
 - Focus on high-level results and what you learned / found surprising. Answer the questions.
 - Don't discuss the implementation in detail, no time.
 - **If you're not prepared (for example have not decided who presents what part) your tutor can fail you!**

Tutorials & Exercise Presentation

- If you fail an exercise: you have **one more attempt** to present during office hours. You can choose when, but need to tell your tutor in advance.
 - Your tutors will share with you more details on their office hours.
 - Your tutor will give you feedback on what you need to improve to pass the exercise.
- Use the chat and the office hours if you have questions for which there is not enough time during the exercises.

Tutorials & Exercise Presentation

- If you fail an exercise: you have **one more attempt** to present during office hours. You can choose when, but need to tell your tutor in advance.
 - Your tutors will share with you more details on their office hours.
 - Your tutor will give you feedback on what you need to improve to pass the exercise.
- Use the chat and the office hours if you have questions for which there is not enough time during the exercises.

First exercise:

- We will be more lenient since group formation took longer for some of you.
- If you did not find a group yet: work on the exercise already, combine / compare your solutions once you have a group.

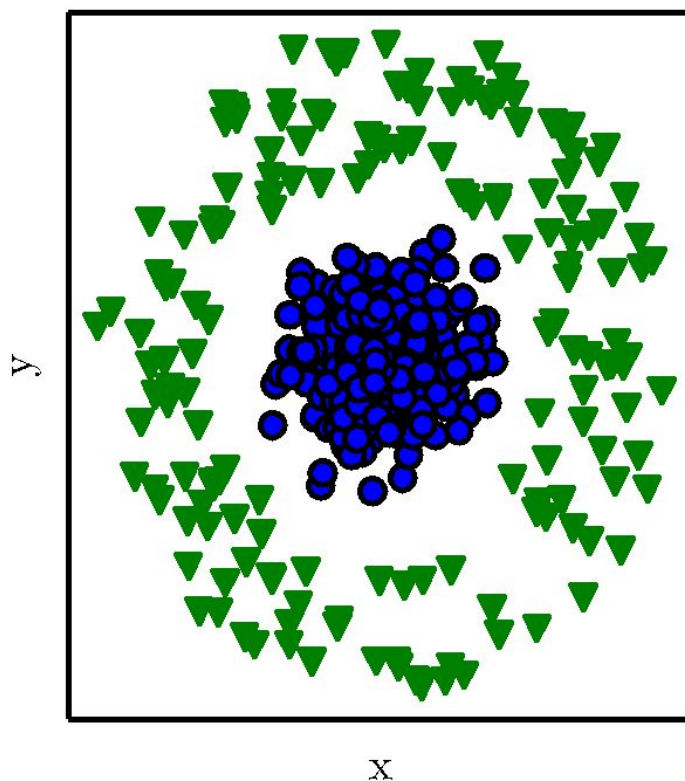
Questions?

Why deep learning?

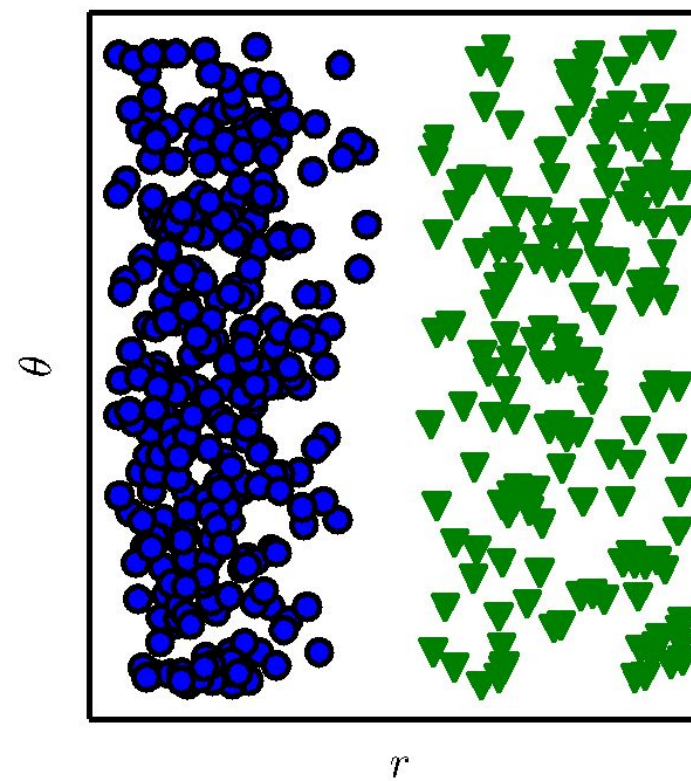
Representations matter

Recap Session 2!

Cartesian coordinates

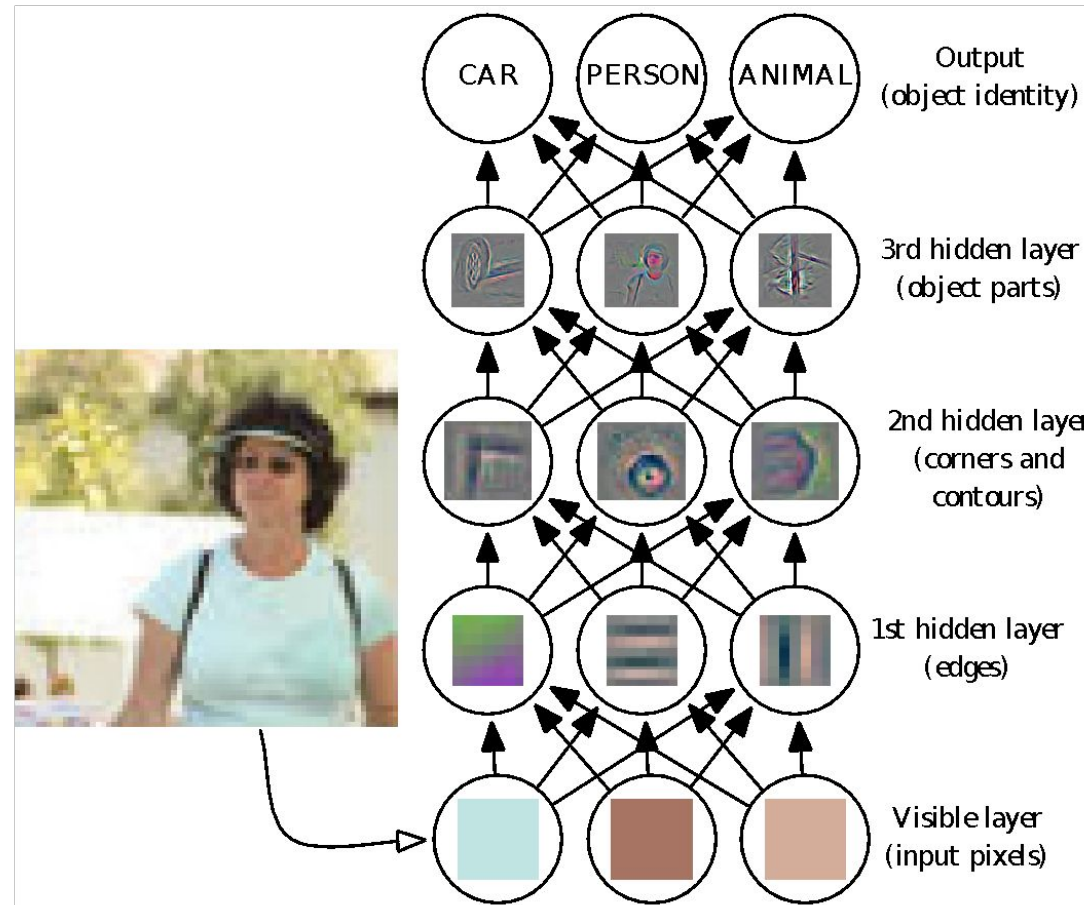


Polar coordinates



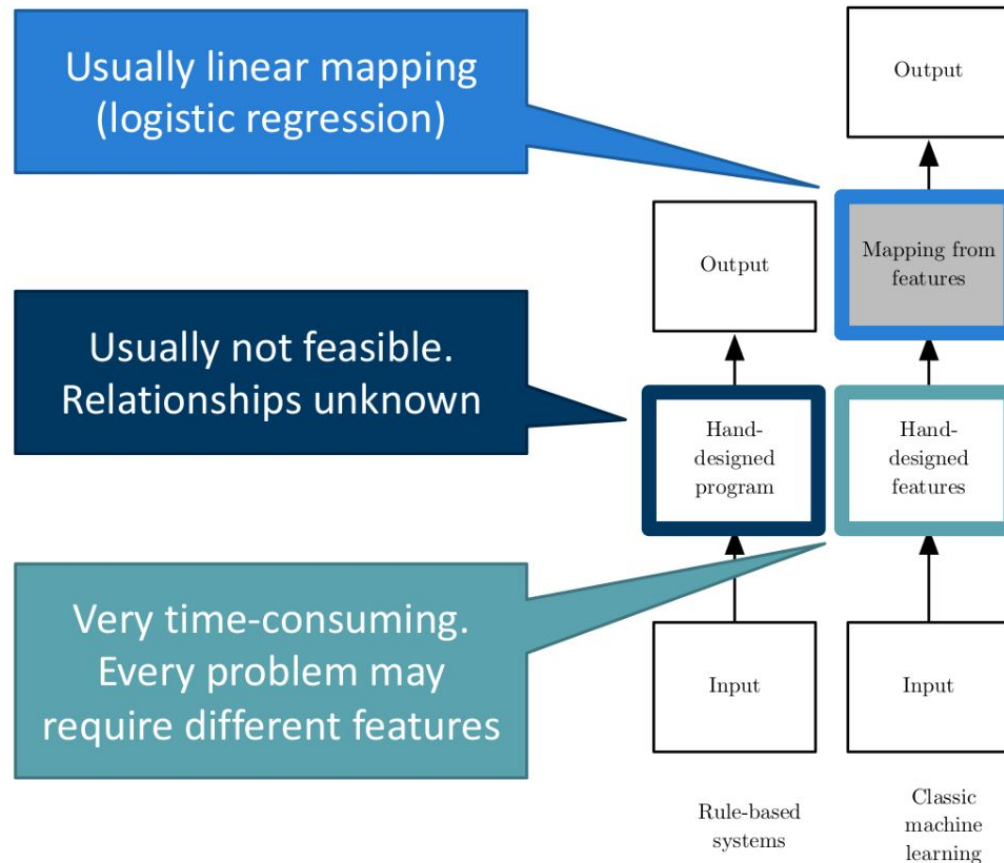
Depth: repeated composition

Relationship between raw data (here: pixels) and class label is very complex



Learning everything

Recap Session 2!

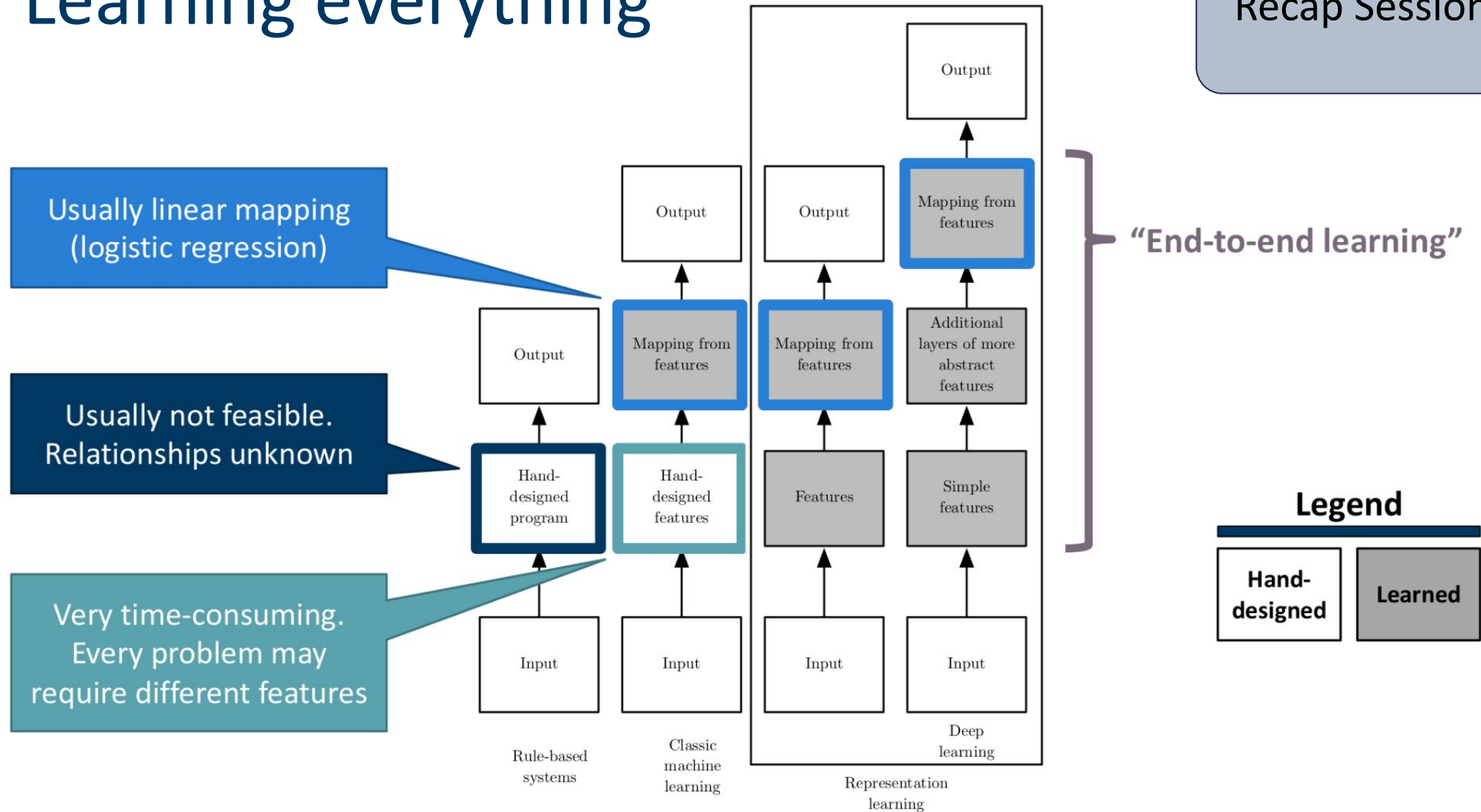


Legend

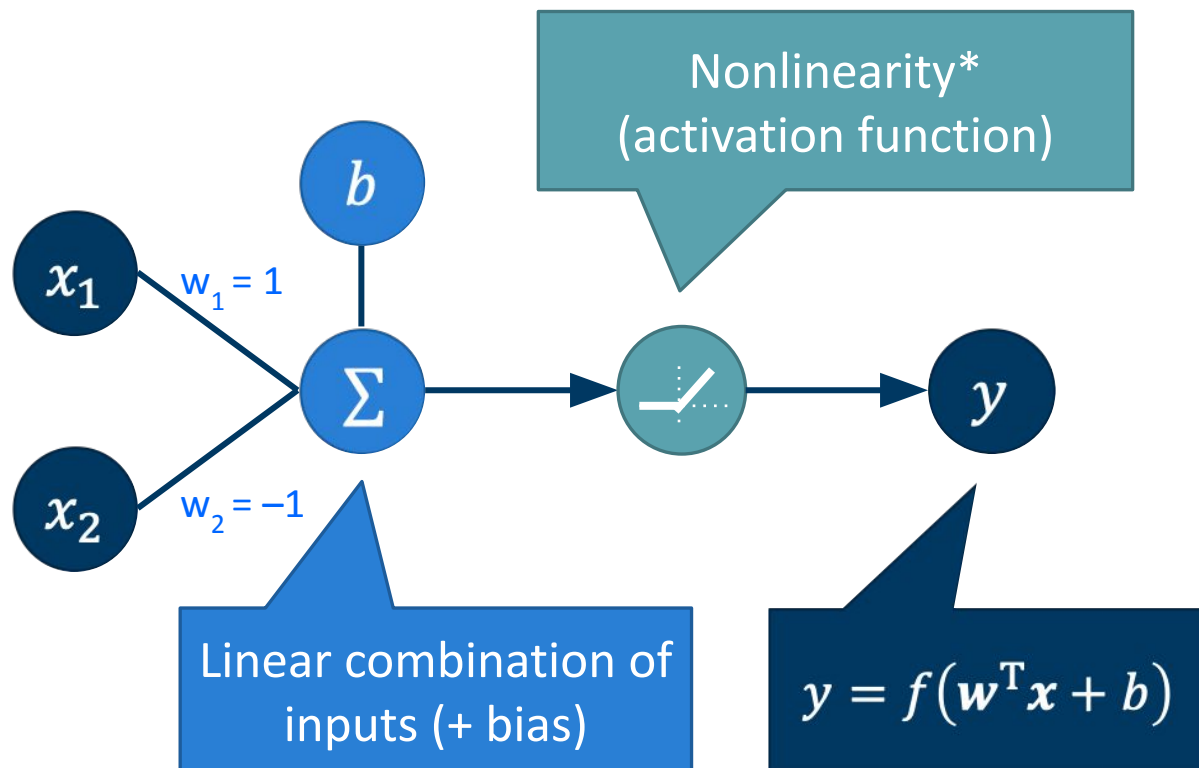


Learning everything

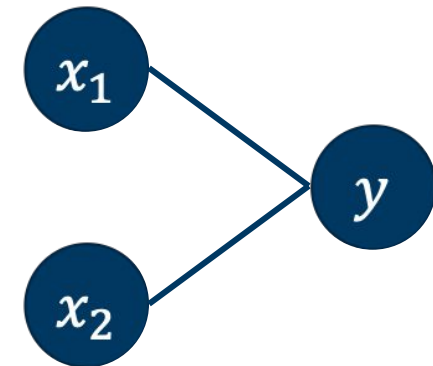
Recap Session 2!



The Perceptron



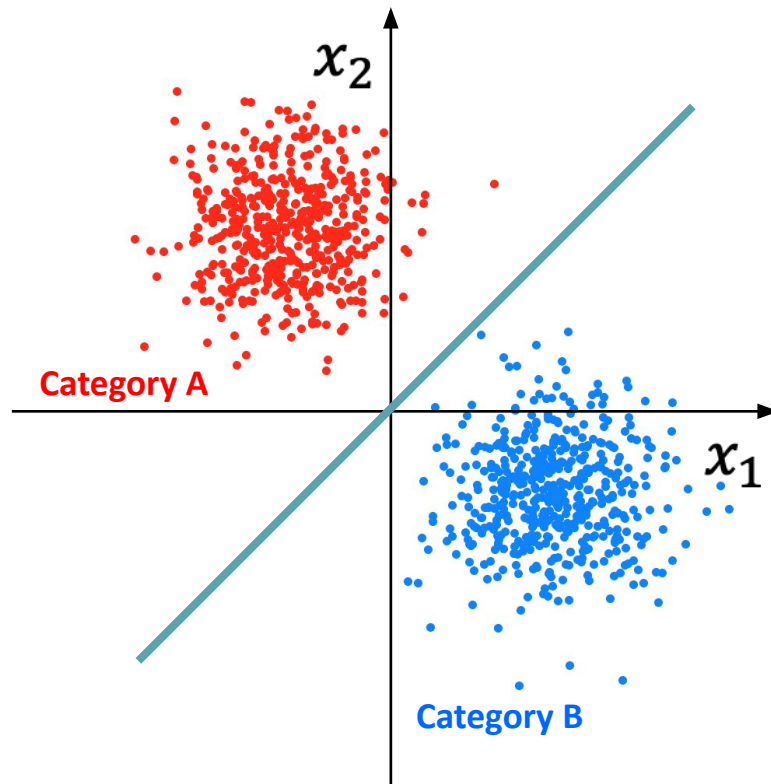
Simplified representation



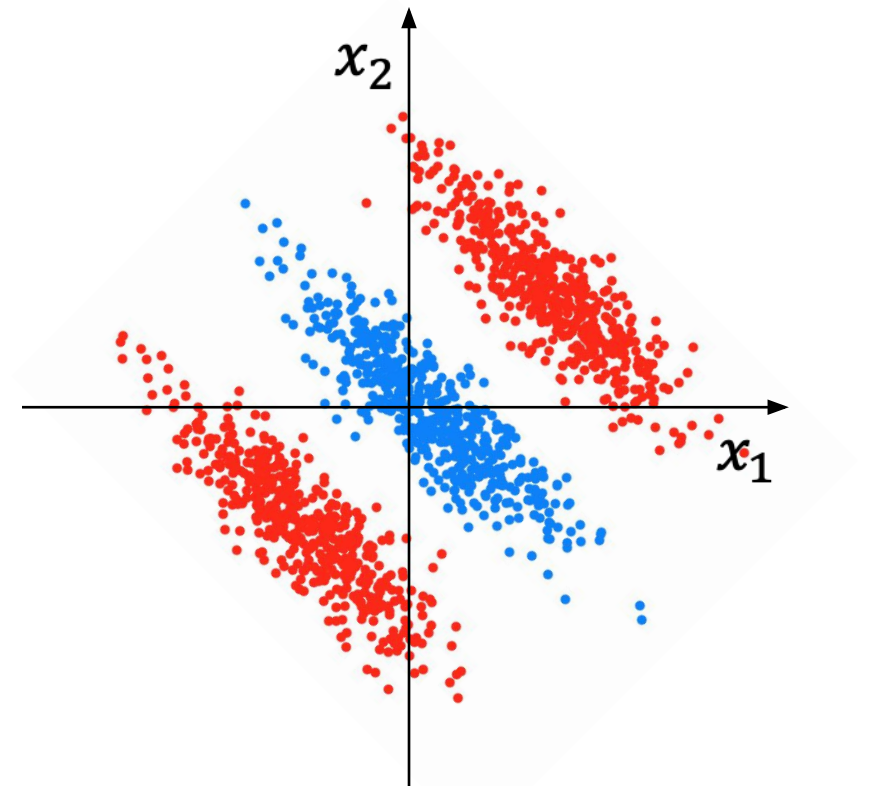
Linear combination and nonlinear
activation function are implicit in
this diagram

Can only solve linearly separable problems

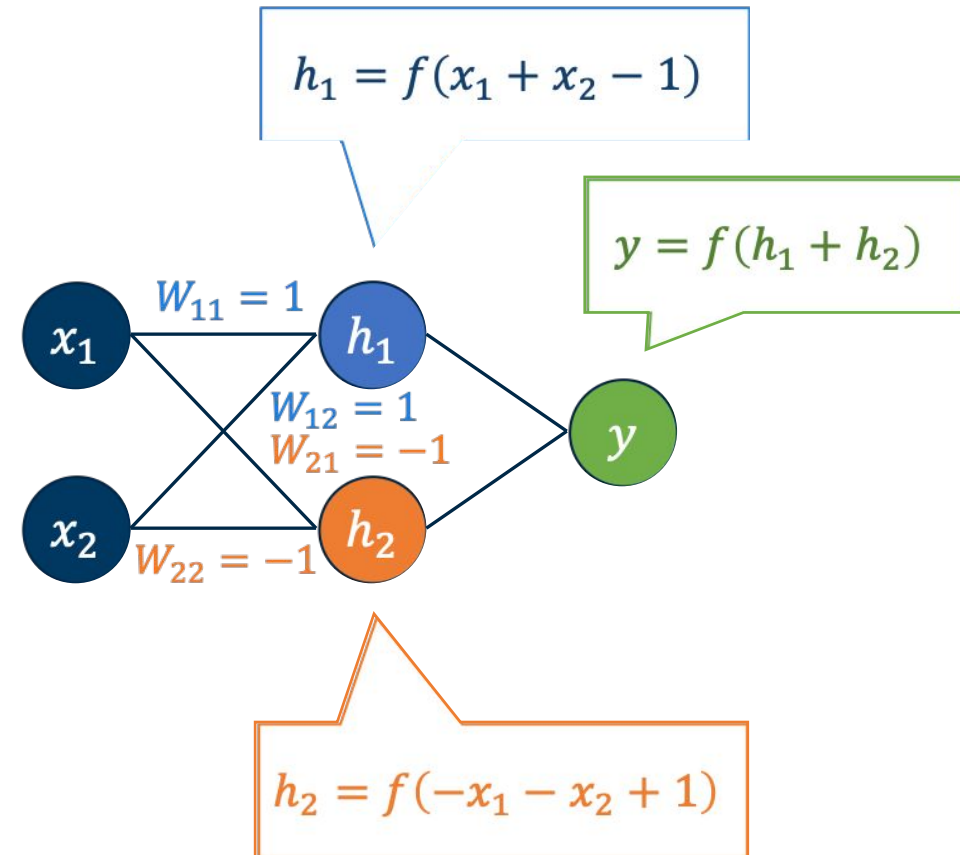
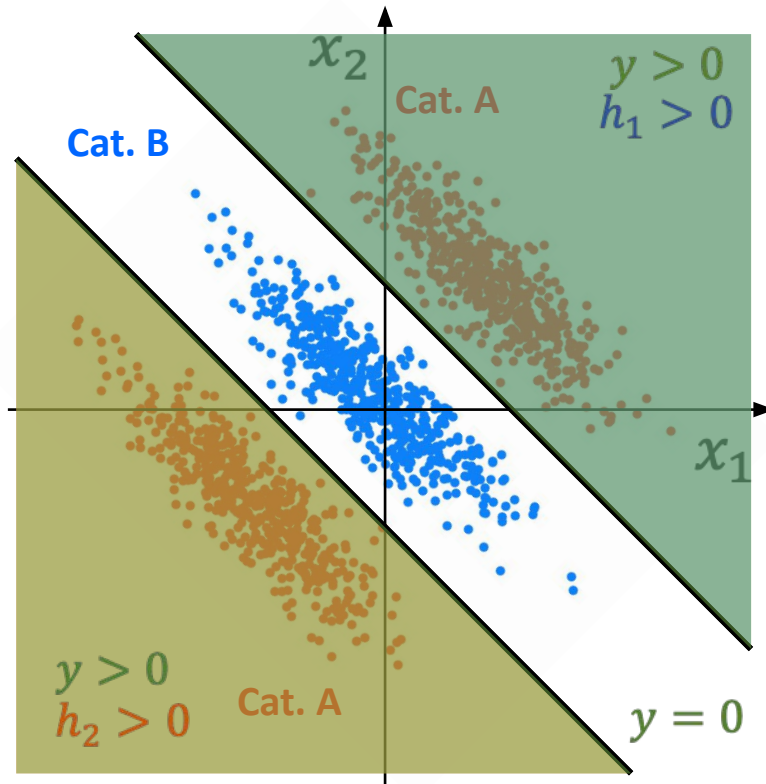
Linearly separable



Not linearly separable

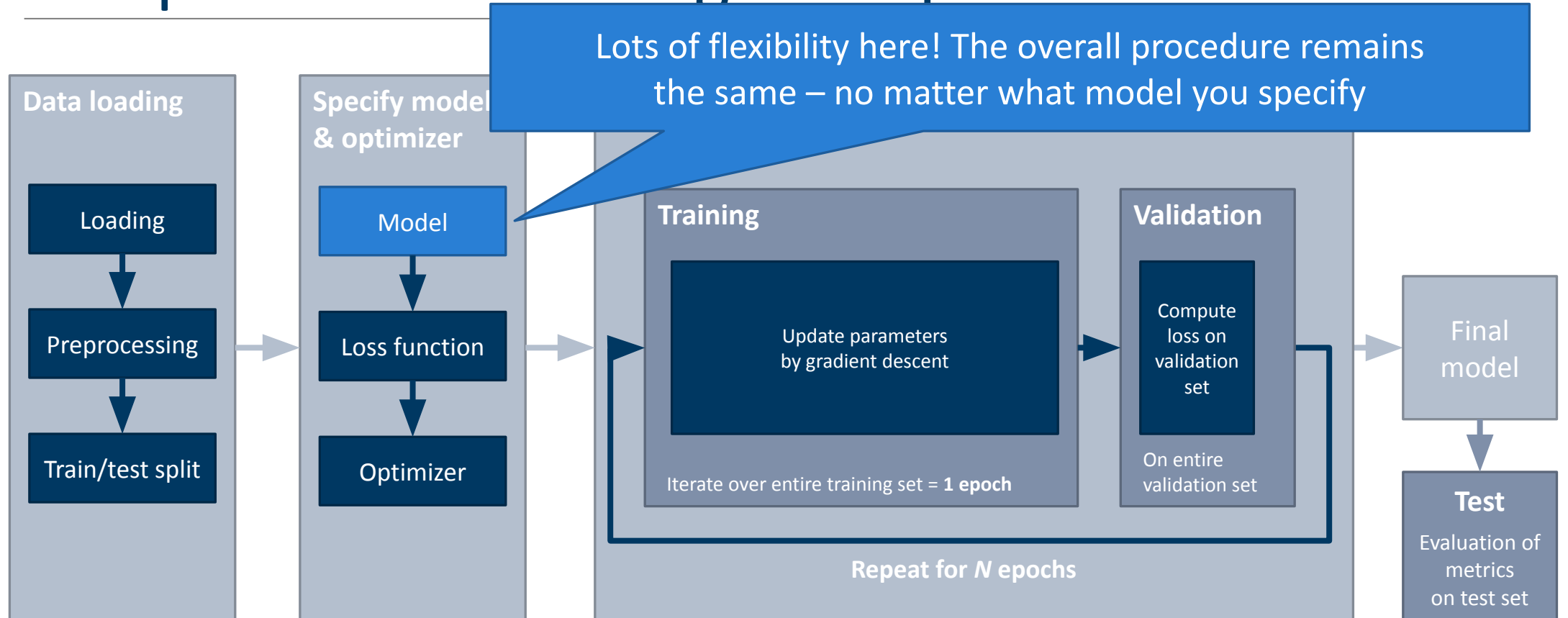


MLP: Maps non-linearly sep. problems to separable ones

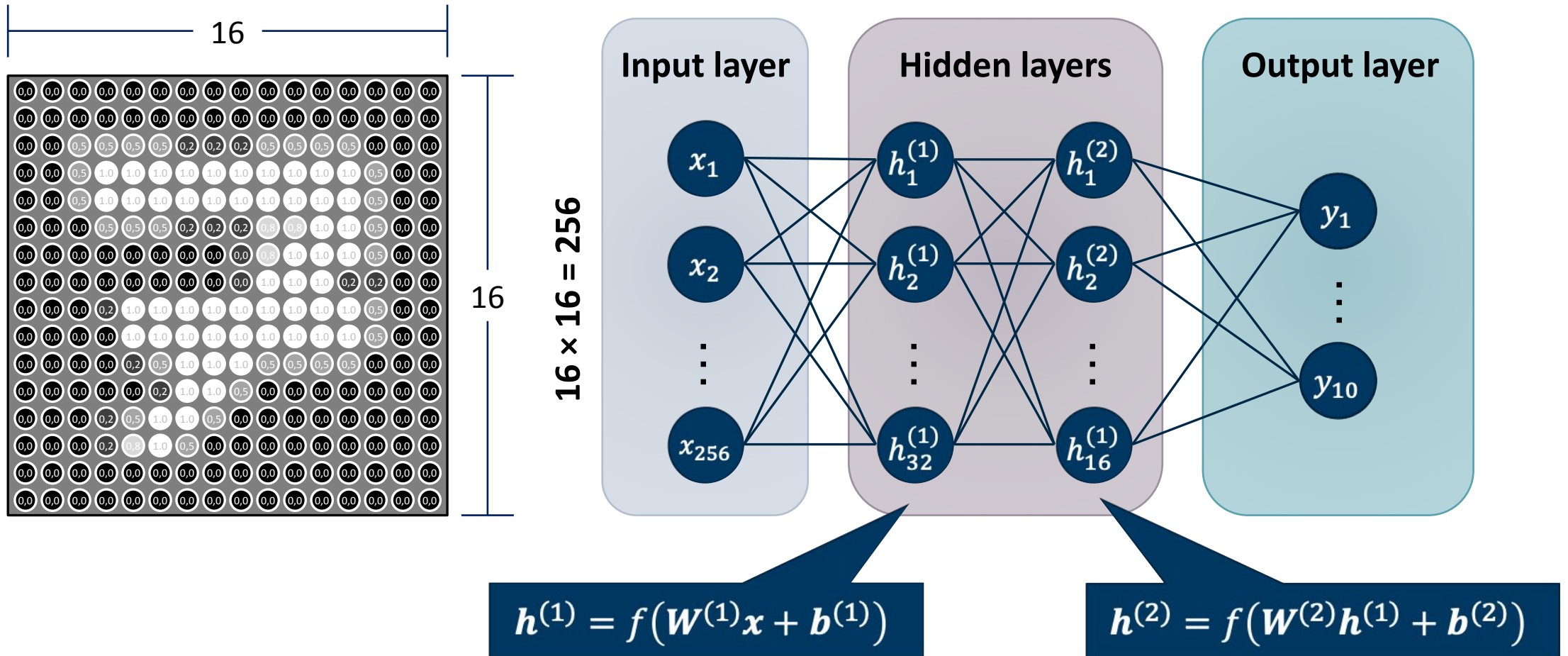


Training Neural Networks

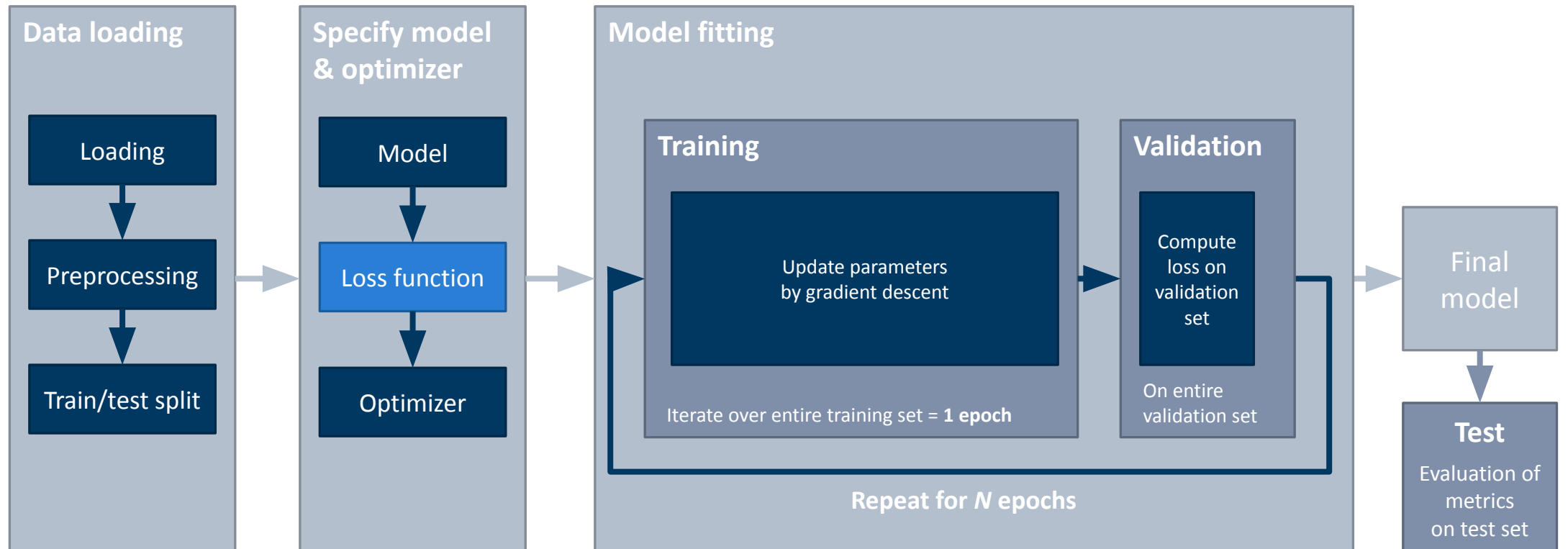
The process of training a deep neural network



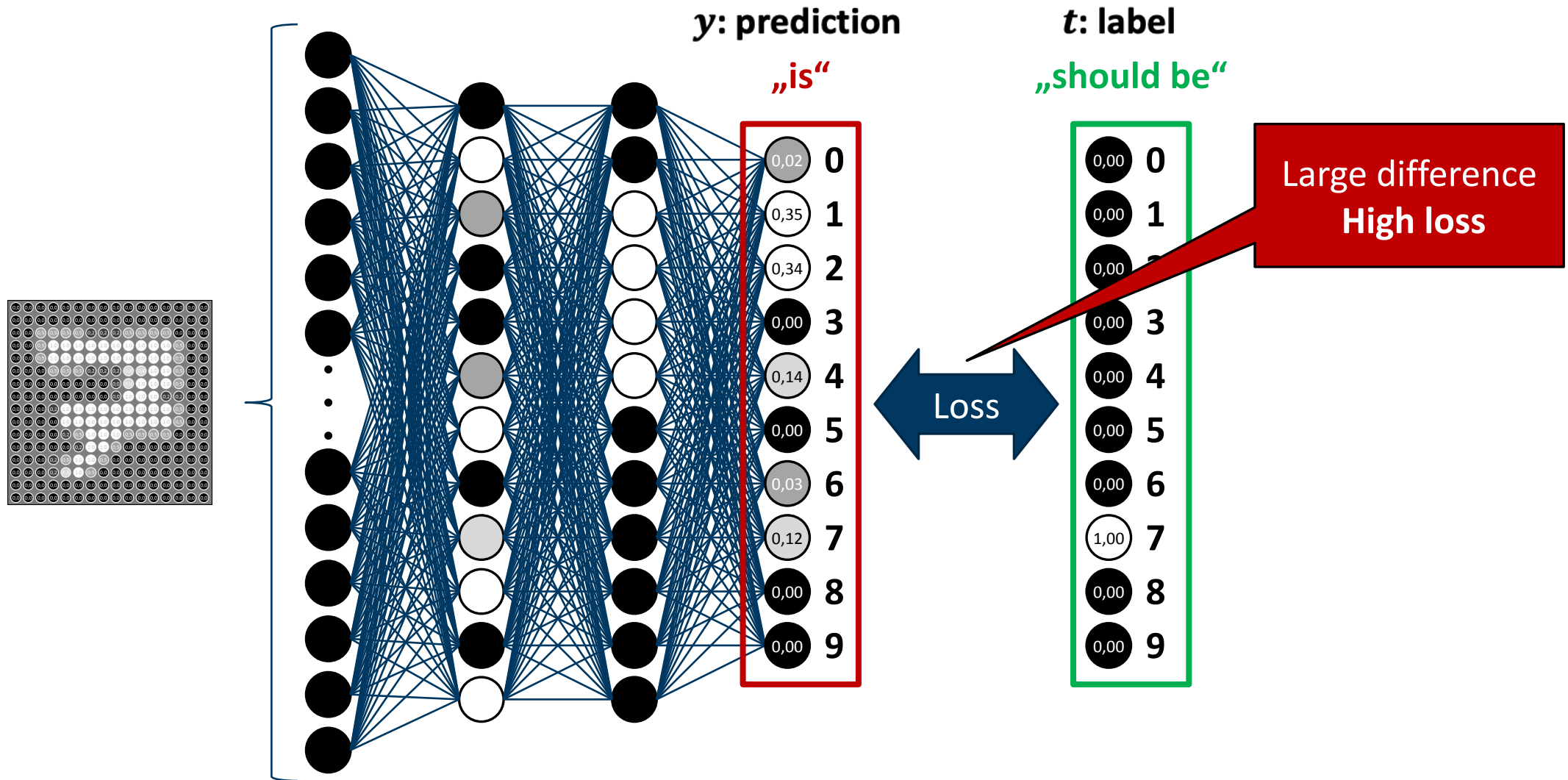
Multi-layer perceptron (MLP) for classifying handwritten digits



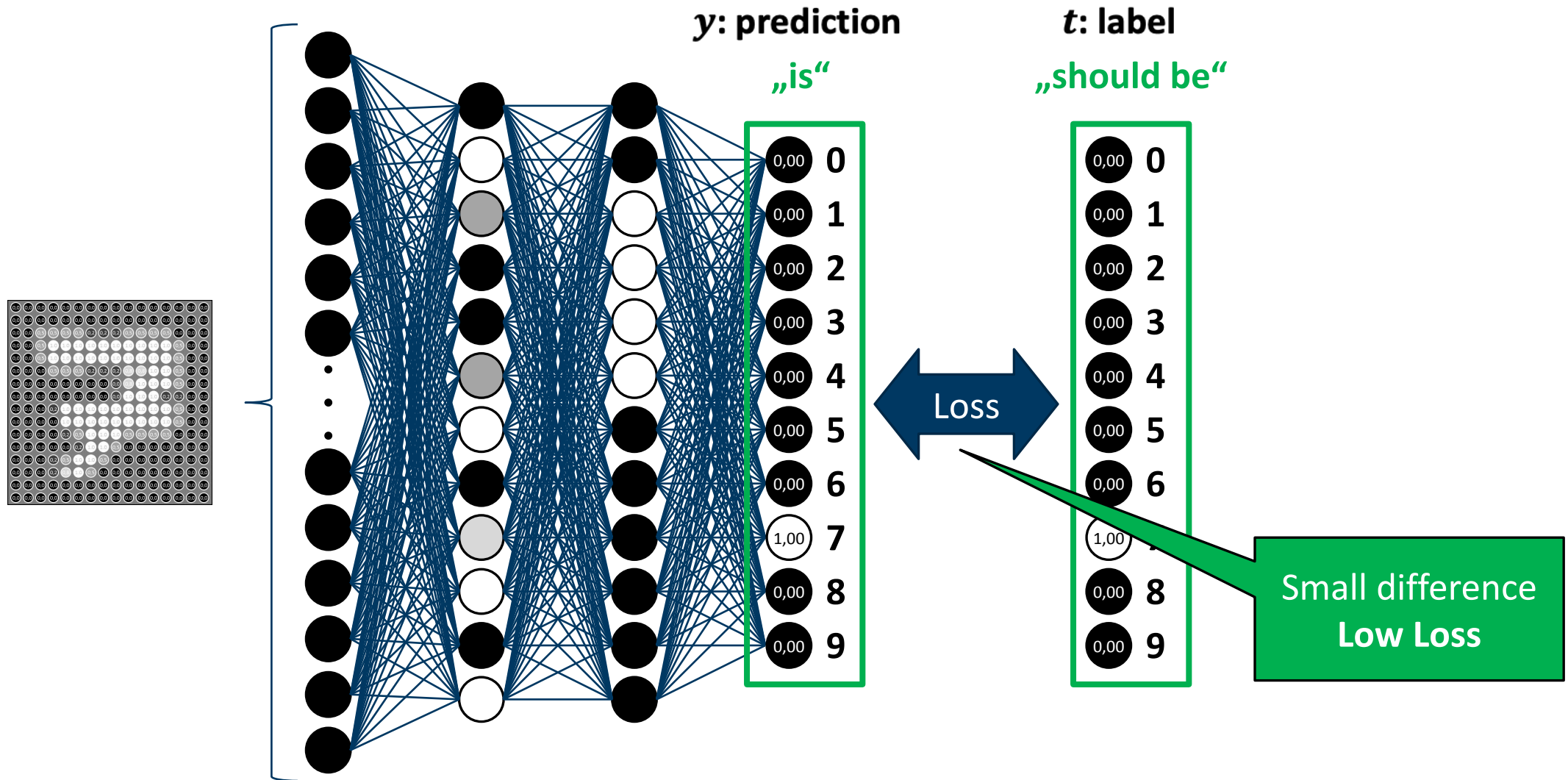
The process of training a deep neural network



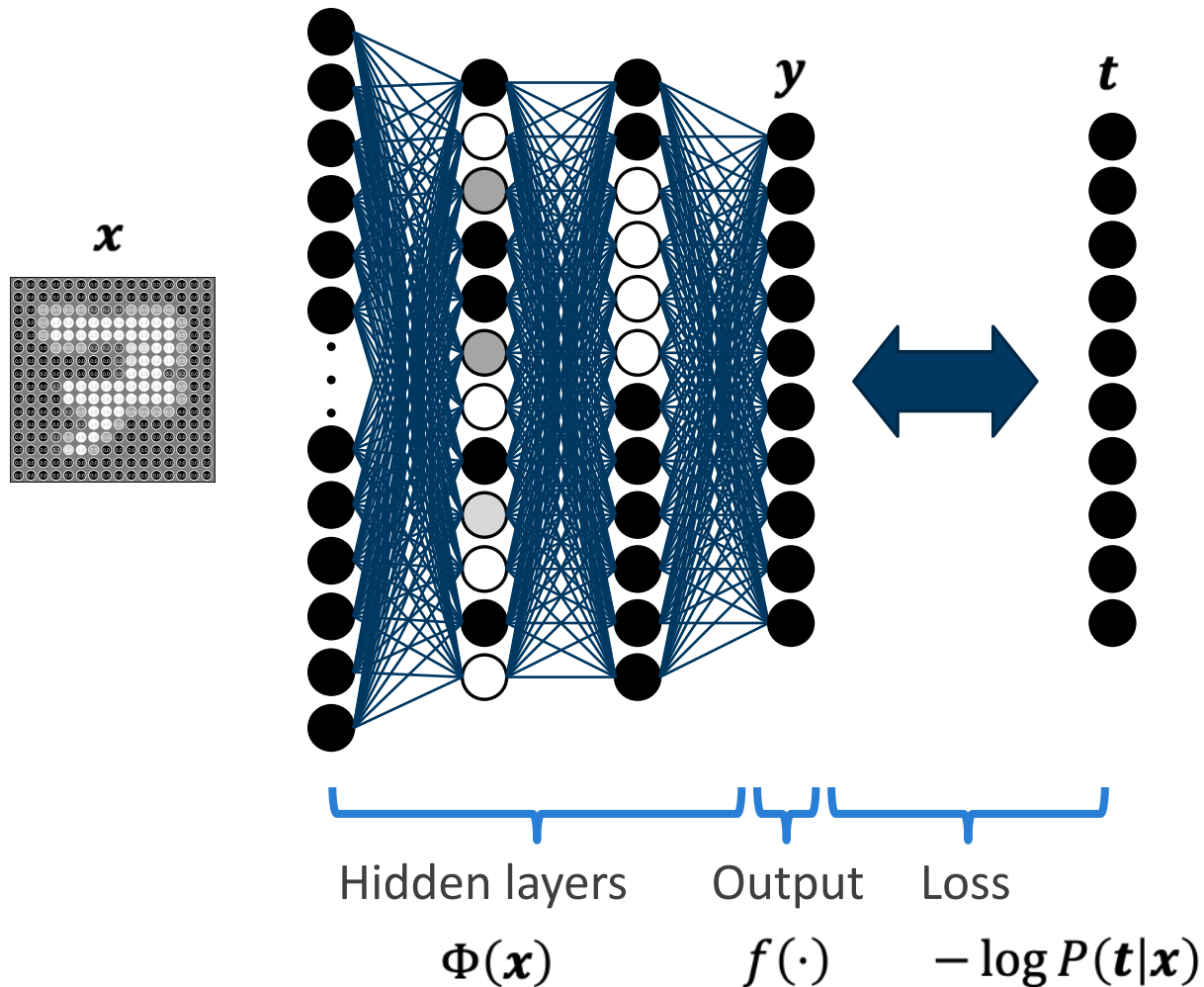
Loss function measures model performance



Loss function measures model performance



Output layers and loss functions



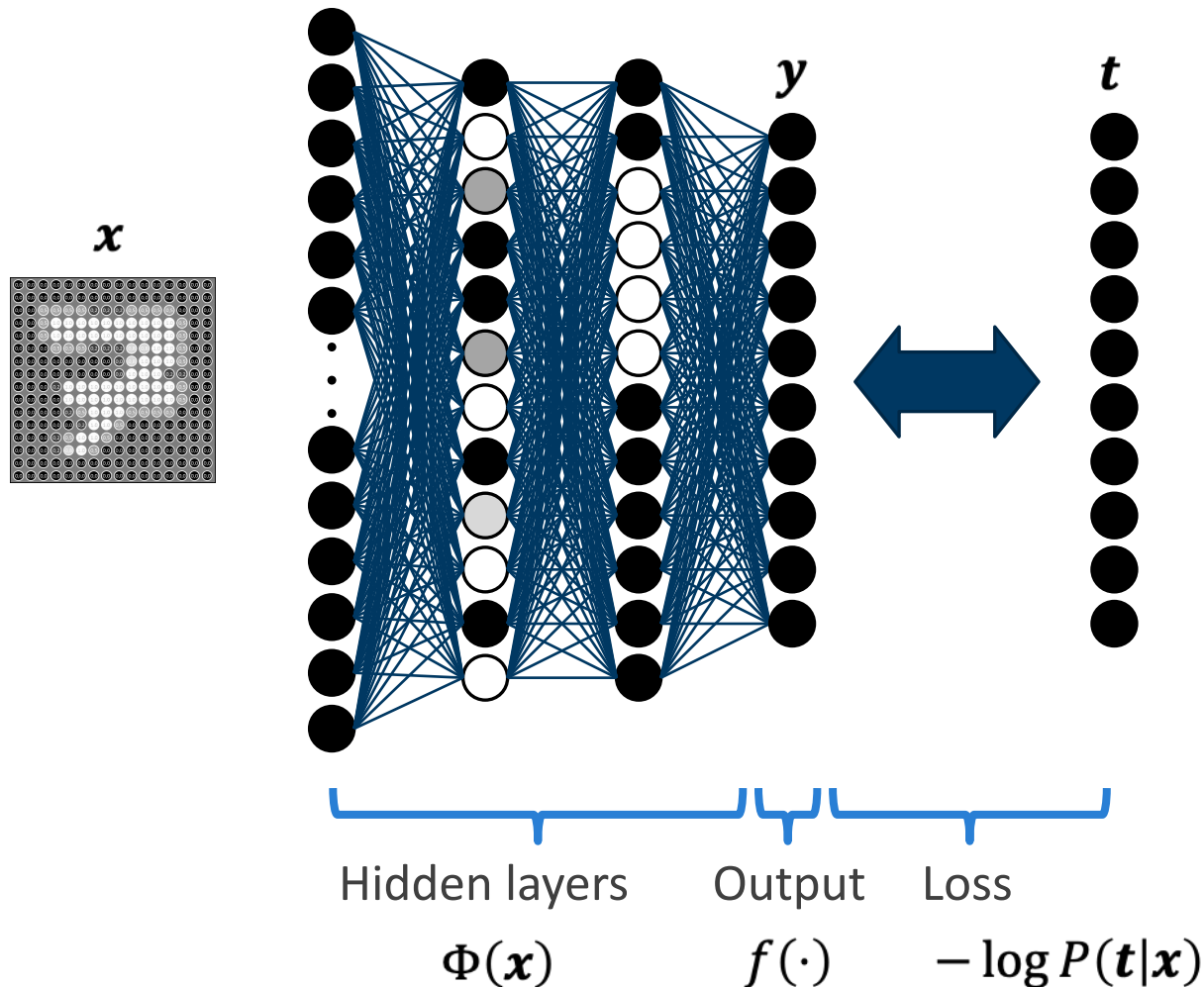
Model

$$P(\mathbf{t}|\mathbf{x}) = f(\Phi(\mathbf{x}))$$

Loss function

$$L = -\log P(\mathbf{t}|\mathbf{x})$$

Output layers and loss functions



Model

$$P(t|x) = f(\Phi(x))$$

Loss function

$$L = -\log P(t|x)$$

Negative log likelihood:
minimizing it -> model output
approaches correct target
distribution

Why negative log likelihood?

Negative: we minimize (instead of maximizing)

Log: distributions are products (of small values) -> log turns into sums (numerically stable)

Likelihood of parameters given the observations <- optimize the to find best parameters

Why negative log likelihood?

Negative: we minimize (instead of maximizing)

Log: distributions are products (of small values) -> log turns into sums (numerically stable)

Likelihood of parameters given the observations <- optimize the to find best parameters

Probability vs. Likelihood:

Probability: $P(O|\theta)$ measure how **observations** fit the **parameters**

Likelihood: $L(\theta|O)$ measure how **parameters** fit the **observations**

<https://stats.stackexchange.com/questions/2641/what-is-the-difference-between-likelihood-and-probability>

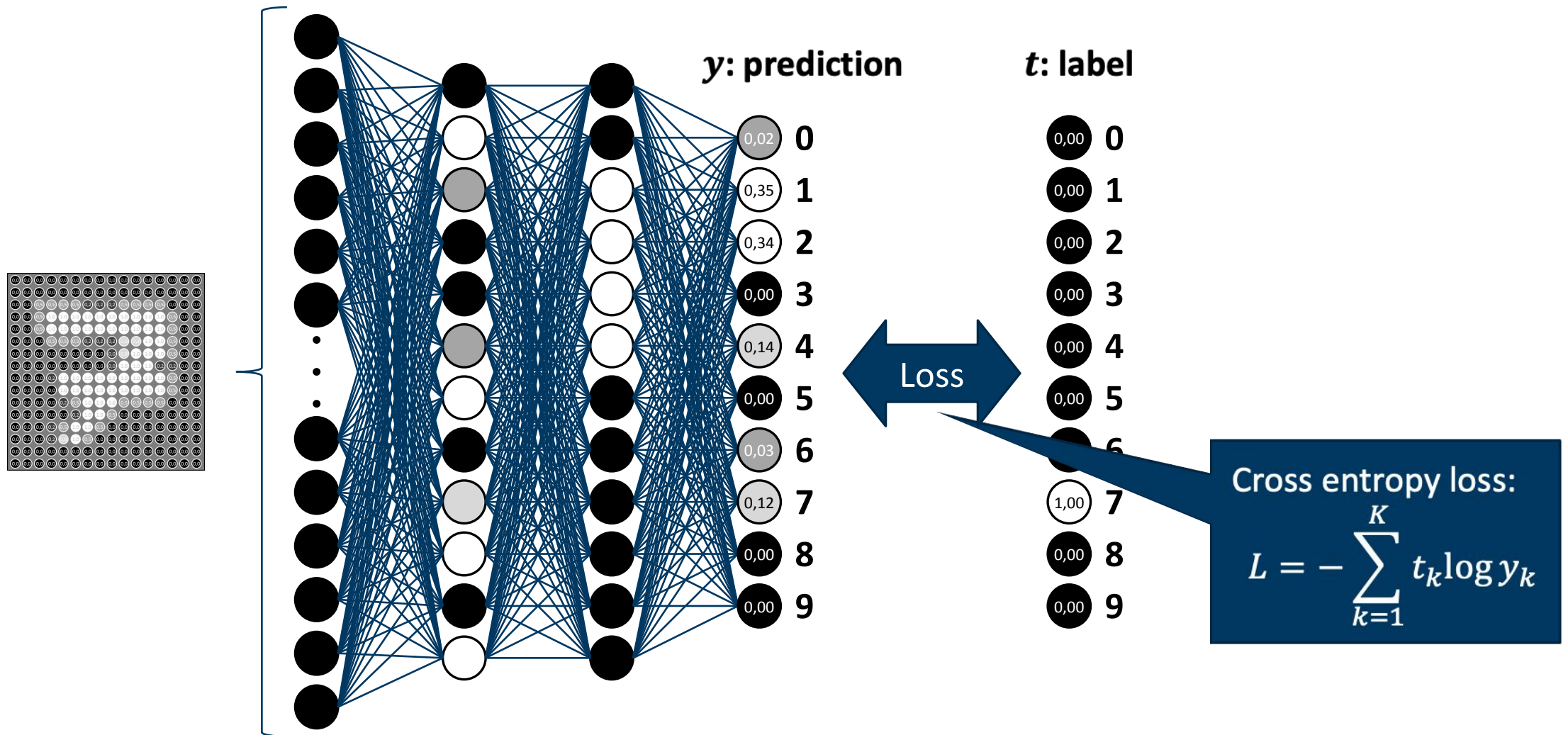
Types of output layer and loss functions

Type of variable	Distribution	Output unit	Activation function	Loss function
t	$P(t x)$		$f(\cdot)$	$-\log P(t x)$
Continuous	Gaussian	Linear	$f(z) = z$	$(y - t)^2$
Binary	Bernoulli	Sigmoid	$f(z) = \frac{1}{1 + \exp(-z)}$	$t \log y$
Categorical	Multinomial	Softmax	$f(z_i) = \frac{\exp(z_i)}{\sum_k \exp(z_k)}$	$\sum_k t_k \log y_k$

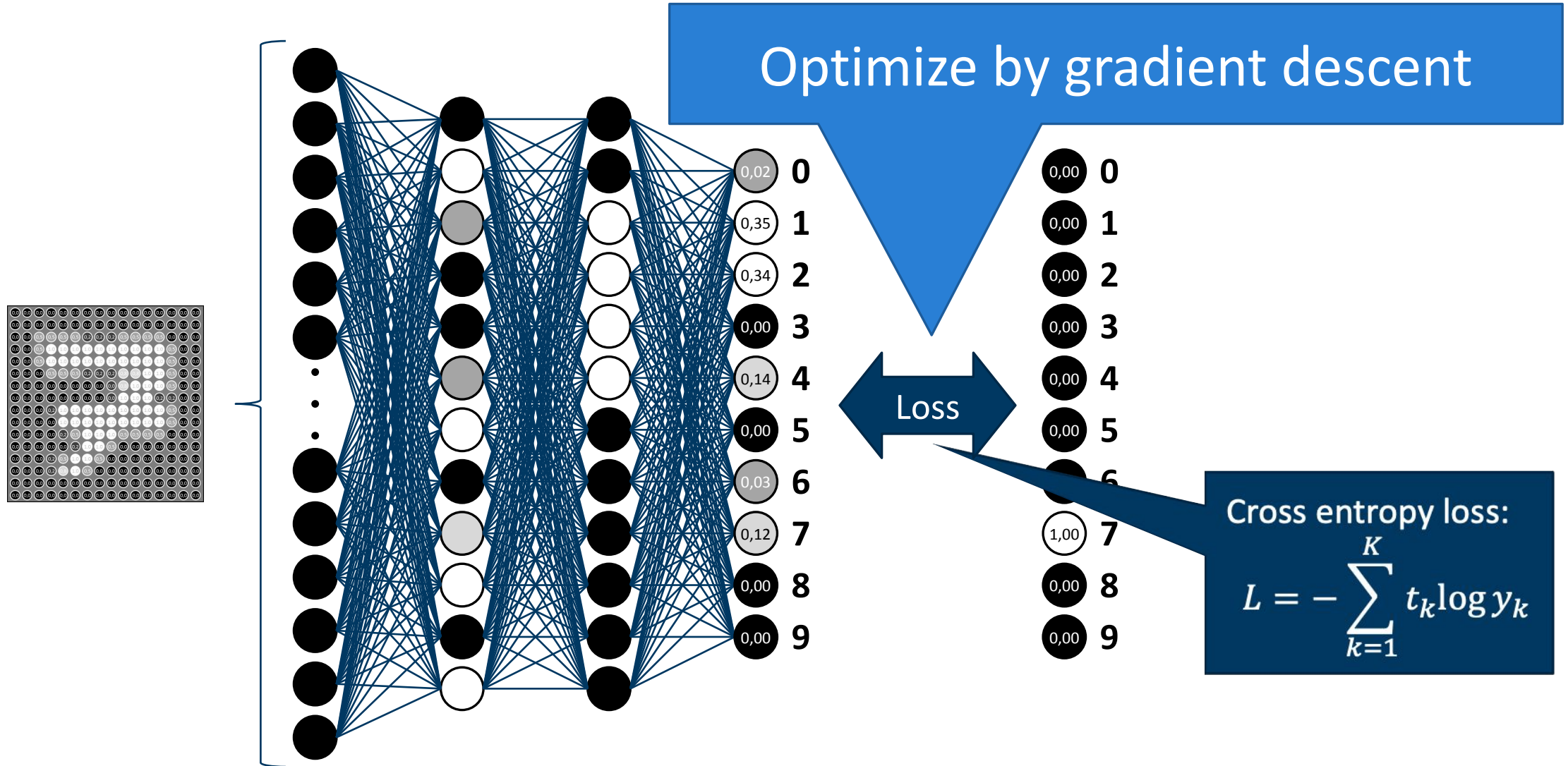
Types of output layer and loss functions

	Type of variable t	Distribution $P(t x)$	Output unit	Activation function $f(\cdot)$	Loss function $-\log P(t x)$
Regression	Continuous	Gaussian	Linear	$f(z) = z$	$(y - t)^2$
Binary Classification	Binary	Bernoulli	Sigmoid	$f(z) = \frac{1}{1 + \exp(-z)}$	$t \log y$
Classification	Categorical	Multinomial	Softmax	$f(z_i) = \frac{\exp(z_i)}{\sum_k \exp(z_k)}$	$\sum_k t_k \log y_k$

Loss function for classification: cross entropy



How to optimize the loss function?

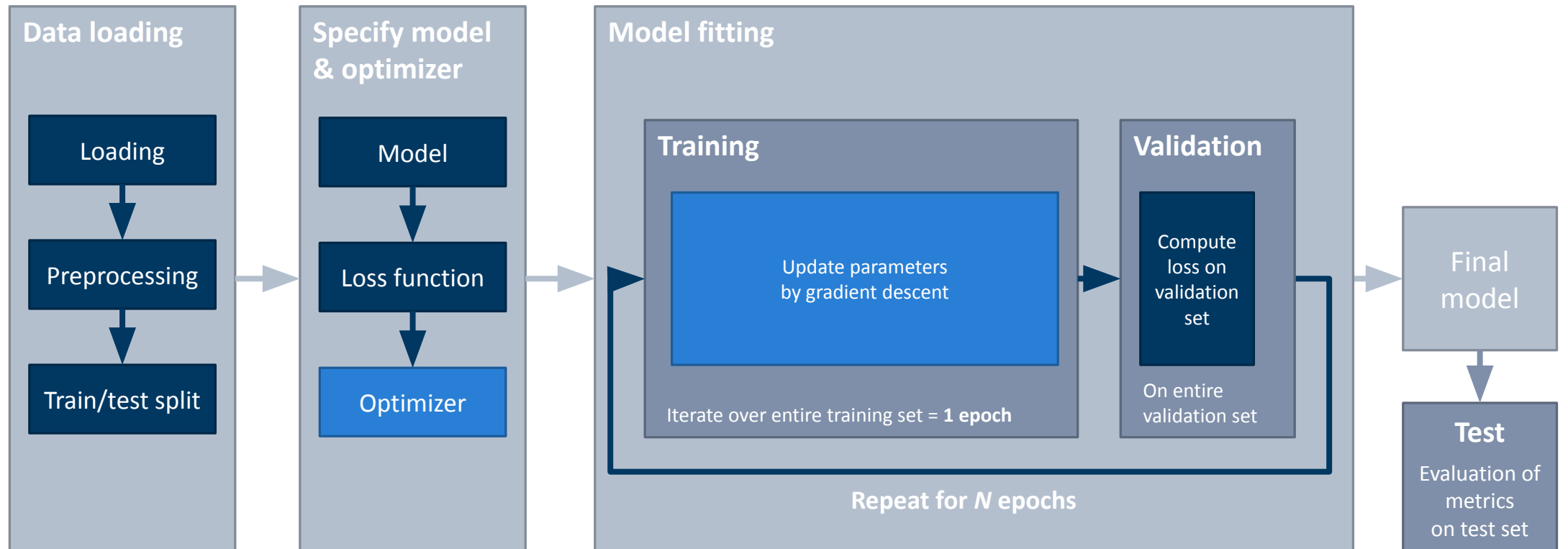


Optimize loss function by gradient descent

We need:

1. Gradient of the loss function w.r.t. the weights, i.e. $\frac{\partial L}{\partial W}$
-> Backpropagation
2. Algorithm to update the weights
-> Stochastic gradient descent

The process of training a deep neural network



Backpropagation (of errors)

Chain rule

Neural network and loss: chained application of functions applied to input

How to compute derivatives?

Chain rule

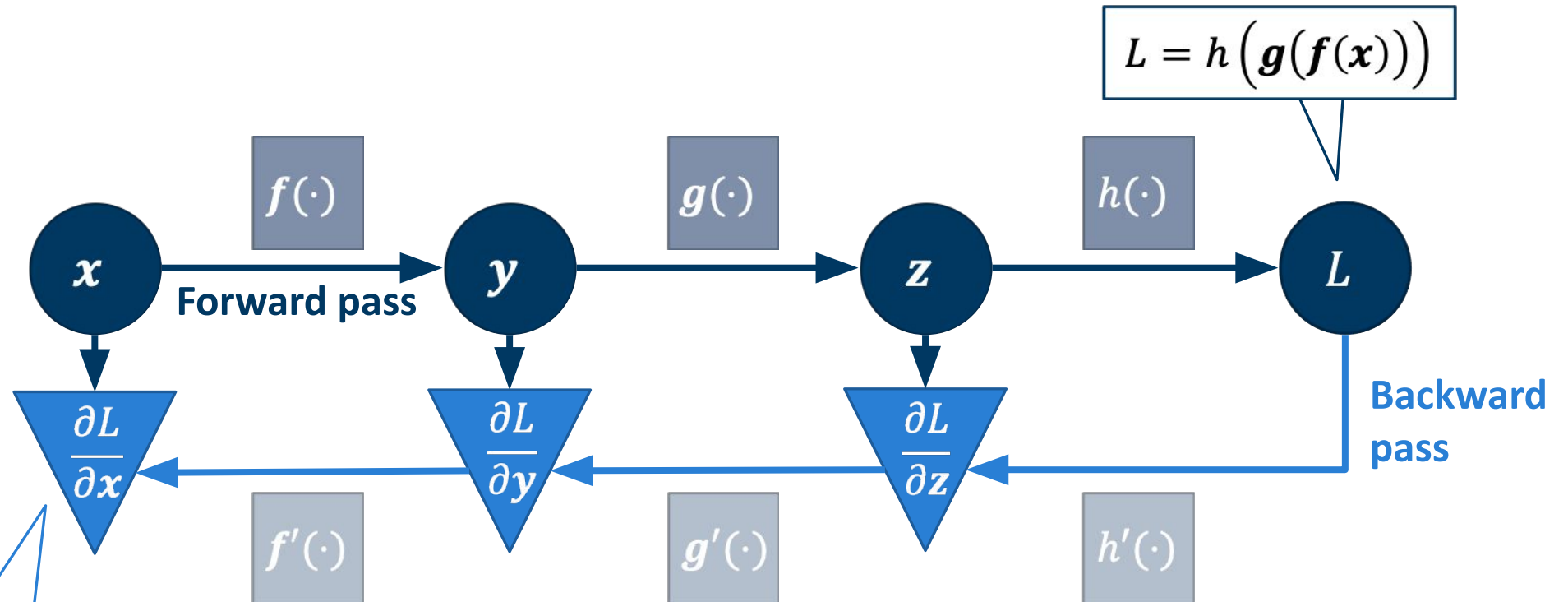
Neural network and loss: chained application of functions applied to input

How to compute derivatives? -> **Chain rule**

Function $h(x) = g(f(x))$

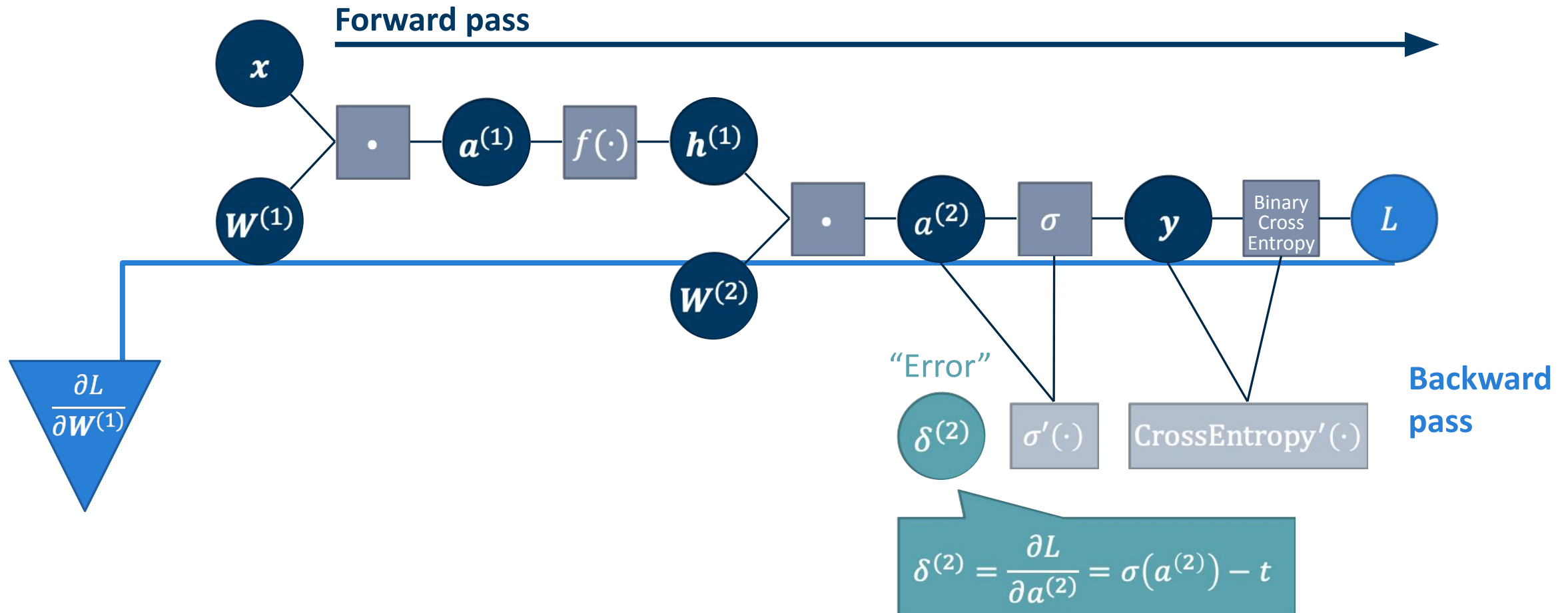
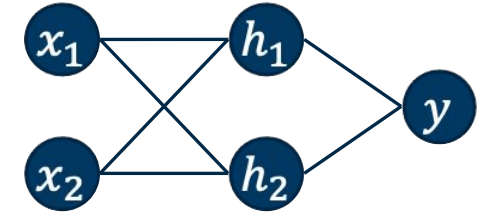
Derivative $h'(x) = g'(f(x)) f'(x)$

Computing gradients: chain rule & backpropagation



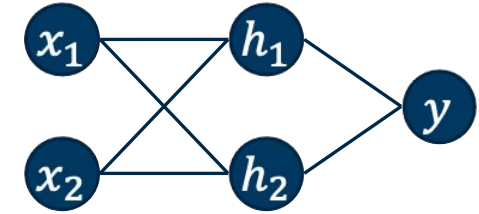
$$\frac{\partial L}{\partial x}(x) = \frac{\partial L}{\partial z} \frac{\partial z}{\partial y} \frac{\partial y}{\partial x}(x) = h'(z) g'(y) f'(x) = h'(g(f(x))) g'(f(x)) f'(x)$$

Backpropagation of errors

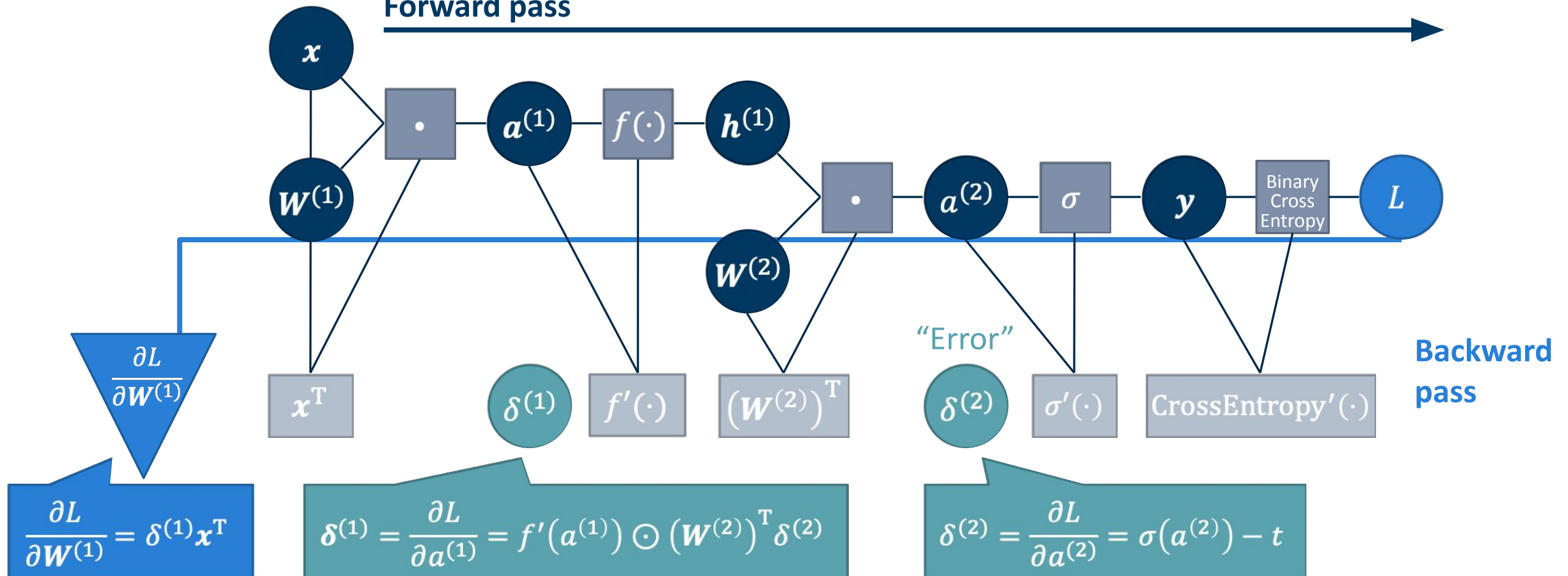


* For simplicity we're ignoring the bias terms in all layers

Backpropagation of errors

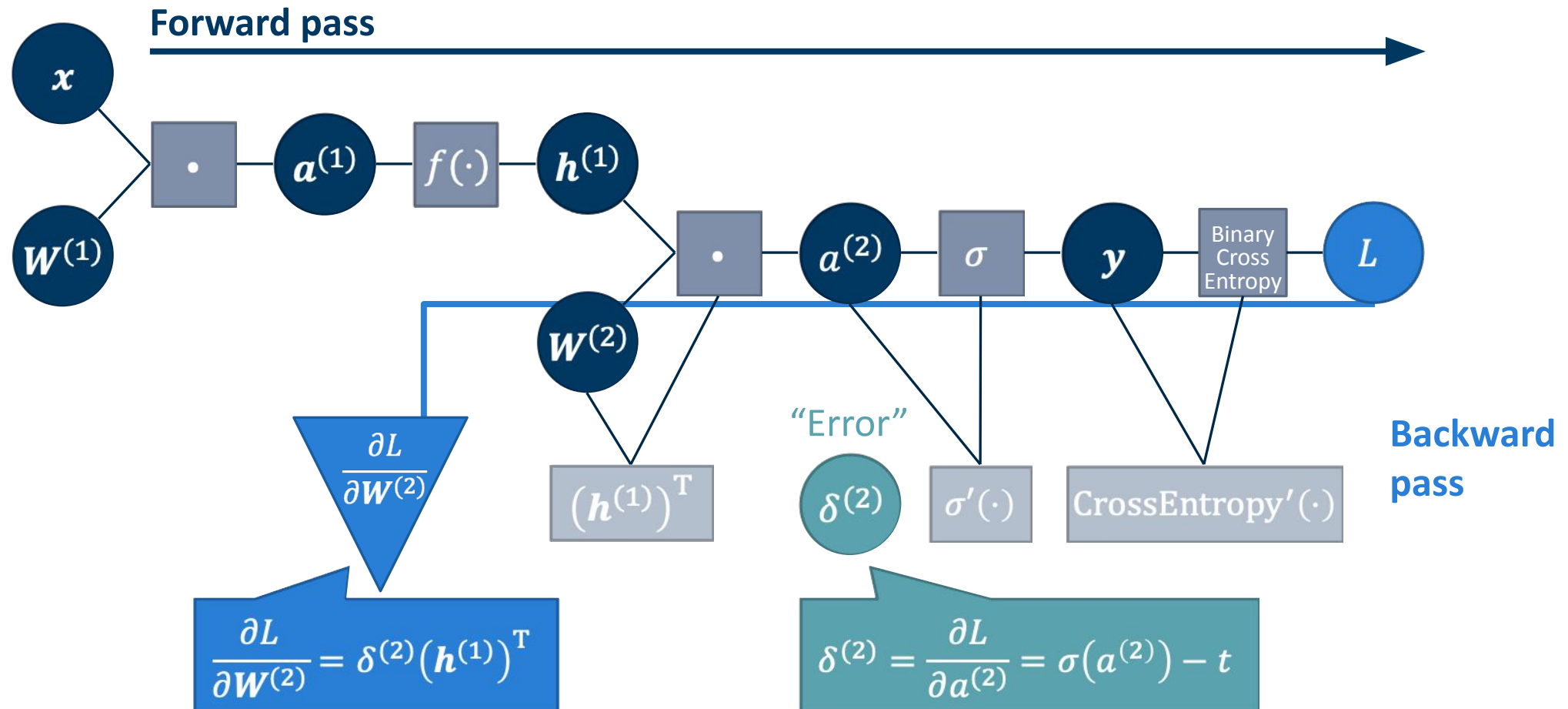
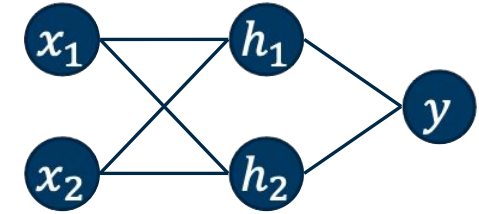


Forward pass



* For simplicity we're ignoring the bias terms in all layers

Backpropagation of errors



* For simplicity we're ignoring the bias terms in all layers

Automatic differentiation

Each operation „knows“ its gradient w.r.t. its inputs

You can compose arbitrary chains of operations (computational graph)

Gradients get computed automatically via chain rule

Modern deep learning frameworks
like *PyTorch*, *Tensorflow* etc. implement automatic differentiation

Stochastic gradient descent

Starting point: gradient descent

Initialize parameters w randomly

Iteratively update w by a small step Δw

$$w \leftarrow w + \eta \Delta w$$

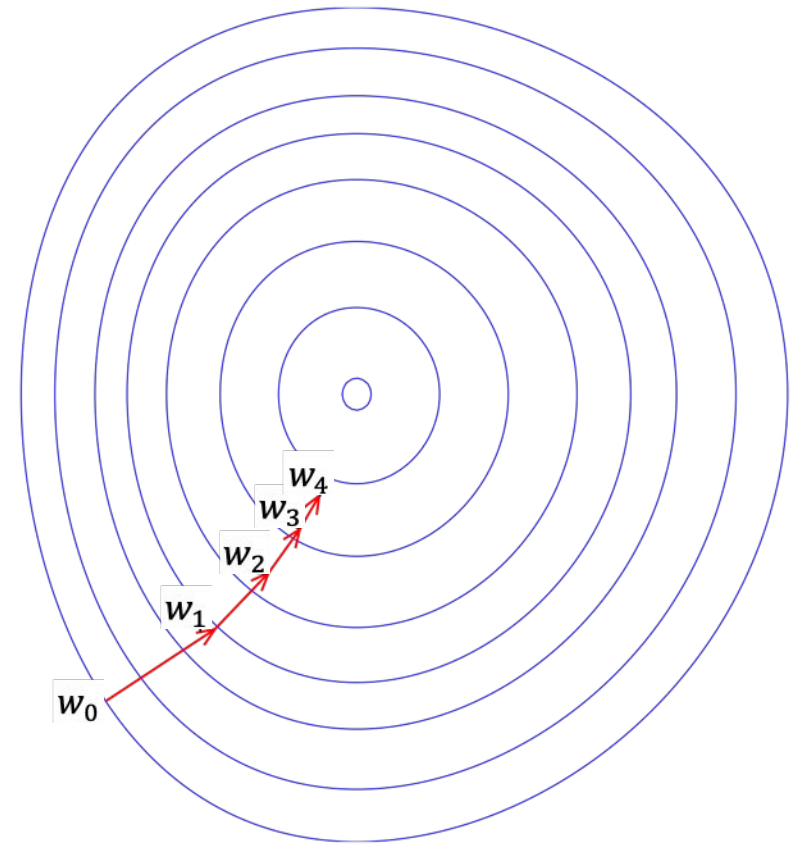
Learning rate

from backprop.

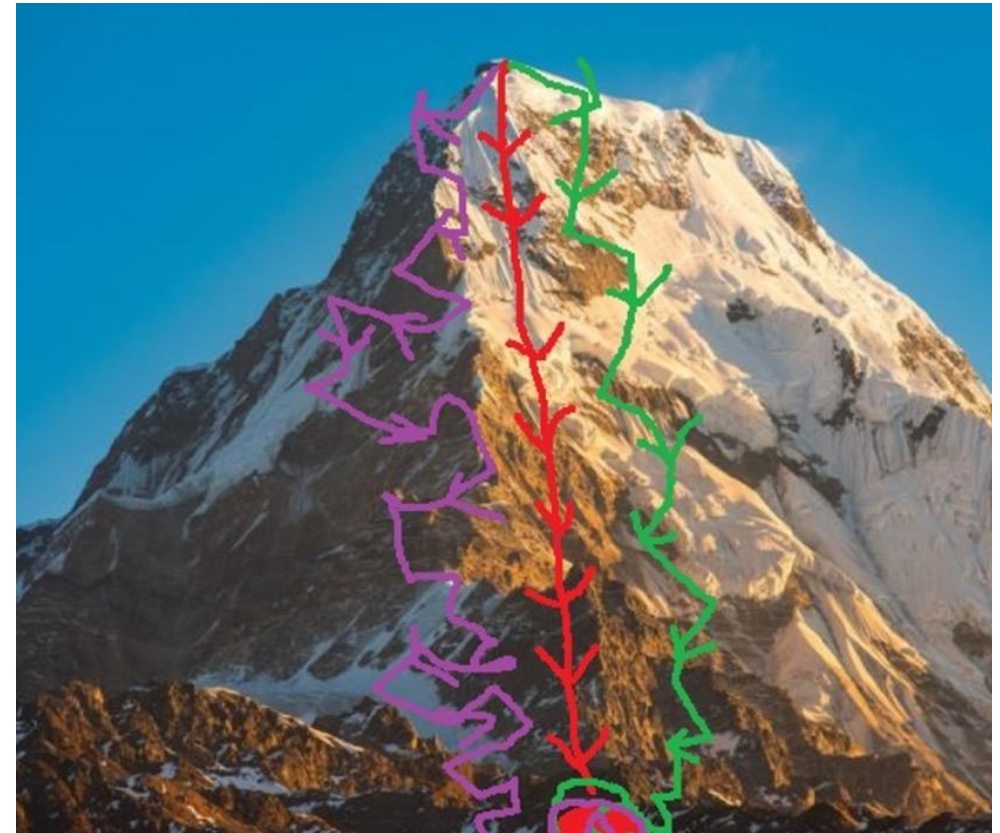
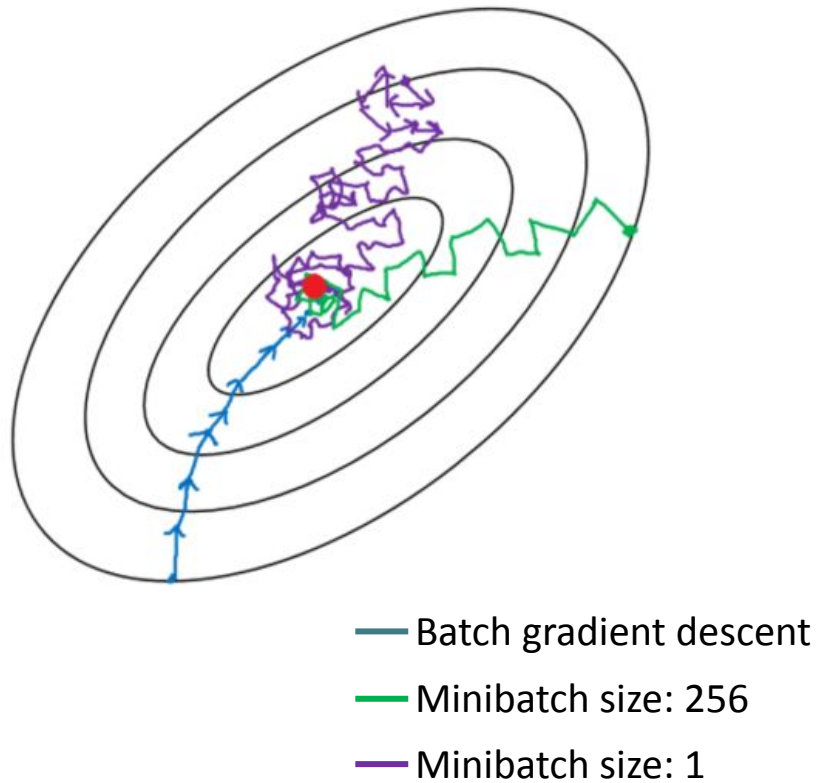


Gradient descent: $\Delta w = -\nabla_w L(w)$

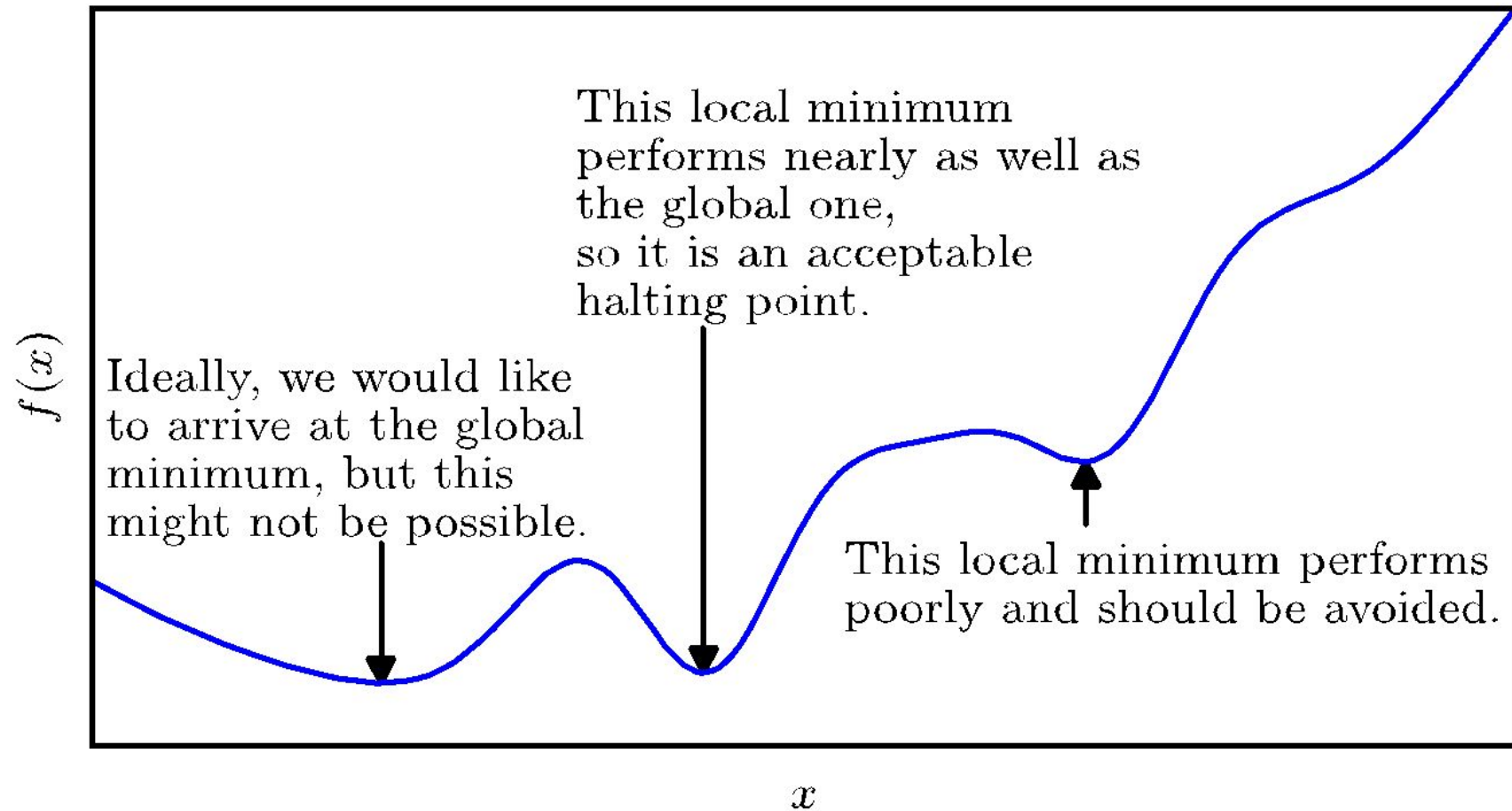
$$w \leftarrow w - \eta \nabla_w L(w)$$



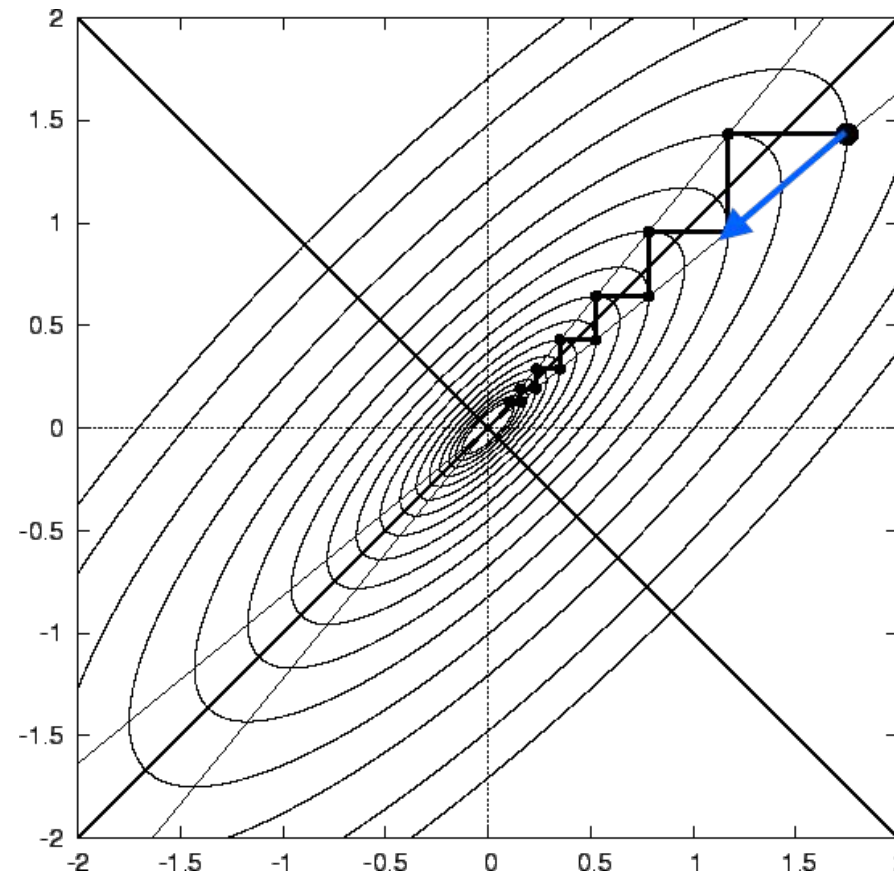
Stochastic gradient descent



Gradient-based optimization does neither always find the absolute minimum...



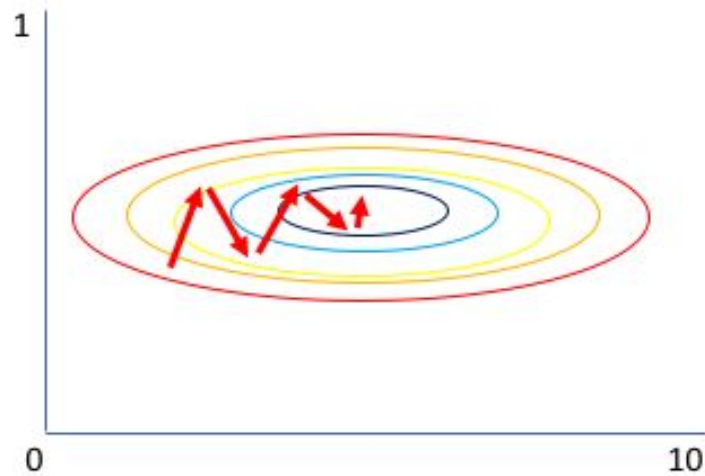
...nor does it find the optimal direction



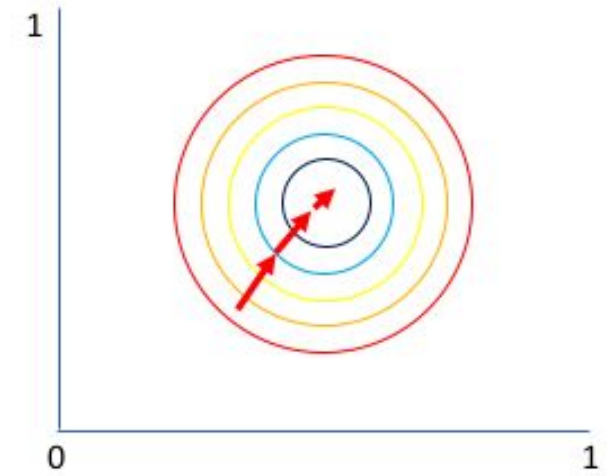
Steepest descent
better direction

Second-order optimizers

Would find a better direction by taking into account the curvature (2^{nd} derivative, Hessian)



Normalize by taking
Hessian into account



BUT: Are generally too computationally expensive.

SGD with momentum

Idea: keep a history of gradient steps to get smoother updates

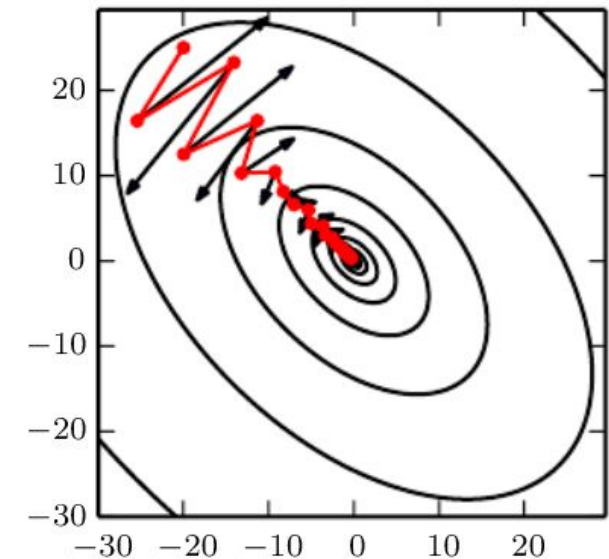
Update velocity v :

Momentum $\beta \in [0,1)$
controls how quickly the effect
of past gradients decays

$$v \leftarrow \beta v + (1 - \beta) \eta \nabla_w L$$

Update parameters:

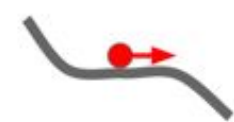
$$w \leftarrow w - v$$



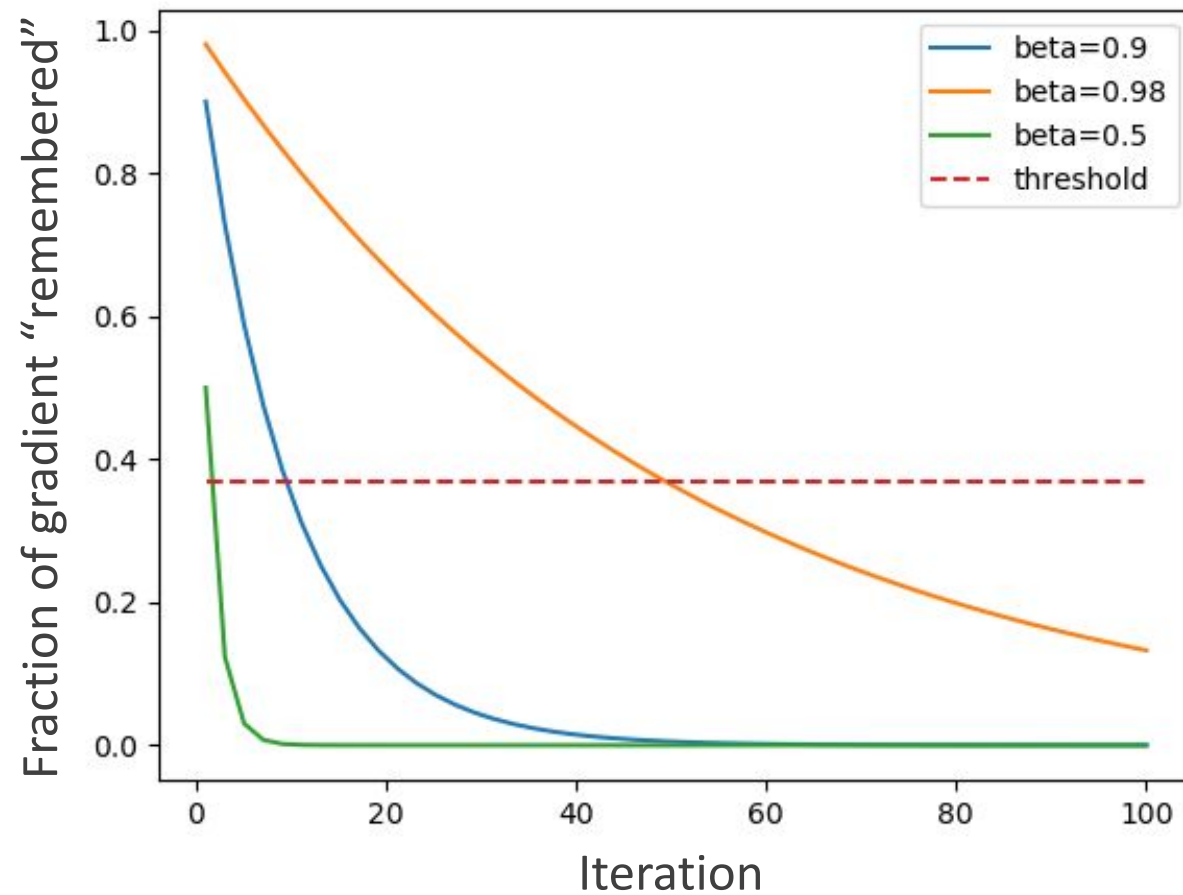
Local Minima



Saddle points



SGD with Momentum: effect of β



AdaGrad

AdaGrad = “adaptive gradient algorithm”

Per-parameter learning rates according to:

$$w_i \leftarrow w_i - \frac{\eta}{\sqrt{G_i}} g_i^{(t)}$$

Iteration t

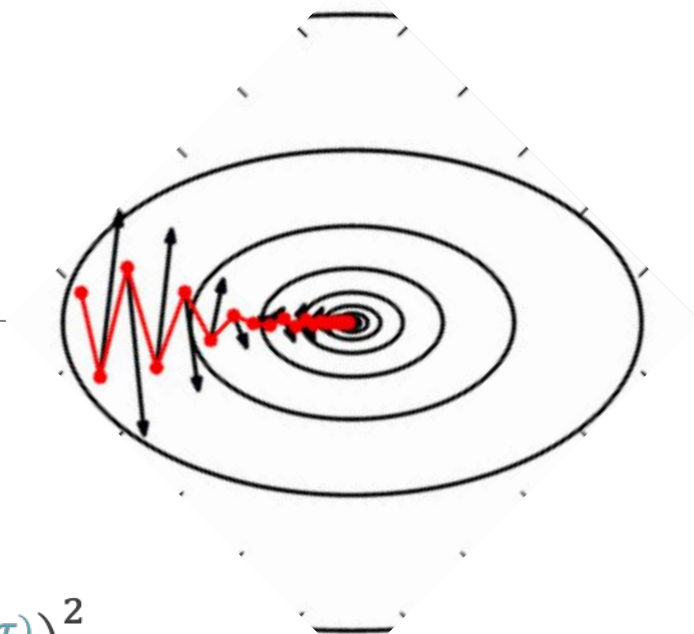
where $G_i = \sum_{\tau=1}^t (g_i^{(\tau)})^2$

$$g_i = \nabla_{w_i} L(w) = \frac{\partial L}{\partial w_i}$$

Gradient of L w.r.t.
 w_i in iteration τ

$\sqrt{G_i}$ is the L_2 norm of previous gradients

- Extreme parameter updates get dampened
- Parameters with small updates get larger learning rates



RMSProp

RMSProp = “root mean square propagation”

Per-parameter learning rates according to:

$$w_i \leftarrow w_i - \frac{\eta}{\sqrt{v^{(t)}}} g_i^{(t)}$$

$$v^{(t)} = \gamma v^{(t-1)} + (1 - \gamma) \left(g_i^{(t)} \right)^2$$

$v^{(t)}$ is an exponential moving average of the squared gradient

→ Similar effects to AdaGrad, but “forgets” earlier gradient magnitudes over time

Most popular optimizer
to achieve results quickly
and robustly

Adam

Adam = “Adaptive Moment Estimation”

Combination of Momentum and RMSProp optimizer

$$\begin{aligned}m_w^{(t)} &\leftarrow \beta_1 m_w^{(t-1)} + (1 - \beta_1) g^{(t)} \\v_w^{(t)} &\leftarrow \beta_2 v_w^{(t-1)} + (1 - \beta_2) (g^{(t)})^2\end{aligned}$$

Exponential moving average of mean $m_w^{(t)}$ and variance $v_w^{(t)}$ of gradient (biased estimate)

$$\hat{m}_w = \frac{m_w^{(t)}}{1 - \beta_1^t} \quad \hat{v}_w = \frac{v_w^{(t)}}{1 - \beta_2^t}$$

*Bias correction
for moments*

$$w^{(t)} \leftarrow w^{(t-1)} - \eta \frac{\hat{m}_w}{\sqrt{\hat{v}_w} + \epsilon}$$

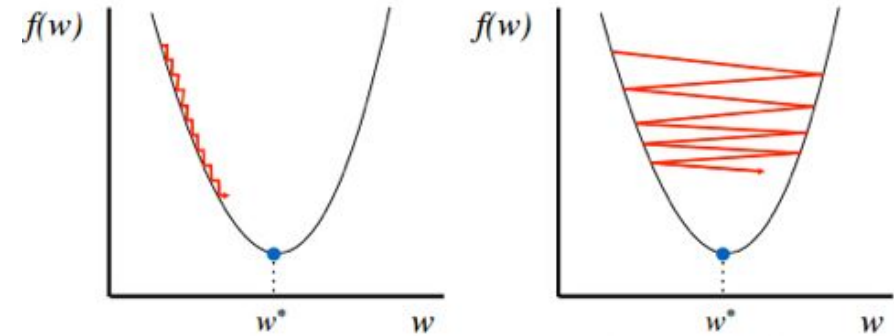
Parameter update

PyTorch defaults:
 $\beta_1 = 0.9$
 $\beta_2 = 0.999$
 $\epsilon = 10^{-8}$

Optimization results depend on hyperparameters

Choice of step size (learning rate) is important

- Too small: converge slowly or find local minimum
- Too large: oscillations or even divergence



Momentum affects convergence

- Too small: get stuck in local minima or saddle points
- Too large: overshoot optimum or spiral around it

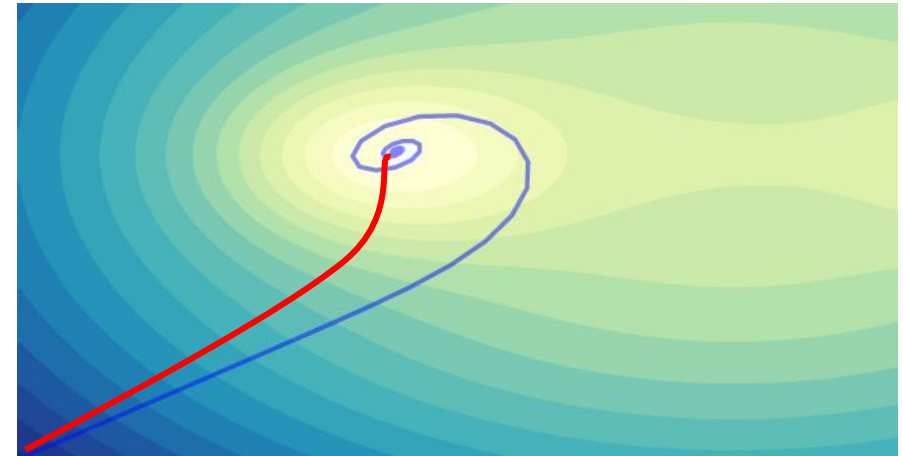
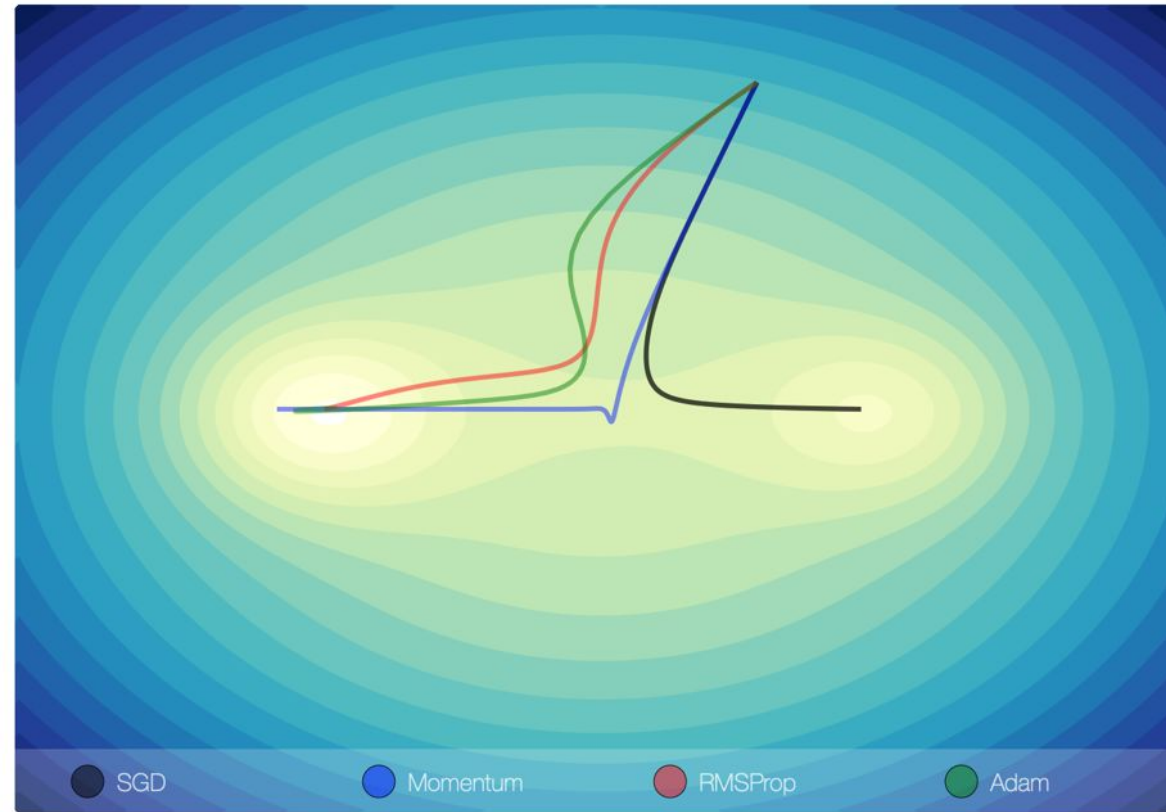
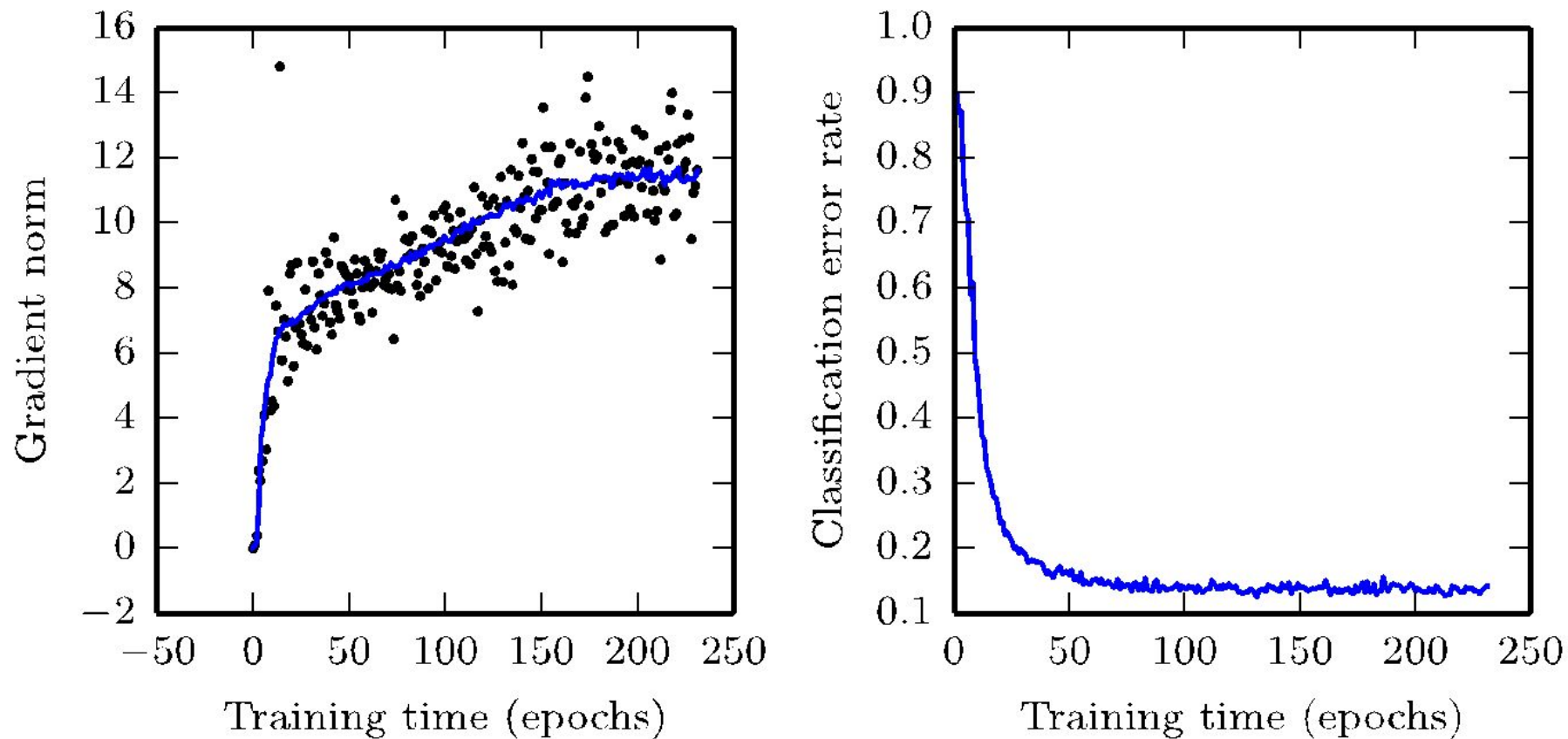


Illustration of trajectories for different optimizers



<https://emiliendupont.github.io/2018/01/24/optimization-visualization/>

We usually don't even reach a local minimum



Deep Learning optimization way of life

PURE MATH WAY OF LIFE

Find literally the smallest value of $f(x)$

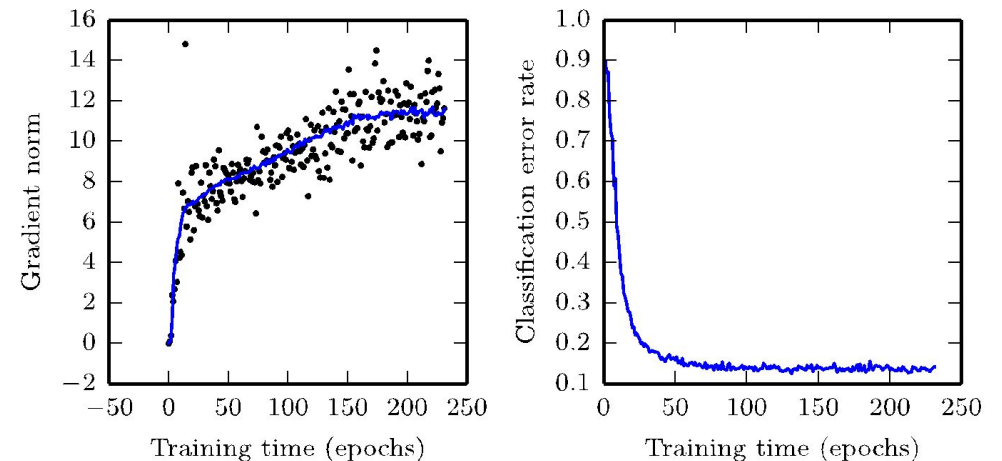
Alternatively: find some critical point of $f(x)$ where the value is locally smallest

(also true for convex methods like logistic regression/SVM)



DEEP LEARNING WAY OF LIFE

Decrease the value of $f(x)$ a lot.



Regularization

Regularization

Definition: process that changes the solution (= network parameters) to be “simpler”.

E.g. smaller absolute weight values.

Goal: avoid overfitting, obtain results for ill-posed problems.

Regularization

Definition: process that changes the solution (= network parameters) to be “simpler”.

E.g. smaller absolute weight values.

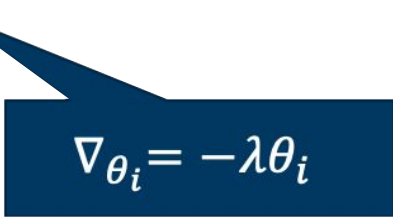
Goal: avoid overfitting, obtain results for ill-posed problems.

- **Explicit regularization:** term added to loss function
- **Implicit regularization:** other training procedure to achieve regularization

L_2 regularization (weight decay)

Assume Gaussian prior on weights θ

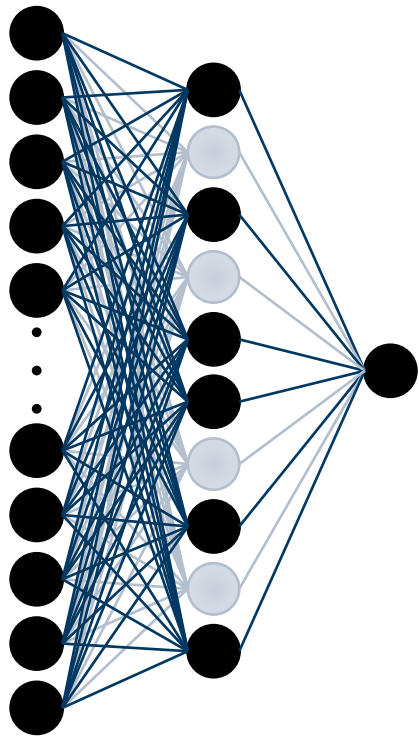
$$L = -\log P(\hat{y}|\mathbf{x}, \theta) - \underbrace{\frac{\lambda}{2} \|\theta\|_2^2}$$


$$\nabla_{\theta_i} = -\lambda \theta_i$$

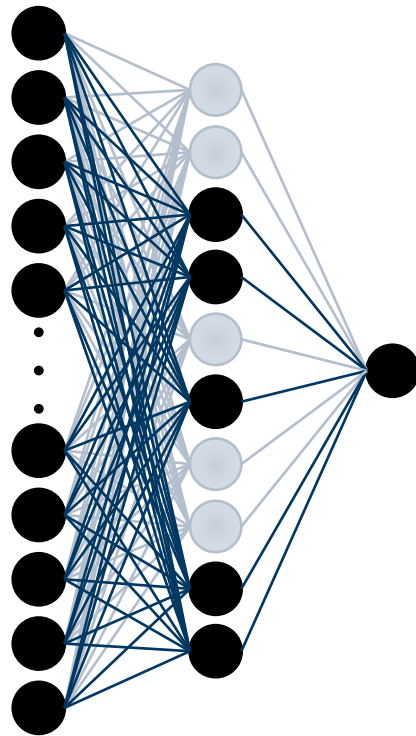
A.k.a **weight decay** because the weights decay towards zero in every iteration.

Dropout:

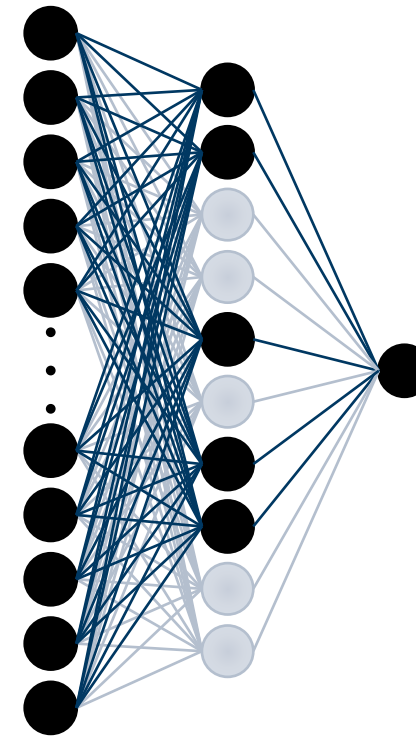
turn off hidden units randomly during training



Iteration 1



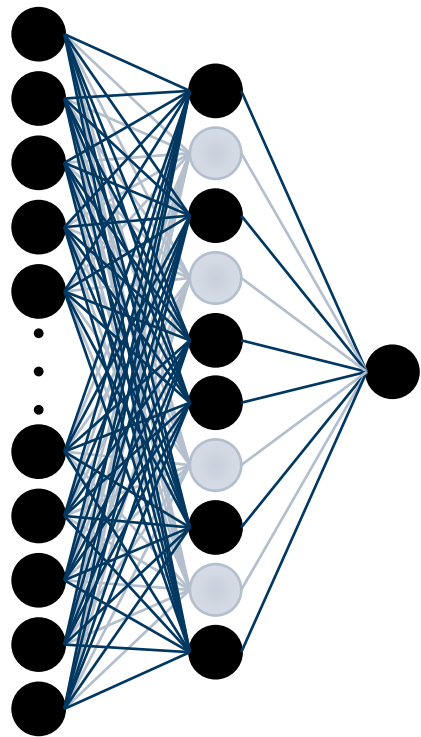
Iteration 2



Iteration 3

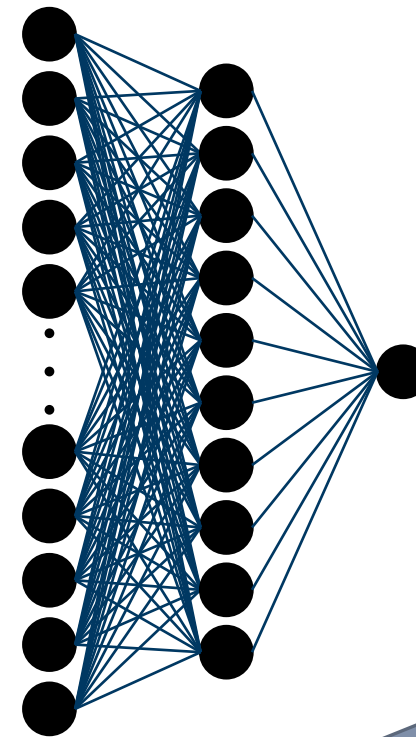
...

Dropout: training vs. inference



Training:
Train an ensemble
of models

$$h_i^{(n)} = \begin{cases} f(W_{i:}^{(n)} \mathbf{h}^{(n-1)}) & \text{with prob. } p \\ 0 & \text{otherwise} \end{cases}$$

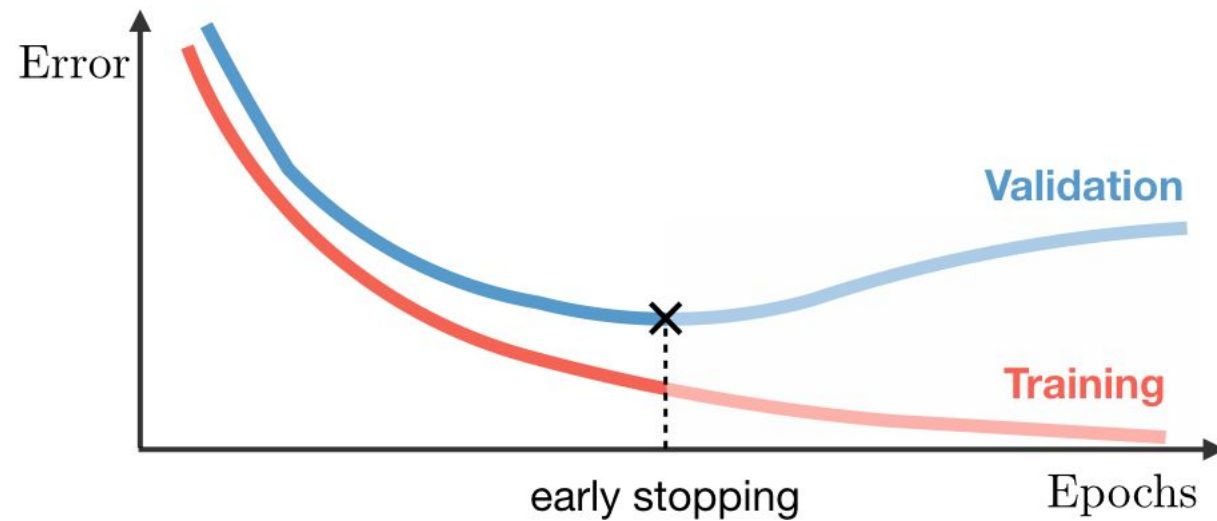


Inference:
Perform model
averaging

Adjust gain to keep
average activation
level constant

$$h_i^{(n)} = p \cdot f(W_{i:}^{(n)} \mathbf{h}^{(n-1)})$$

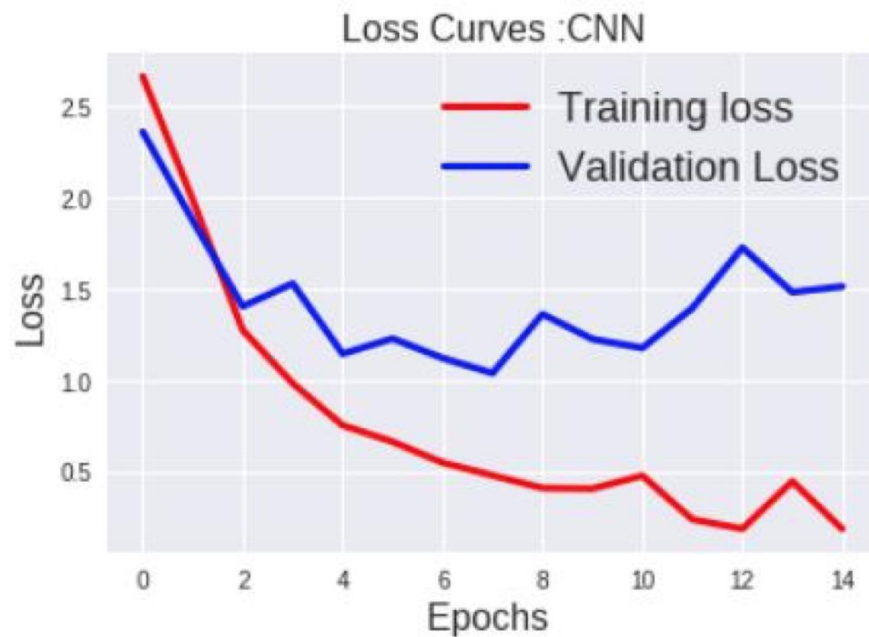
Early stopping



Idea: Stop training when generalization error increases

Early stopping: in practice

PROBLEM: LOSS FLUCTUATES



POSSIBLE SOLUTIONS

Smooth validation loss by computing running average (e.g. exponential moving average)

Introduce “patience” parameter τ : stop when loss has not decreased for τ steps/epochs in a row



Babysitting training neural networks

“How to train your network”

Babysitting neural network training

- **Step 0: Check the training data**

Inspect the data directly from the training loader.

- Does the data processing work (correct normalization, not all zeros etc.)
- Are data and labels matching? (Make sure they're not randomly shuffled etc.)

Babysitting neural network training

- Step 0: Check the training data
- **Step 1: Check initial loss**

Verify that initial loss makes sense, e.g.

- Regression: mean squared error not larger than variance of the targets
- Classification: $\log(C)$ for softmax with C classes

If not, bad initialization or some issue with architecture

Babysitting neural network training

- Step 0: Check the training data
- Step 1: Check initial loss
- **Step 2: Overfit on a single minibatch / small sample**

You should be able to train to 100% training accuracy on a single minibatch (or sample of a few minibatches).

If not, check initialization, learning rate.

Loss not going down? Learning rate too low, bad initialization

Loss explodes to Inf or NaN? Learning rate too high, bad initialization

Babysitting neural network training

- Step 0: Check the training data
- Step 1: Check initial loss
- Step 2: Overfit on a single minibatch / small sample
- **Step 3: Find a learning rate that reduces loss significantly**

Use the full dataset and find a learning rate that makes the loss go down significantly within 100 iterations (**not epochs!**) or so.

Good learning rates to try: 0.1, 0.01, 10^{-3} , 10^{-4}

Babysitting neural network training

- Step 0: Check the training data
- Step 1: Check initial loss
- Step 2: Overfit on a single minibatch / small sample
- Step 3: Find a learning rate that reduces loss significantly
- **Step 4: Coarse grid, train a few epochs**

Choose a few learning rates around what worked for step 3 and train for a few (1–5) epochs.

If you use regularization, test a few values. Good values for weight decay to try: 10^{-4} , 10^{-5} , 0.

Babysitting neural network training

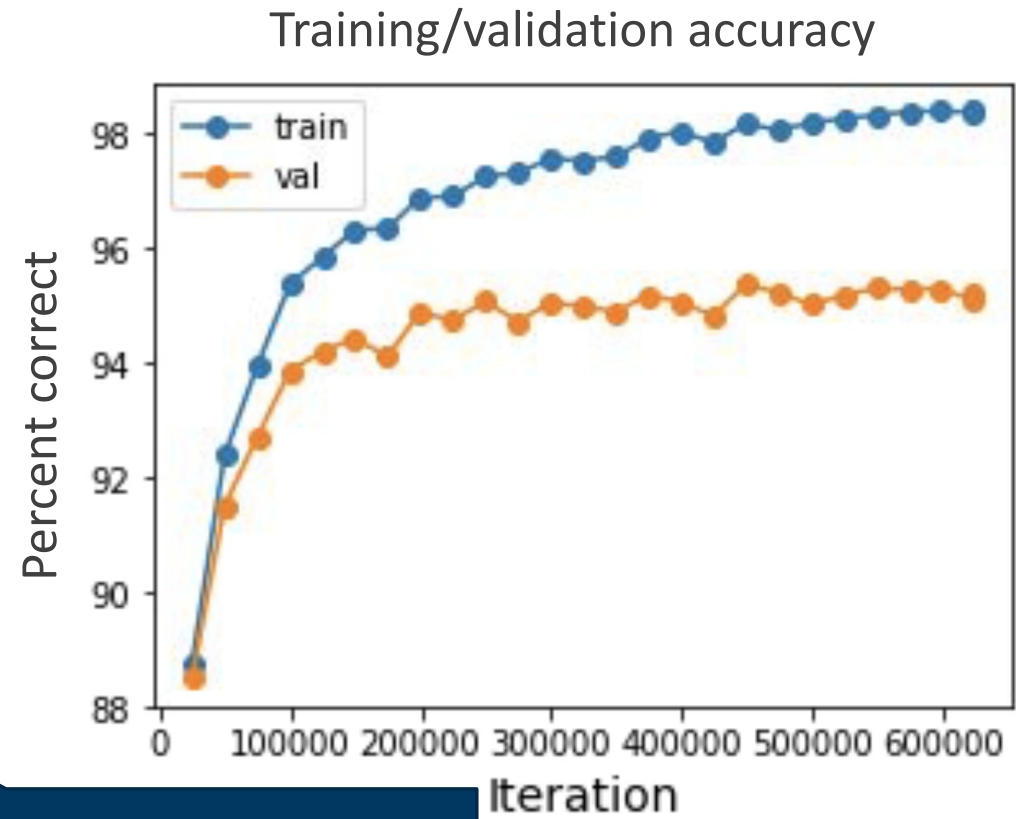
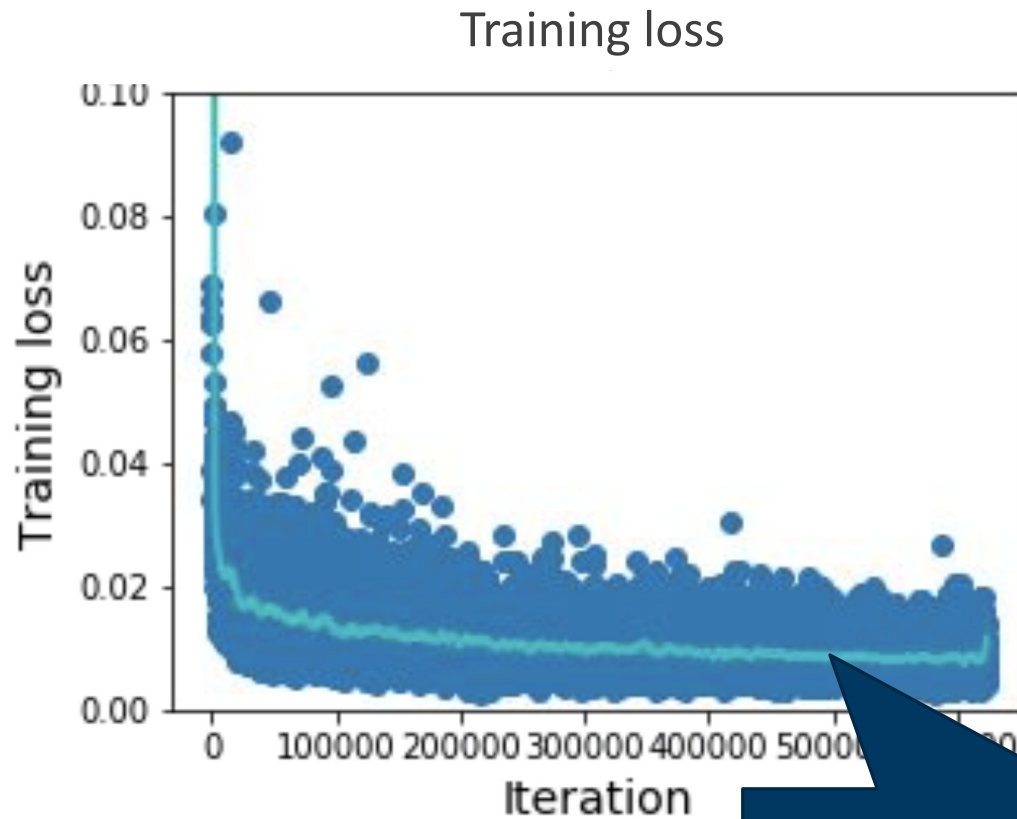
- Step 0: Check the training data
- Step 1: Check initial loss
- Step 2: Overfit on a single minibatch / small sample
- Step 3: Find a learning rate that reduces loss significantly
- Step 4: Coarse grid, train a few epochs
- **Step 5: Refine grid, train longer**

Pick best model from step 4 and train longer (10–50 epochs) using a constant learning rate

Babysitting neural network training

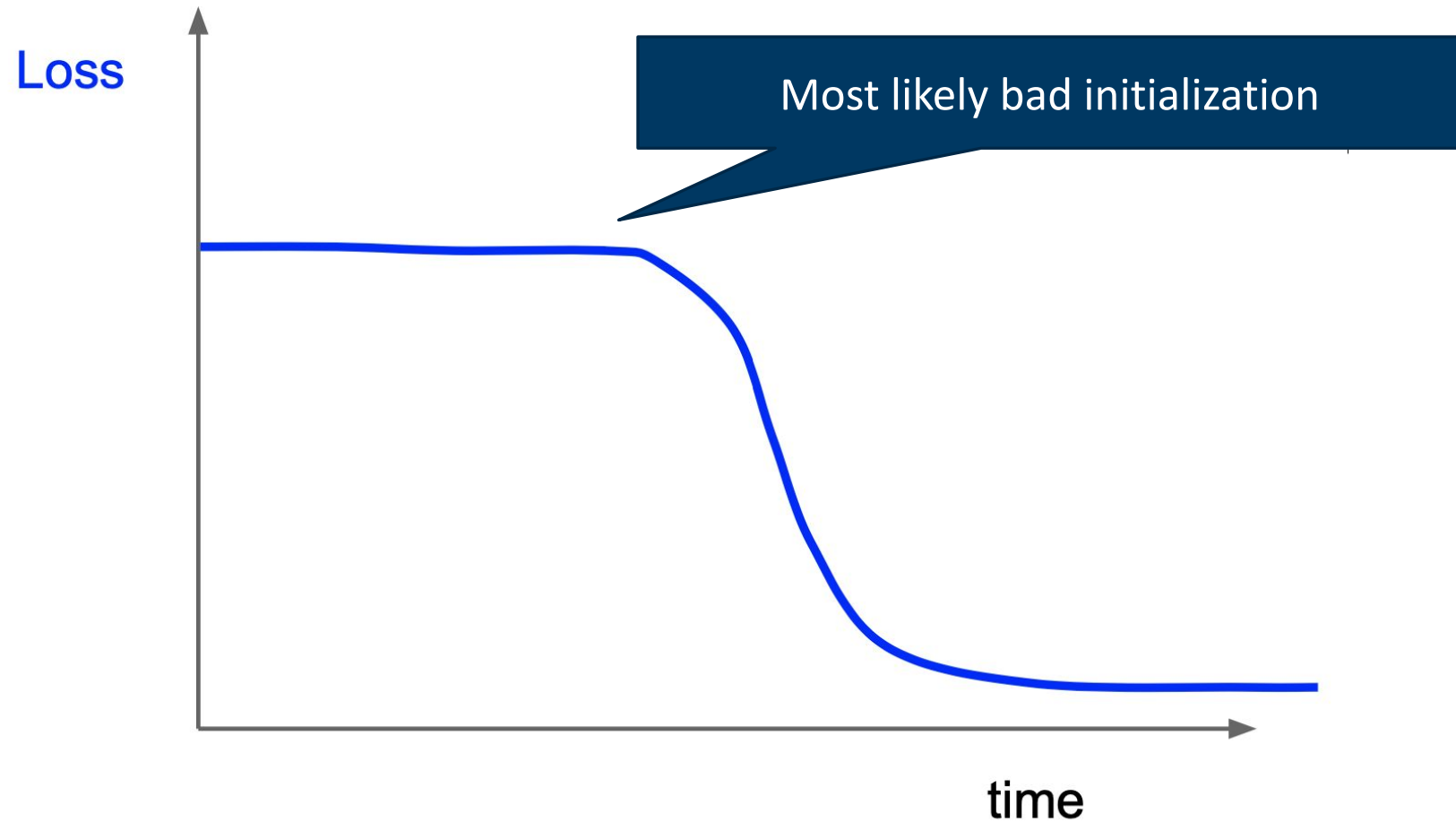
- Step 0: Check the training data
- Step 1: Check initial loss
- Step 2: Overfit on a single minibatch / small sample
- Step 3: Find a learning rate that reduces loss significantly
- Step 4: Coarse grid, train a few epochs
- Step 5: Refine grid, train longer
- **Step 6: Look at loss curves!**

Look at learning curves!

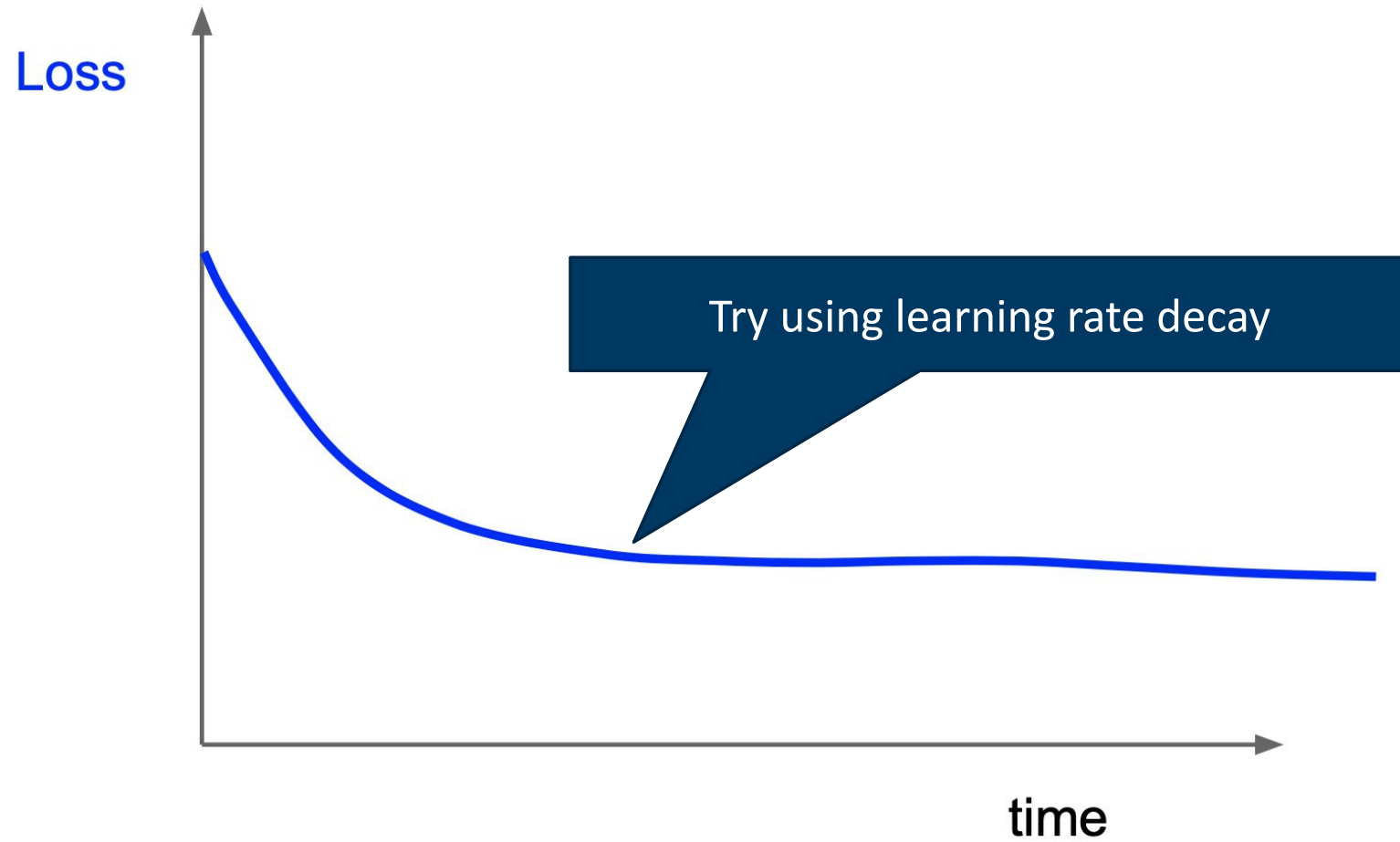


Loss curves may be noisy ->
Use exponential moving average

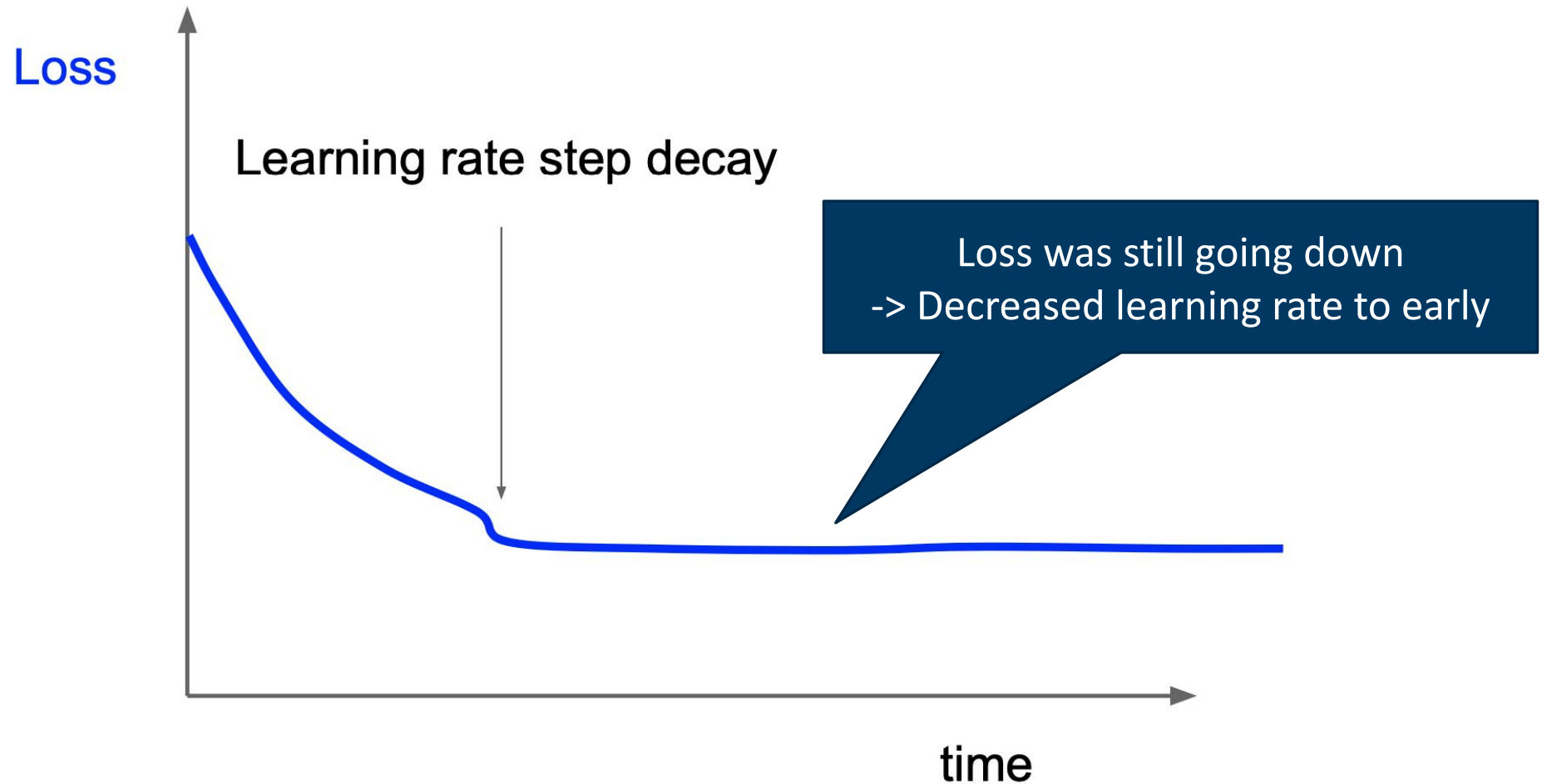
Loss does not decrease for a long time



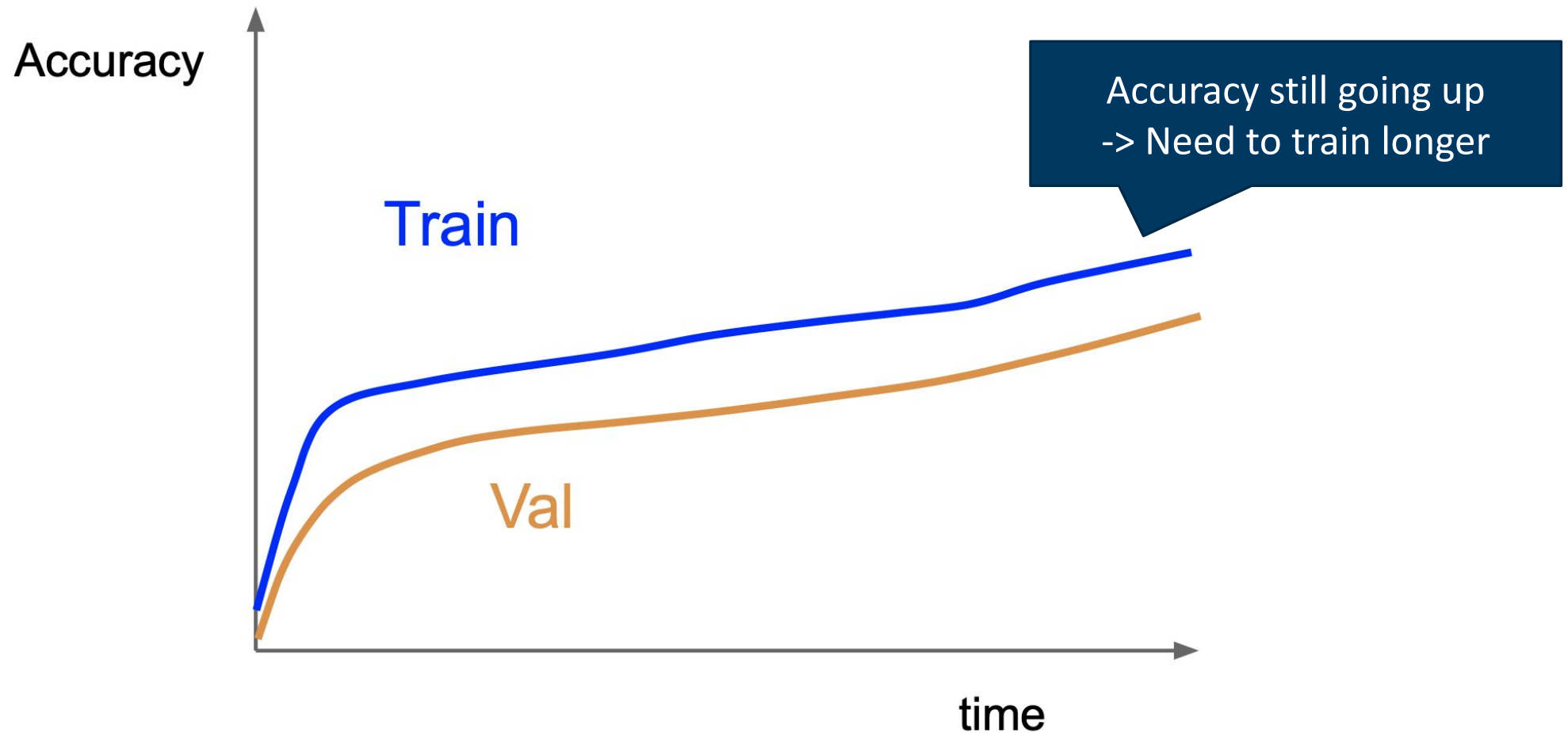
Loss plateaus



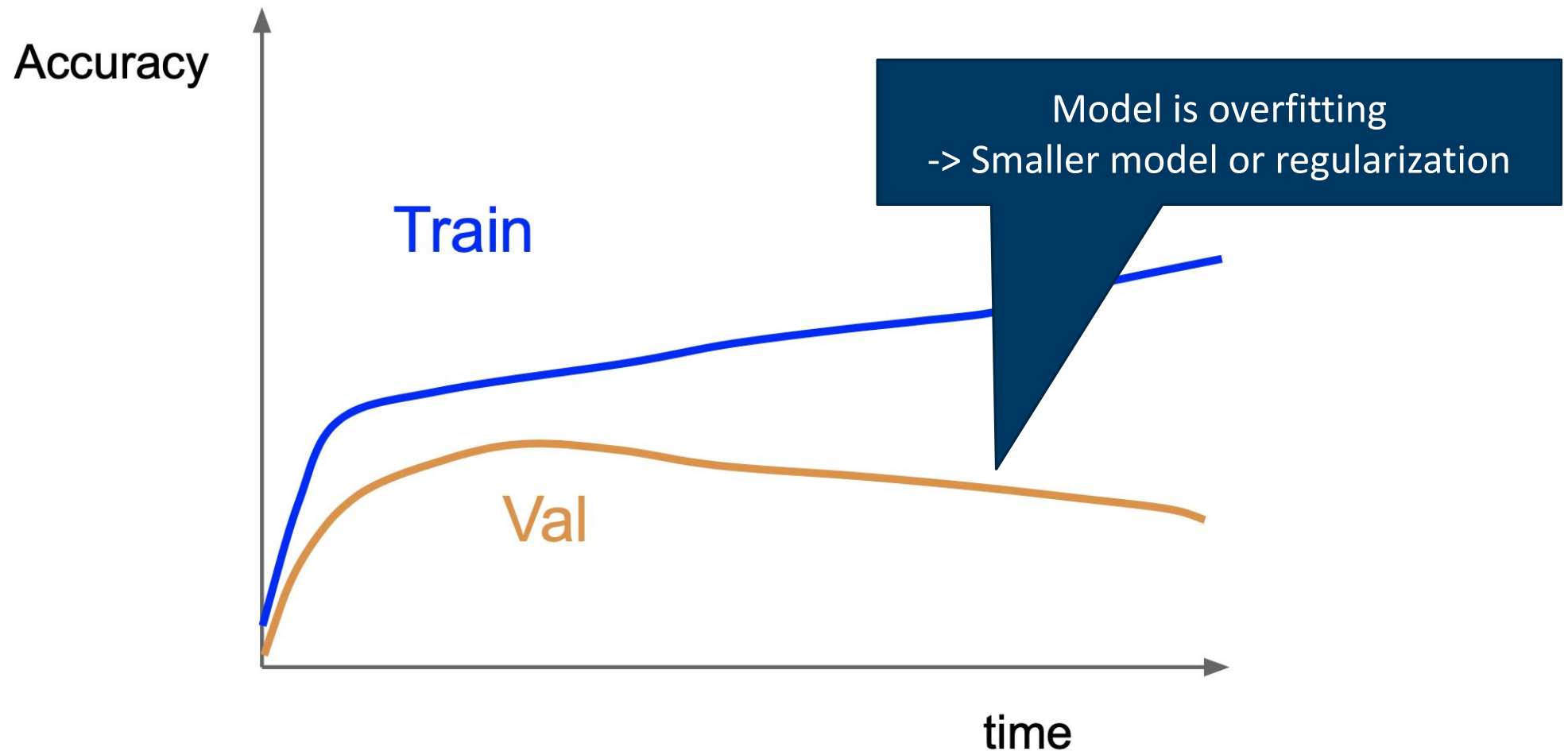
Timing of learning rate decay



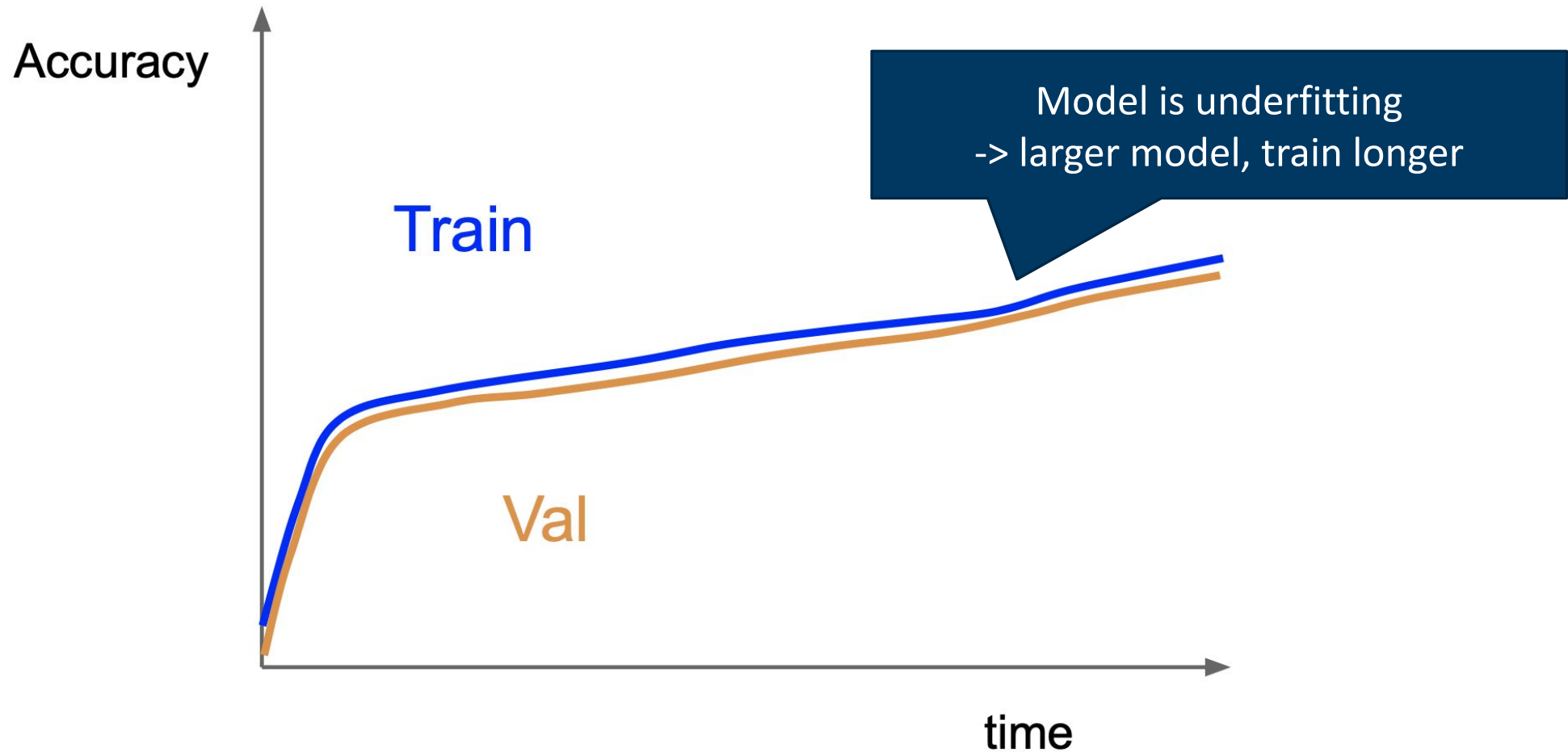
Also look at accuracy: loss is only a surrogate!



Large gap between training and validation set



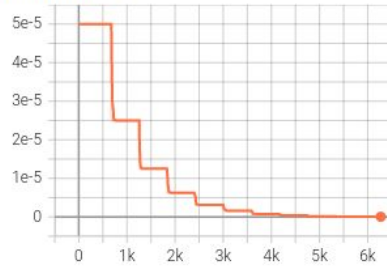
No gap between training and validation set



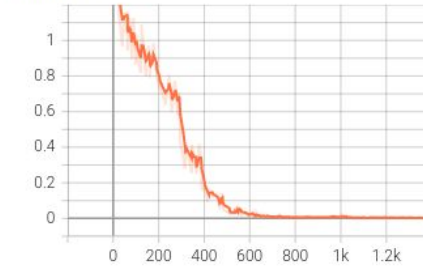
Zero train loss - high, uncorrelated validation

train

train/learning_rate
tag: train/learning_rate

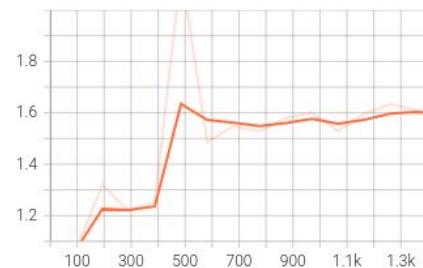


train/loss
tag: train/loss

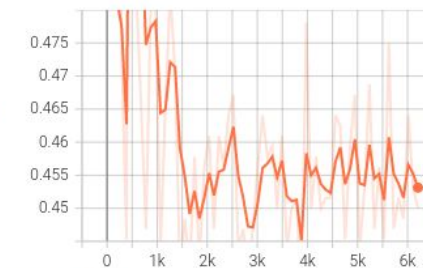


validation

validation/loss
tag: validation/loss

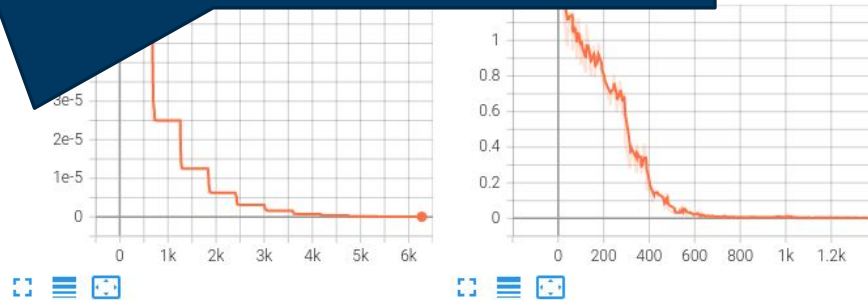


validation/metric
tag: validation/metric



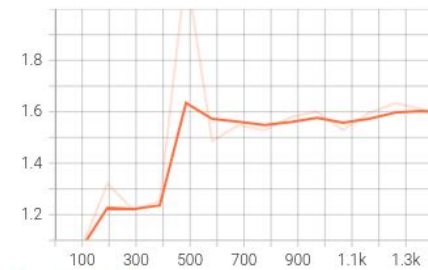
Zero train loss - high, uncorrelated validation

Labels shuffled! Model overfits completely to the training data!

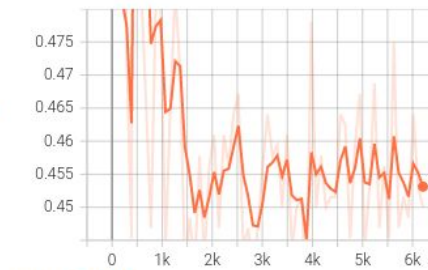


validation

validation/loss
tag: validation/loss



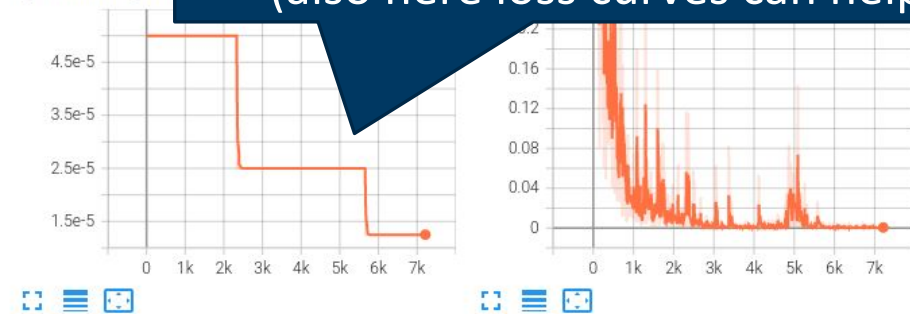
validation/metric
tag: validation/metric



With correct labels.
-> Should notice in Step 0
(also here loss curves can help)

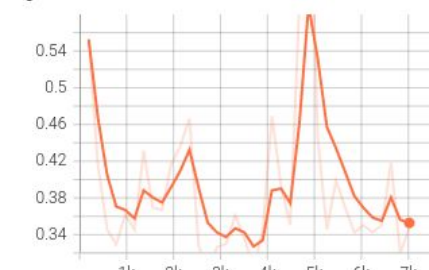
train

train/learning_rate
tag: train/learning_rate

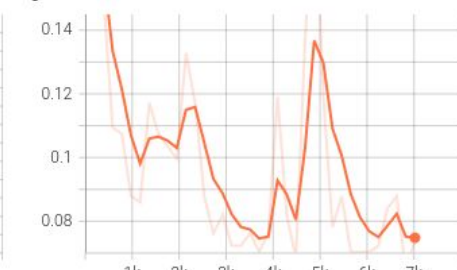


validation

validation/loss
tag: validation/loss



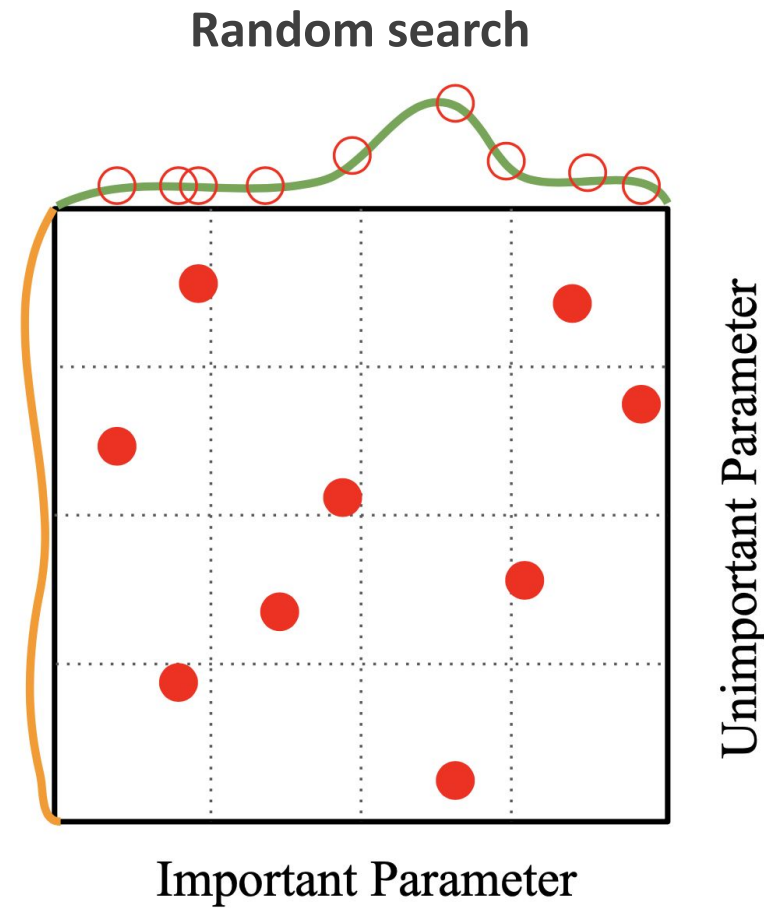
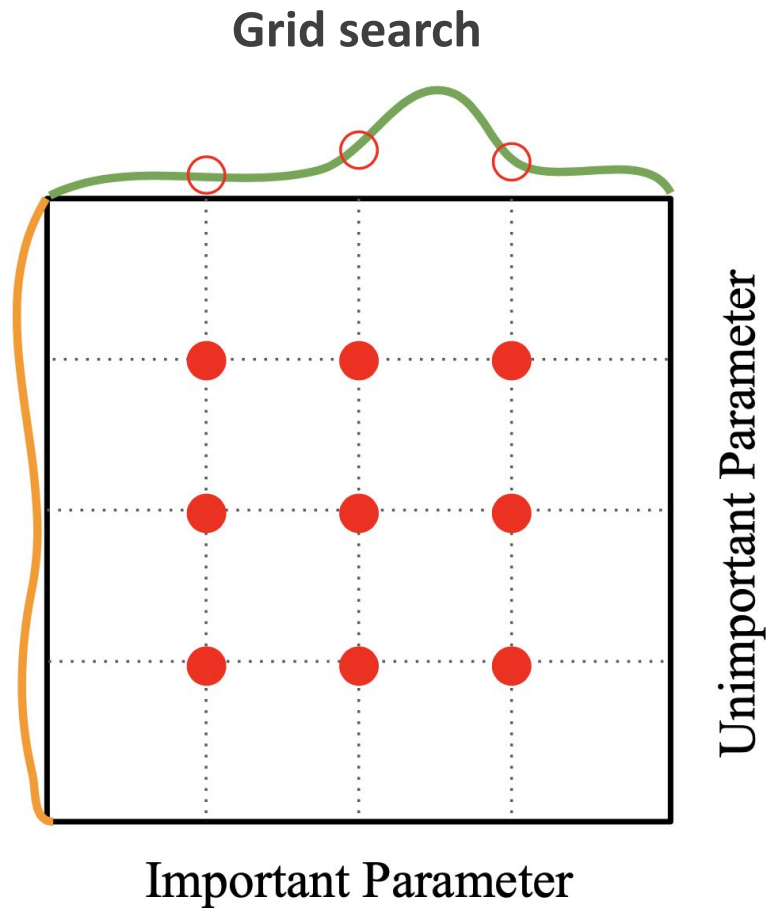
validation/metric
tag: validation/metric



Babysitting neural network training

- Step 0: Check the training data
- Step 1: Check initial loss
- Step 2: Overfit on a single minibatch / small sample
- Step 3: Find a learning rate that reduces loss significantly
- Step 4: Coarse grid, train a few epochs
- Step 5: Refine grid, train longer
- Step 6: Look at loss curves!
- **Step 7: Go back to step 5**

A note on hyperparameter search



Summary on training + hyperparameters

Improving training error

- Choosing optimizer
- Proper initialization
- Tweaking learning rate

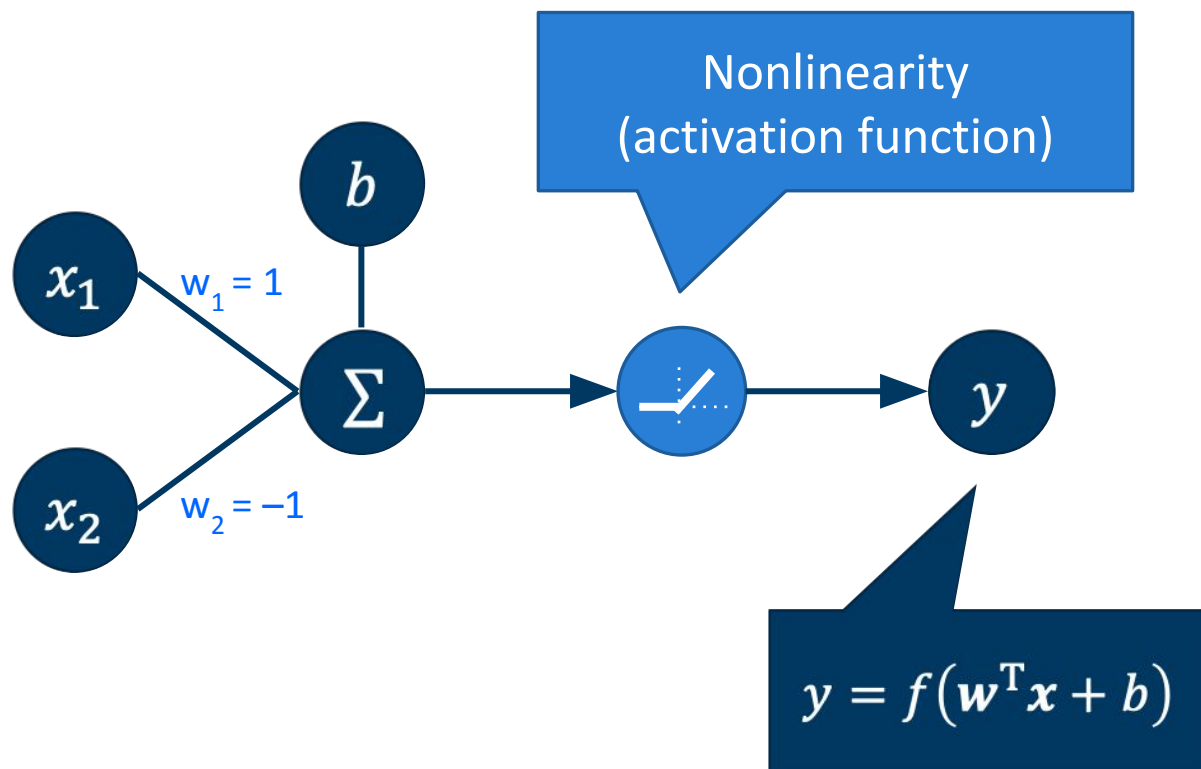
Improving validation/test error

- Architecture/model size
- Regularization
- Choosing hyperparameters

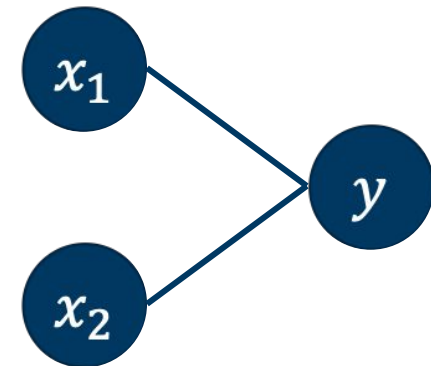
Architecture choices

Architecture choices: Activation functions

Perceptron / MLP hidden units



Simplified representation



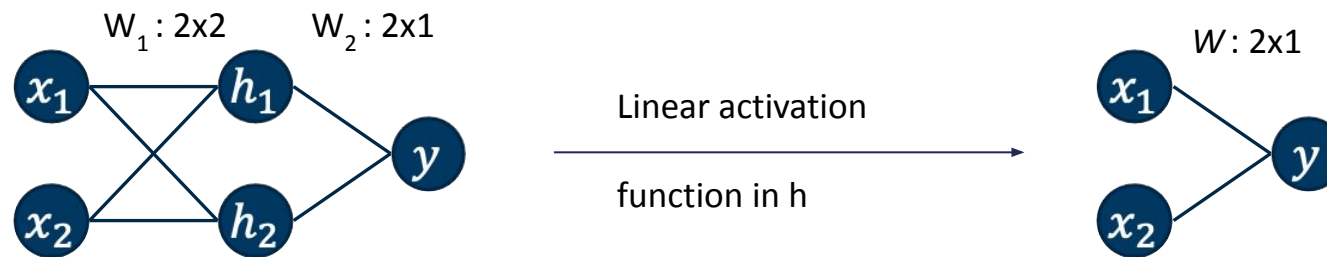
Linear combination and nonlinear activation function are implicit in this diagram

Requirements activation function

- Differentiable: need to take gradients for backprop
- Non-linear: multi-layer networks with linear activation functions “collapse” to a single layer

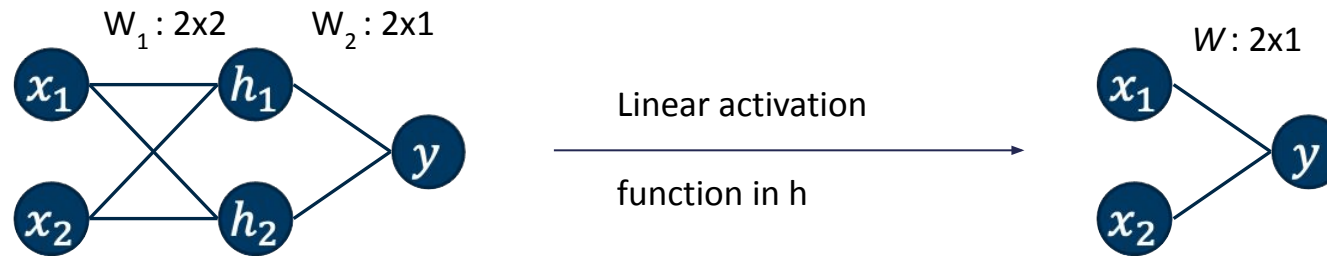
Requirements activation function

- Differentiable: need to take gradients for backprop
- Non-linear: multi-layer networks with linear activation functions “collapse” to a single layer



Requirements activation function

- Differentiable: need to take gradients for backprop
- Non-linear: multi-layer networks with linear activation functions “collapse” to a single layer

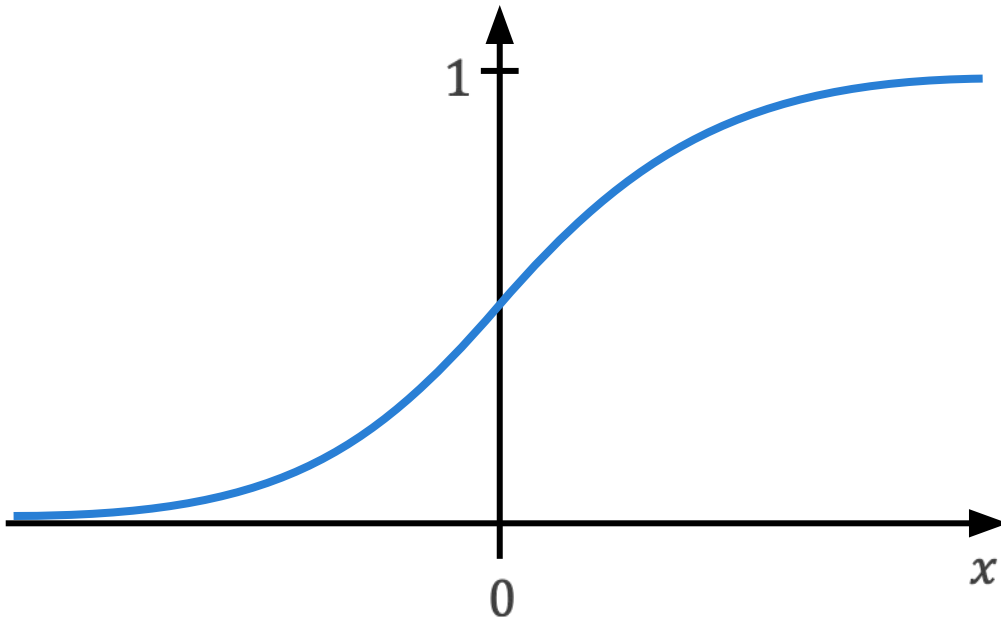


- Perceptron: threshold $\rightarrow$ not differentiable
- So far: sigmoid = soft threshold

Sigmoids

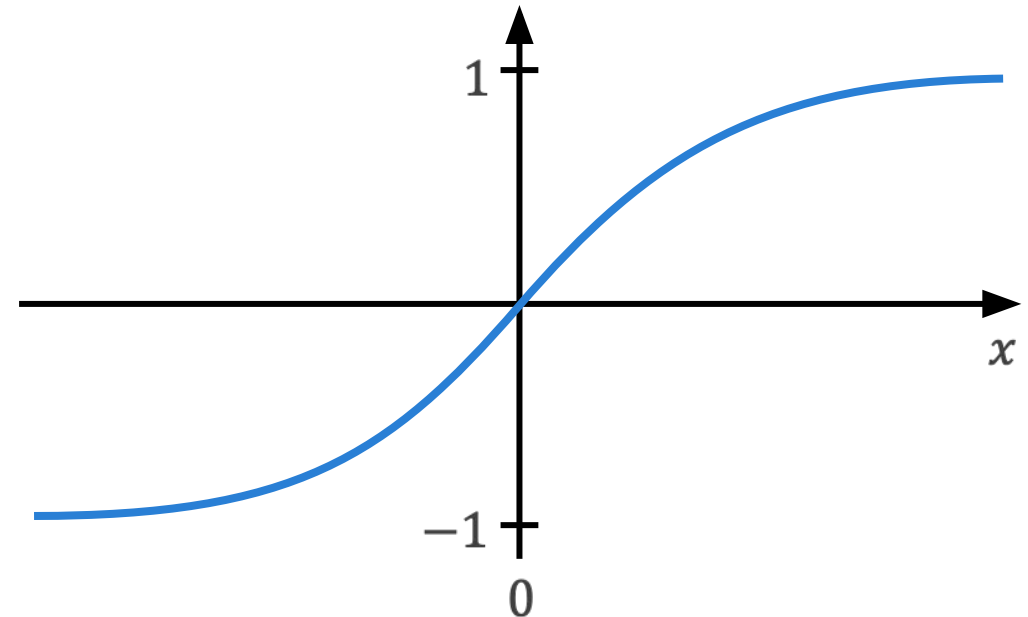
Logistic function

$$f(x) = \frac{1}{1 + \exp(-x)}$$

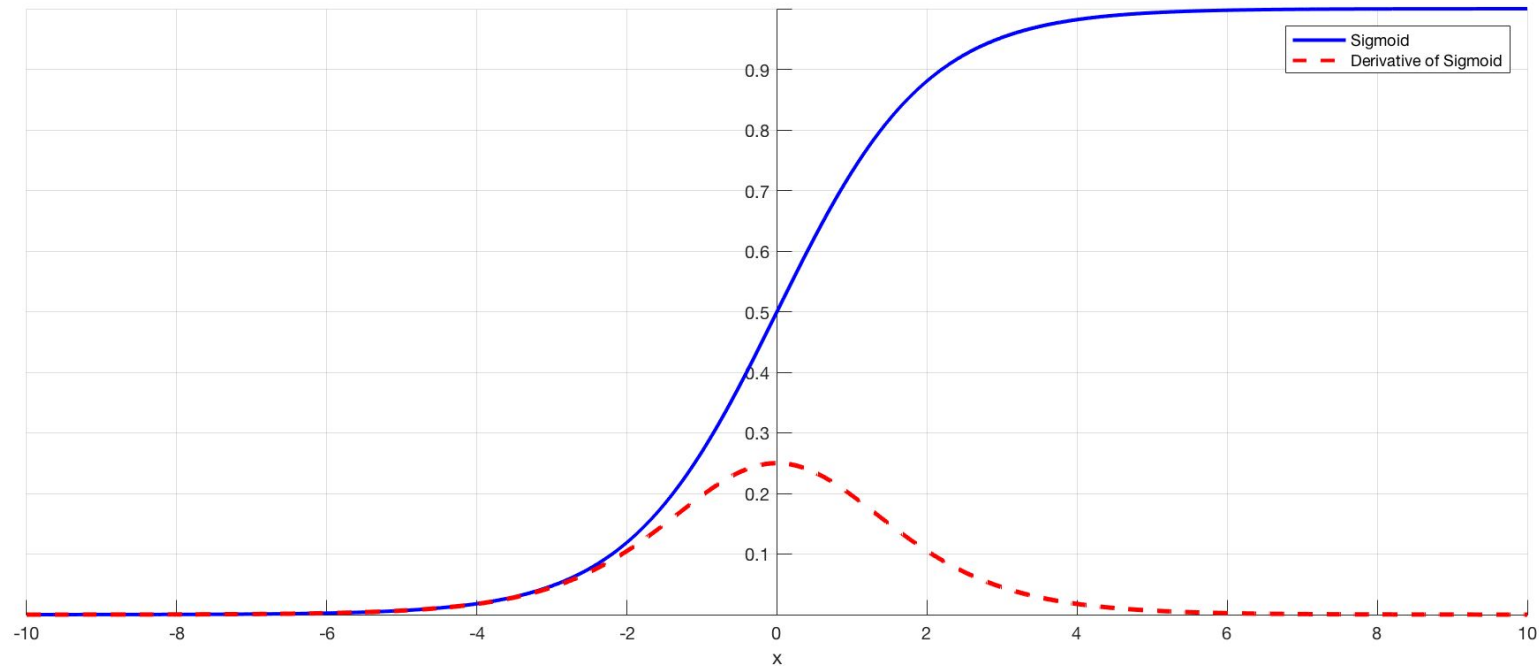


Hyperbolic tangent (tanh)

$$f(x) = \tanh(x) = \frac{\exp(2x) - 1}{\exp(2x) + 1}$$

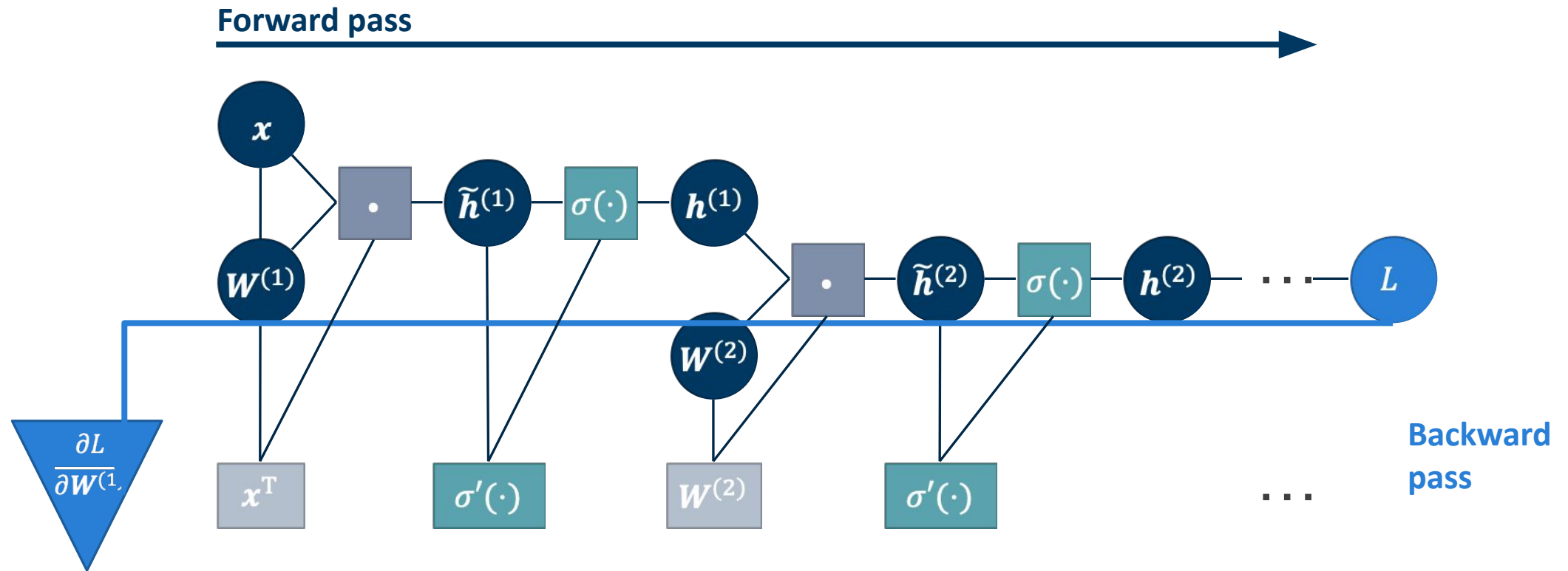


Sigmoid activation functions: vanishing gradients



Gradient close to zero for large positive or negative inputs

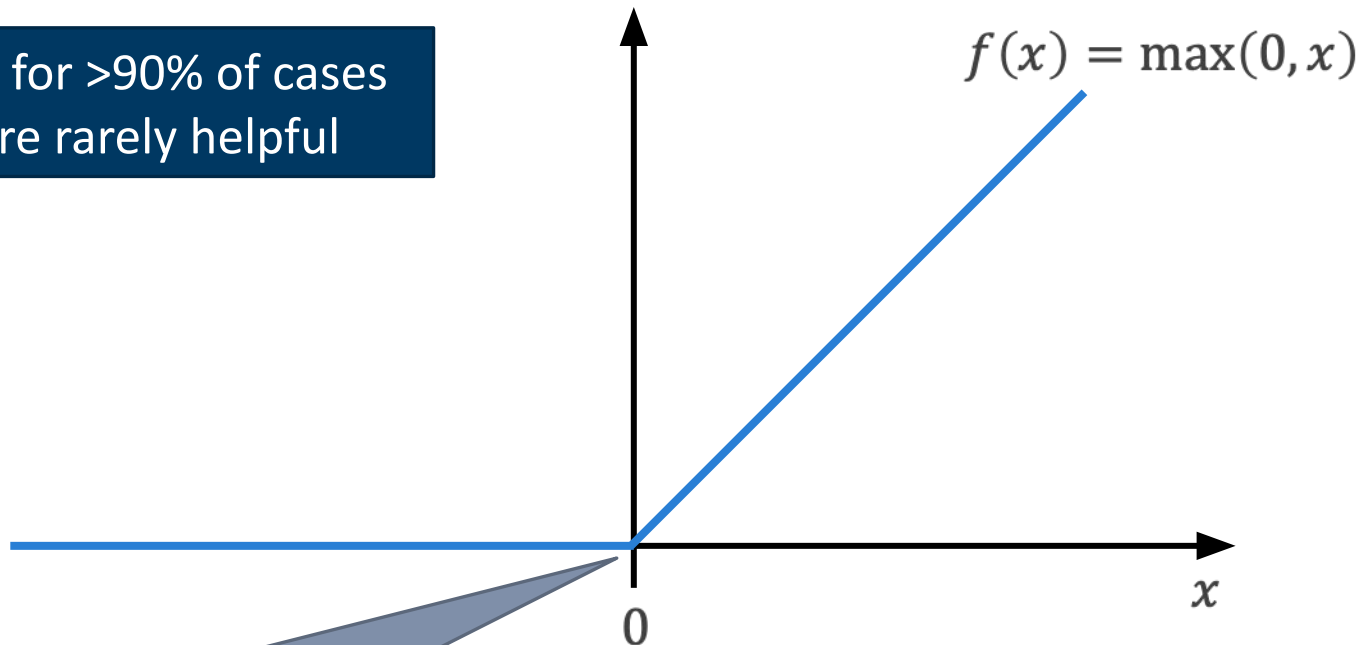
Stacking multiple layers with sigmoid activations



For deep networks: overall gradient very likely to be close to zero

Rectified linear unit (ReLU)

Standard choice for >90% of cases
Alternatives are rarely helpful



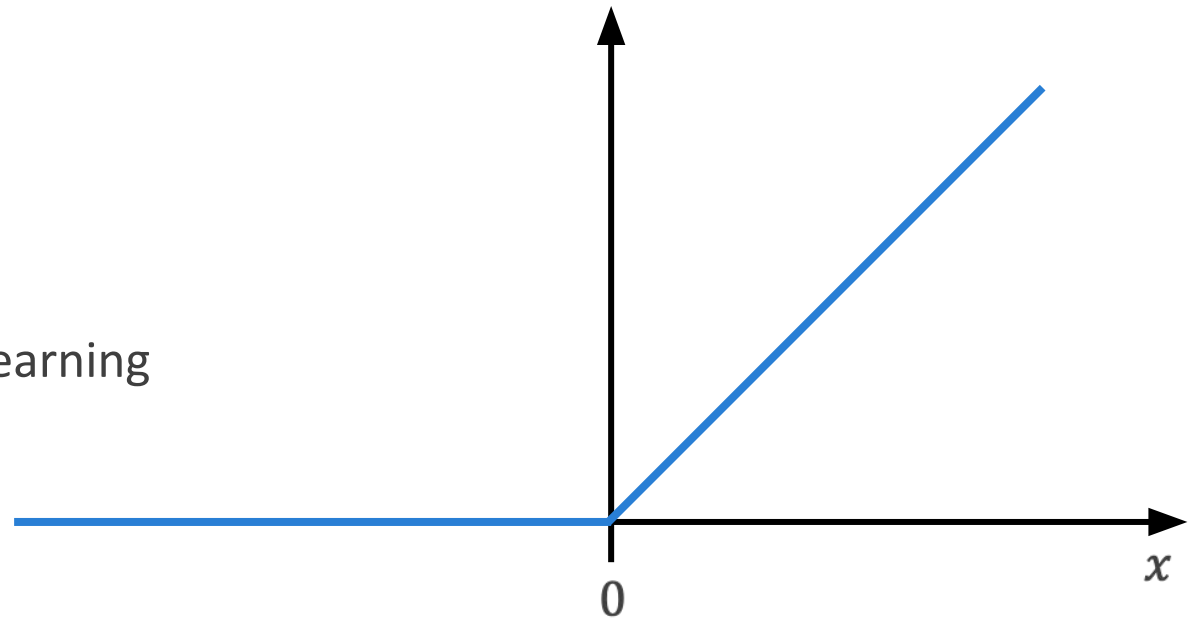
No gradient at zero -> not an issue: can be defined as zero and almost never happens

ReLU fixes vanishing gradient issue (somewhat)

Deep ReLU networks are locally linear
-> gradients well behaved

“Dying ReLU” problem:

all ReLU inputs smaller than zero for
-> no gradients & unit is “dead”
if happens to many units: no more learning



<https://towardsdatascience.com/the-dying-relu-problem-clearly-explained-42d0c54e0d24>

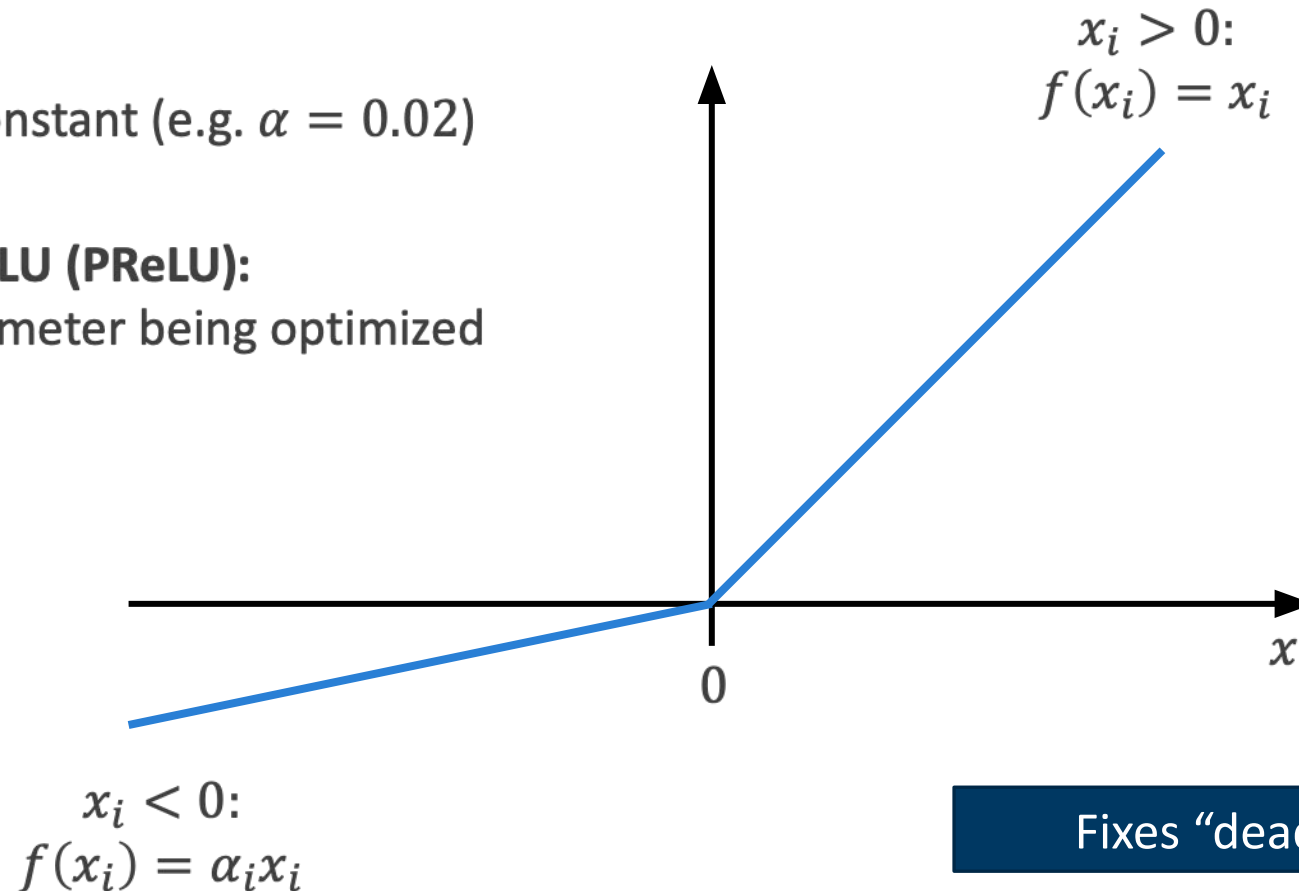
Leaky ReLU / Parametric ReLU

Leaky ReLU:

$\alpha_i = \alpha < 1$ constant (e.g. $\alpha = 0.02$)

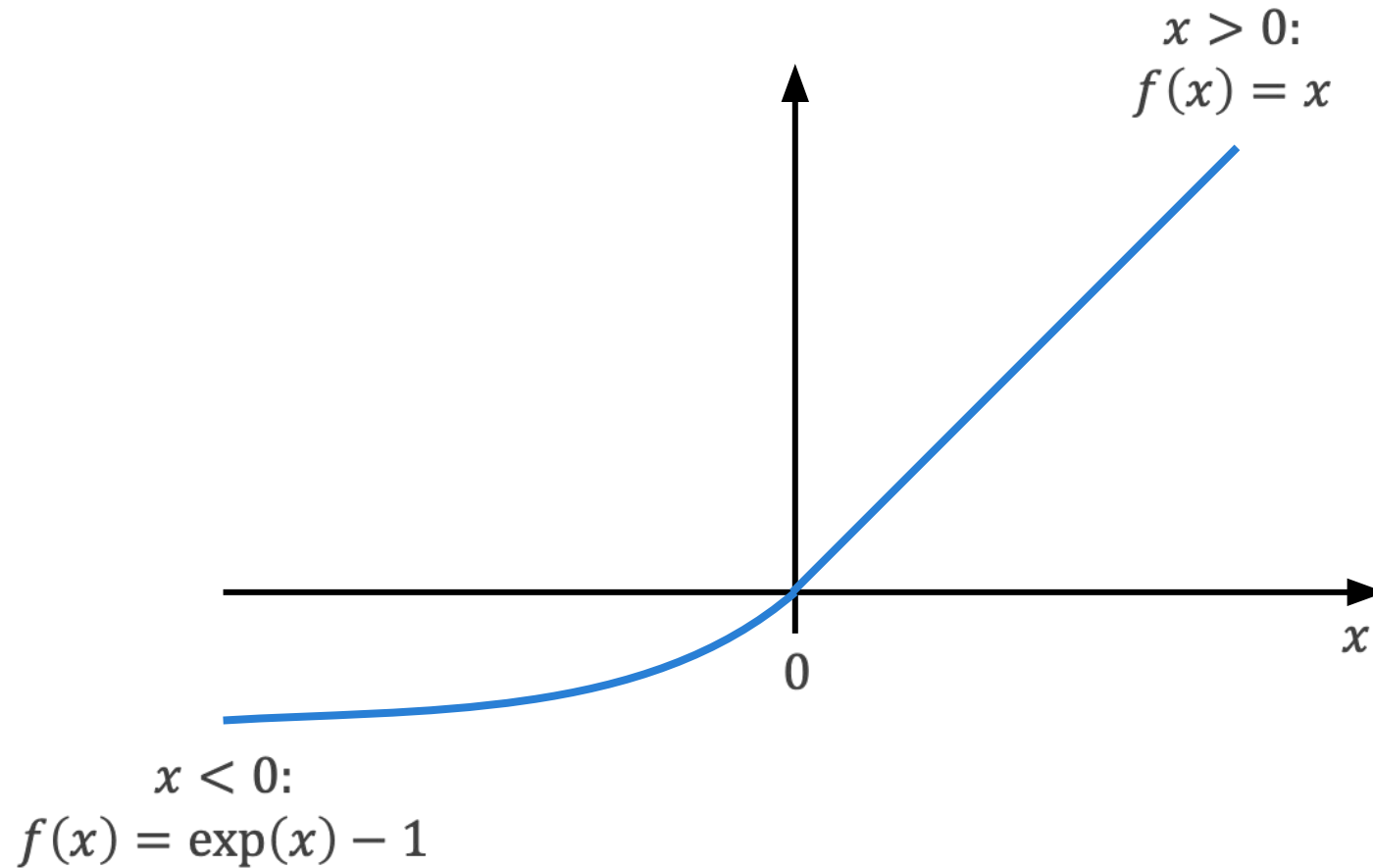
Parametric ReLU (PReLU):

α_i model parameter being optimized

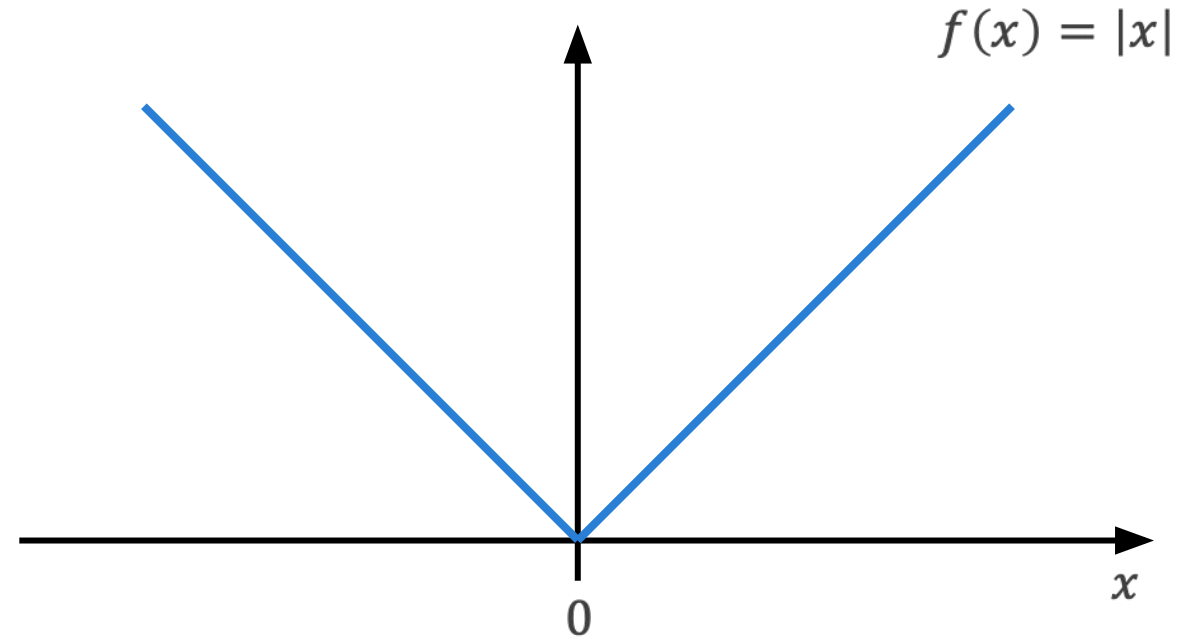


Fixes “dead ReLU” problem

Exponential linear unit (eLU)



Absolute value activation



Not really used in practice as it can be built from two ReLU units with inverted weights

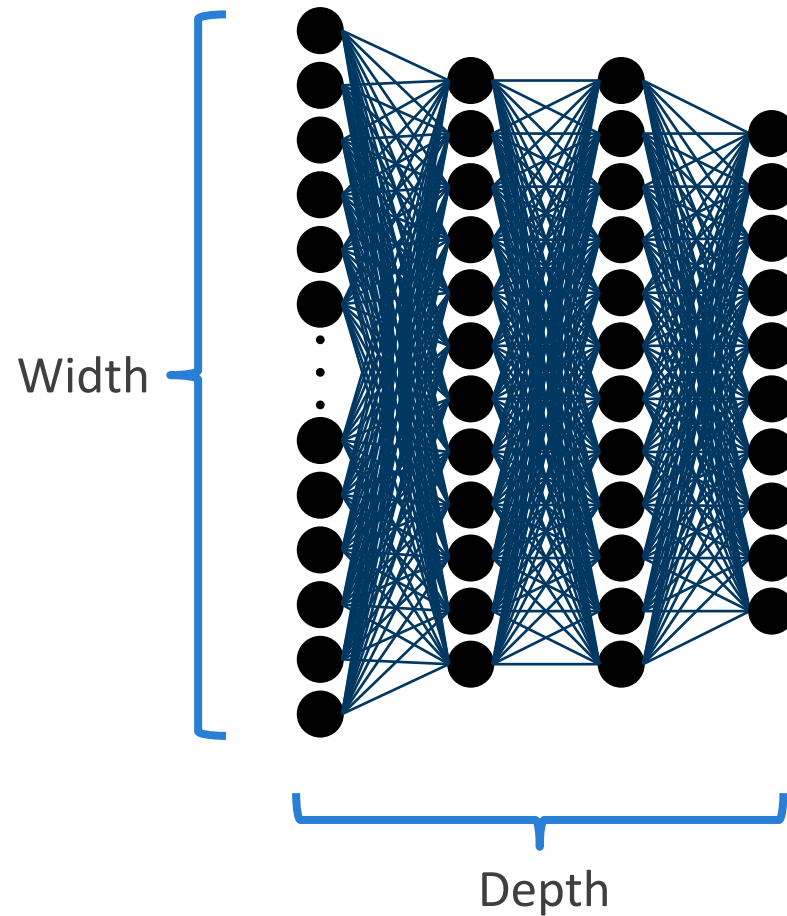
Weight Initialization

Other strategy to deal with vanishing gradients due to activation (e.g. Dead ReLU):
Weight initialization schemes to avoid this.

-> Will cover this next lecture

Architecture choices: Deep vs. wide networks

Architecture basics



Universal Approximation Theorem for one-hidden-layer neural networks

$$\phi : \mathbb{R} \rightarrow \mathbb{R}$$

continuous, bounded, non-constant activation function

$$f : [0,1]^M \rightarrow \mathbb{R}$$

arbitrary continuous function on unit hypercube

$$F(\mathbf{x}) = \sum_{i=1}^N v_i \phi(\mathbf{w}_i^T \mathbf{x} + b_i)$$

one-hidden-layer neural network of width N

Theorem

For any continuous function $f \in [0,1]^M \rightarrow \mathbb{R}$ and any $\epsilon > 0$
there exist
a finite integer N , real constants $v_i, b_i \in \mathbb{R}$ and real vectors $\mathbf{w}_i$
such that
 $|F(\mathbf{x}) - f(\mathbf{x})| < \epsilon$

Universal Approximation Theorem in words

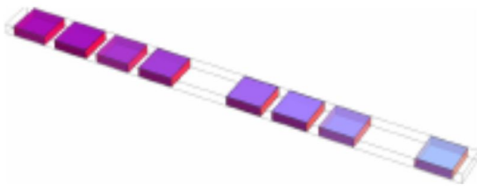
One hidden layer is enough to approximate any continuous function $f \in [0,1]^N \rightarrow \mathbb{R}$ to an arbitrary degree of accuracy

So why deeper?

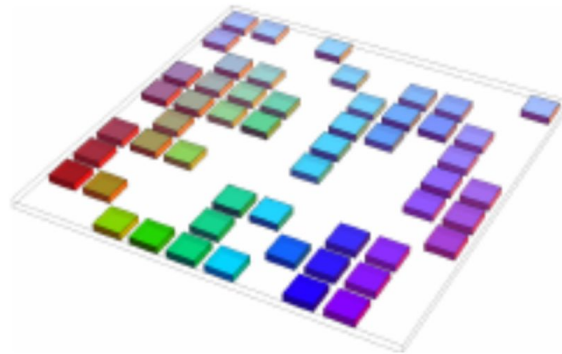
- Able to *represent* $\neq$ able to *learn*
- Shallow networks need exponentially more width
- Shallow networks overfit more

Curse of Dimensionality

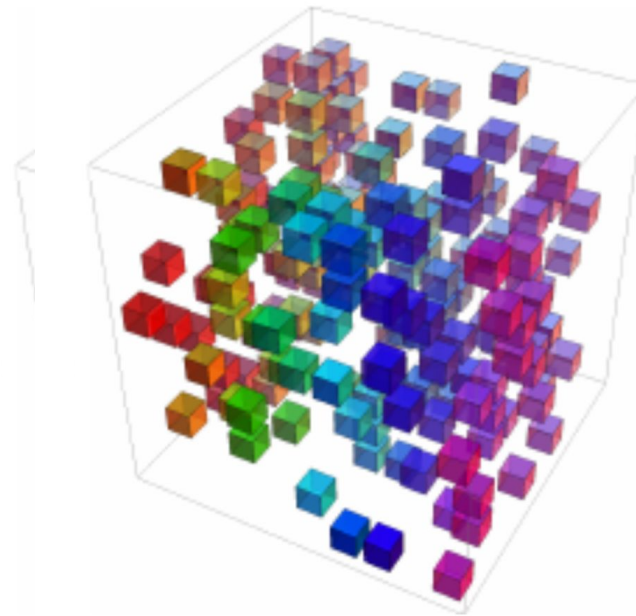
https://en.wikipedia.org/wiki/Curse_of_dimensionality



1D : $N = 10$



2D : $N = 100$



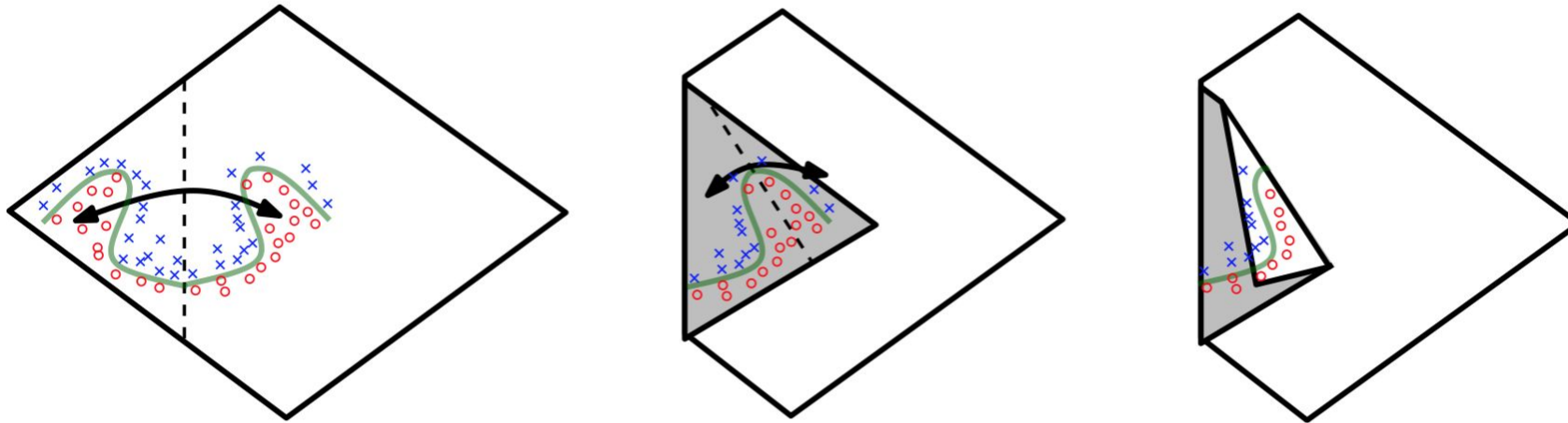
3D : $N = 1000$

...

k -D : $N = 10^k$

Exponential representation: advantage of depth

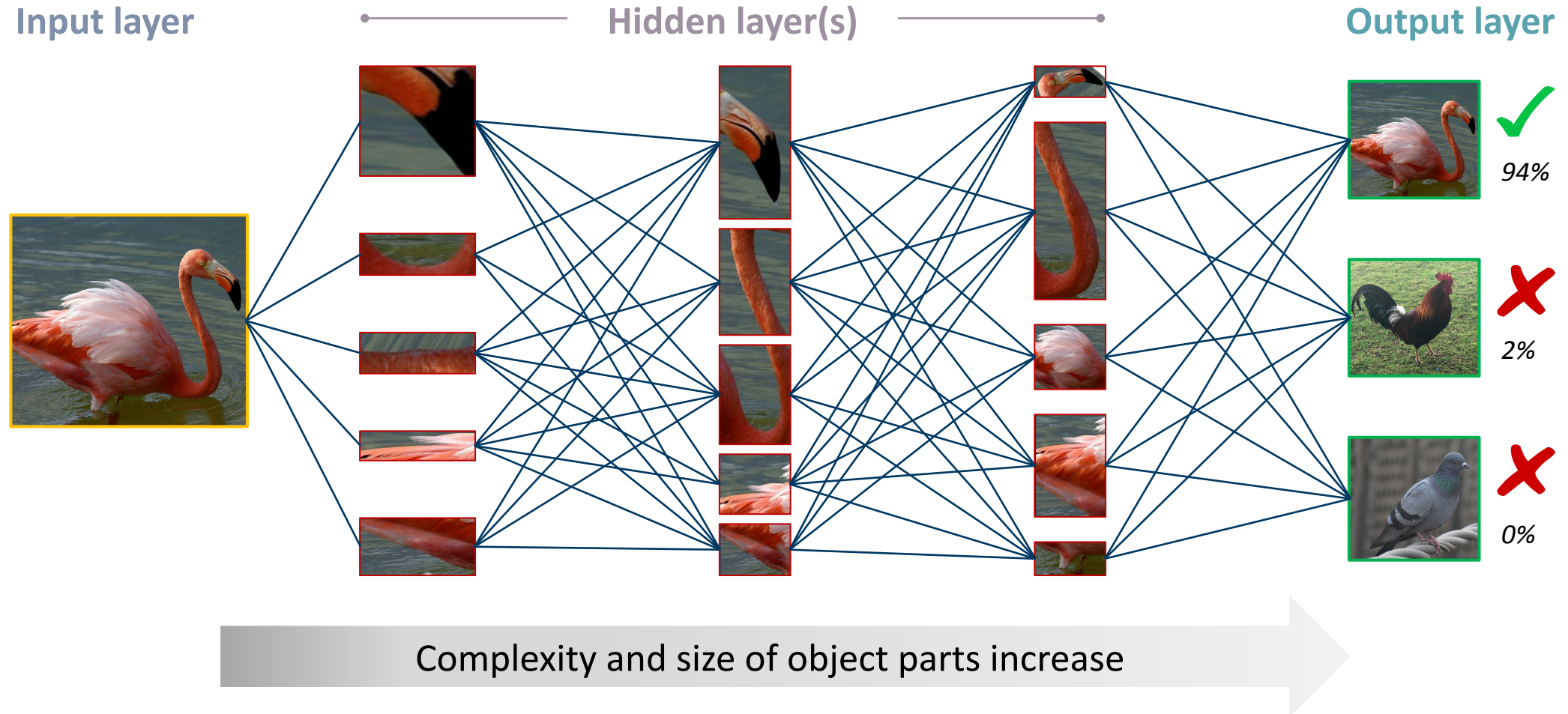
Absolute value activation function “folds” the space



Number of linear regions carved out by **deep rectifier network**
with d inputs, depth l , n hidden units per hidden layer:

$$O\left(\binom{n}{d}^{d(l-1)} n^d\right)$$

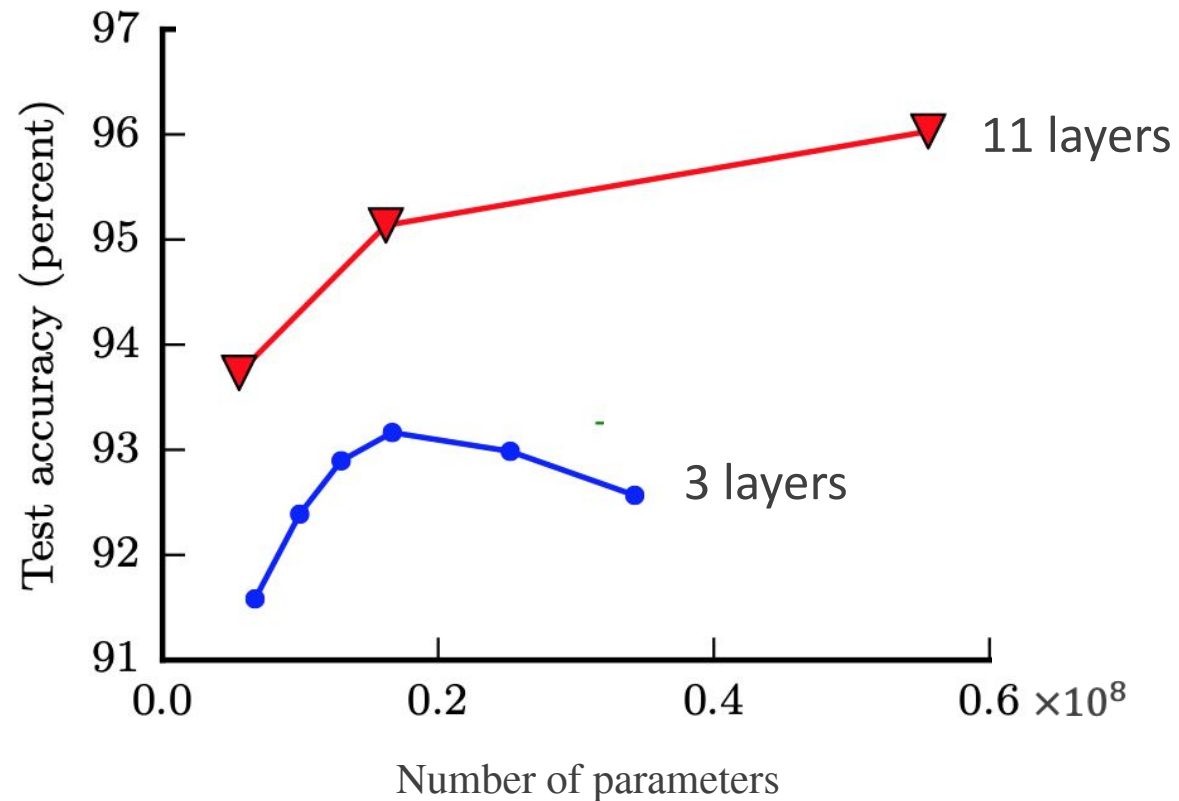
Neural networks can detect increasingly complex features throughout their layers



Large shallow models overfit more

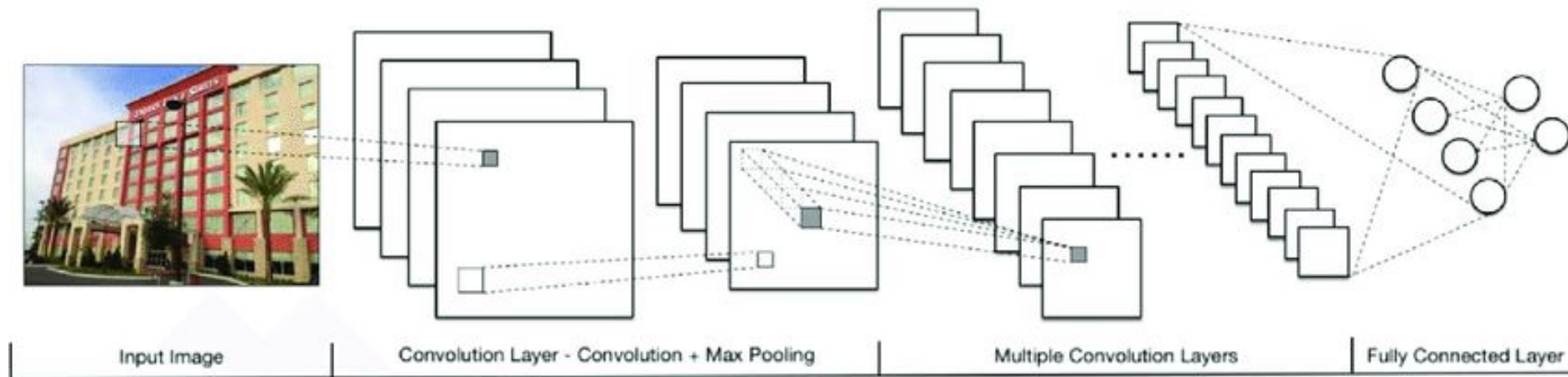
Task being solved:
House number
transcription from
Google Street View

Goodfellow et al., *ICLR* 2014

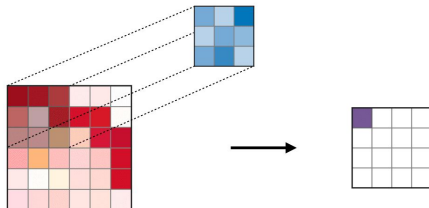


Architecture choices: architecture types

Neural network architectures: convolutional neural networks

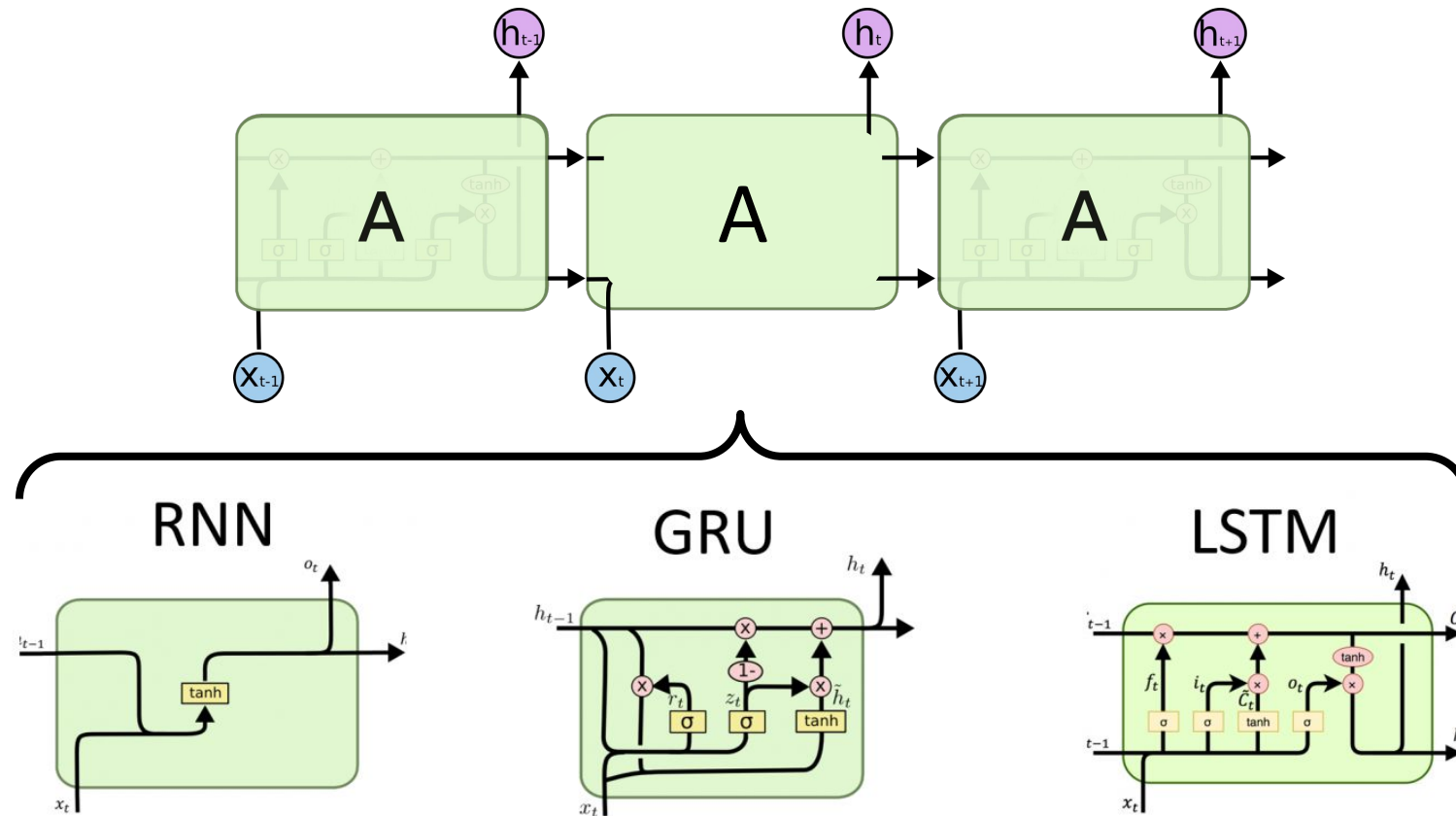


Convolution:



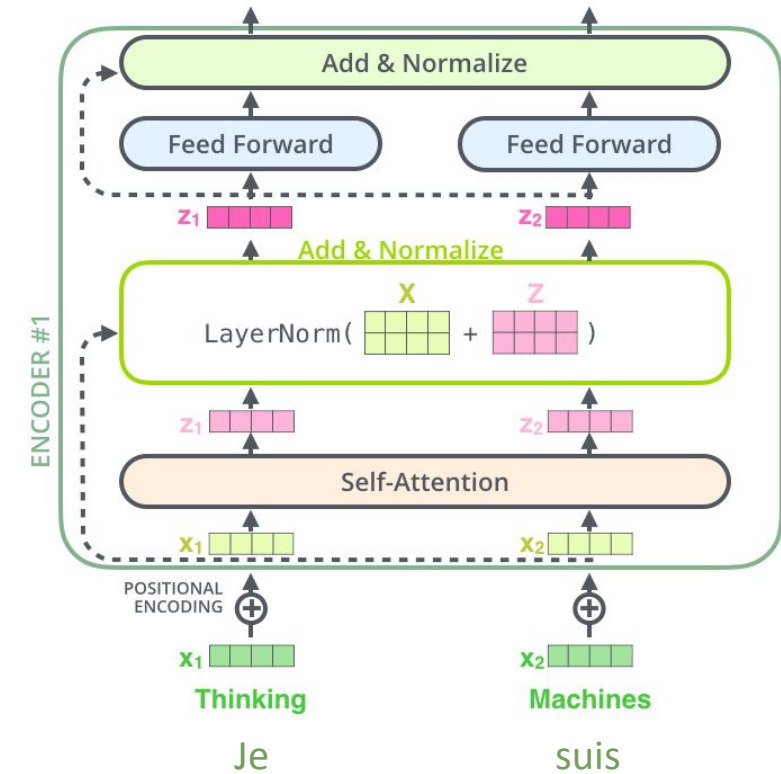
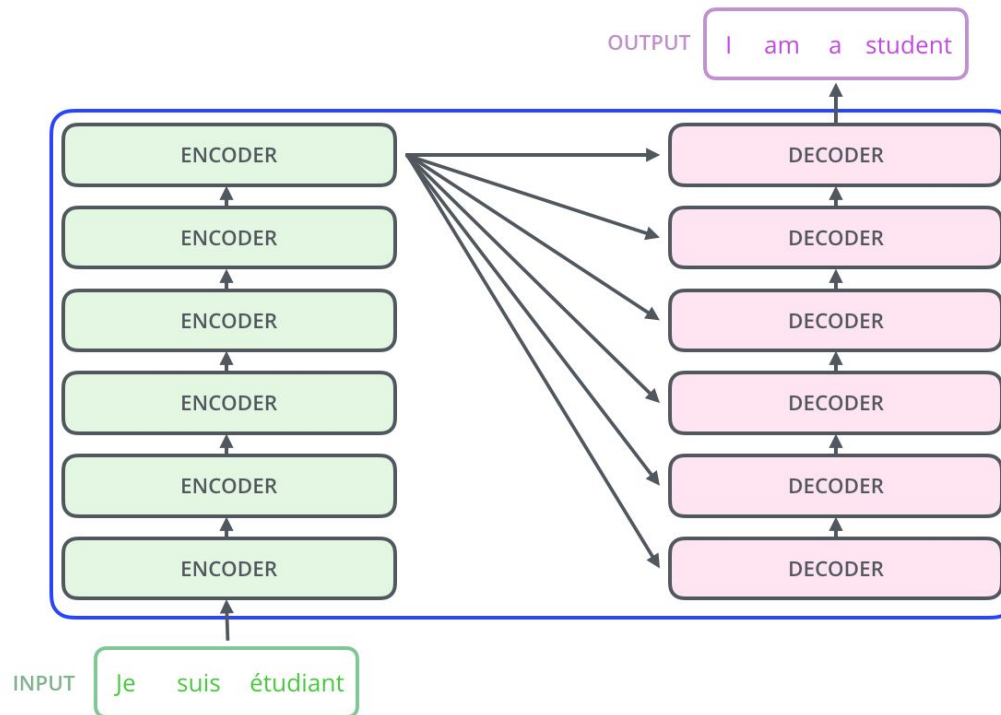
Used for image/video processing

Neural network architectures: recurrent nets



Traditionally used for sequence modeling (e.g. natural language processing)

Neural network architectures: Transformers



Mostly replaced RNNs for natural language processing since early 2019
(and are currently also getting more popular in computer vision)

When to use deep learning?

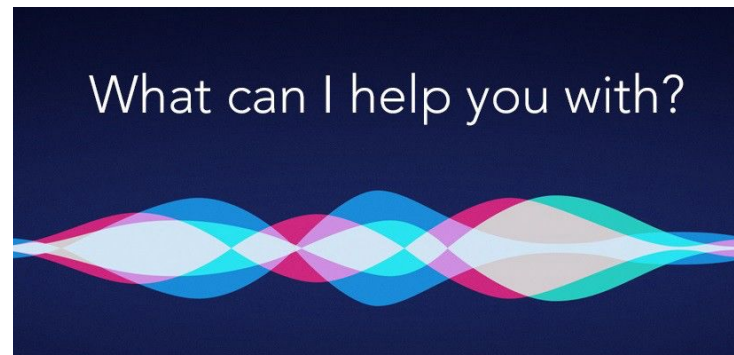
Problems amenable to deep learning are:

- low-noise
- highly nonlinear
- unstructured data

Examples



Image recognition



Natural language processing

...

Today's lecture was a lot...

Because you should know most of this from the ML lecture!

- Next lectures will be less material and in more detail.
- If you don't know what we covered today, check out Recap 2 and the materials referenced in there.

Questions?
